COMP11212 Fundamentals of Computation
Part 2: Understanding Computation

Andrea Schalk
<andrea.schalk@manchester.ac.uk>

Department of Computer Science
University of Manchester

March 2025

Contents

Contents i

Organisation 1

Introduction 3

1 The While Language 6
- 1.1 Variables and numerals . . . 6
- 1.2 Arithmetic expressions . . . 6
- 1.3 Boolean expressions . . . 7
- 1.4 Statements . . . 7
- 1.5 The syntax of While . . . 8
- 1.6 Input and output . . . 9
- 1.7 Example programs . . . 9
- 1.8 Summary . . . 13
- 1.9 Solutions to Exercises . . . 14

2 Computing with While programs 18
- 2.1 States . . . 18
  - 2.1.1 Describing states . . . 19
  - 2.1.2 Manipulating states . . . 19
- 2.2 Extended example: Computation via updating states . . . 20
- 2.3 Computation via updating states step by step . . . 21
  - 2.3.1 Arithmetic expressions . . . 21
  - 2.3.2 Boolean expressions . . . 22
  - 2.3.3 Statements . . . 22
  - 2.3.4 The stepwise operational semantics of While . . . 24
- 2.4 Example: A formal stepwise computation . . . 24
- 2.5 Non-termination . . . 26
- 2.6 Following an execution . . . 28
- 2.7 Computing final states . . . 29
  - 2.7.1 The big step operational semantics of While . . . 30
- 2.8 Example: A formal big step computation . . . 32
- 2.9 Moving beyond individual states . . . 35
- 2.10 Summary . . . 36
- 2.11 Solutions to Exercises . . . 37

3 Complexity and Asymptotic Analysis 53
- 3.1 General ideas . . . 53
  - 3.1.1 Time and space complexity . . . 53
  - 3.1.2 Counting steps . . . 54
  - 3.1.3 Best, worst, average case . . . 54
  - 3.1.4 Examples . . . 54

3.2 Asymptotic analysis—rates of growth . . . 56
3.2.1 Basic assumptions . . . 56
3.2.2 Comparing functions . . . 57
3.2.3 Investigating $O(-)$ . . . 58
3.2.4 Notation for $O(-)$ . . . 59
3.3 Properties of $O(-)$ . . . 59
3.4 Complexity Classes . . . 61
3.4.1 Logarithmic growth . . . 61
3.4.2 Polynomial growth and eyond . . . 62
3.5 Determining complexity classes: examples . . . 63
3.6 Complexity of problems versus the complexity of algorithms . . . 65
3.6.1 Complexity of algorithms . . . 65
3.6.2 Complexity of problems versus the complexity of algorithms . . . 66
3.7 Summary . . . 67
3.8 Solutions to Exercises . . . 68

4 Proving While Programs Correct **74**
4.1 Criteria for programs . . . 74
4.2 Example: Using predicates . . . 75
4.3 Partial and total correctness . . . 77
4.4 A system for partial correctness . . . 77
4.4.1 Generalized arithmetic and boolean expressions . . . 77
4.4.2 Statements . . . 78
4.4.3 Loop invariants . . . 79
4.4.4 Chaining assignment rules . . . 79
4.4.5 A system for partial correctness . . . 80
4.5 Extended example: establishing partial correctness . . . 80
4.6 A system for total correctness . . . 82
4.6.1 A system for total correctness . . . 83
4.6.2 Loop variants . . . 83
4.6.3 Extended example: establishing total correctness . . . 83
4.7 Further examples . . . 84
4.7.1 If statements . . . 85
4.7.2 Greatest common divisor . . . 85
4.7.3 Solving a quadratic equation . . . 88
4.7.4 The Collatz conjecture . . . 90
4.8 Establishing properties using Hoare triples . . . 90
4.8.1 Which Hoare triple to aim for? . . . 90
4.8.2 Loop invariants . . . 91
4.8.3 Loop variants . . . 91
4.9 Properties of our systems . . . 94
4.10 Summary . . . 94
4.11 Solutions to Exercises . . . 95

5 Computability **104**
5.1 Computable Functions . . . 105
5.1.1 Computable functions . . . 105
5.1.2 Computability for partial functions . . . 107
5.1.3 Generalizing the notion of computability . . . 108
5.2 There are uncomputable functions . . . 110
5.3 Decidability . . . 110
5.3.1 Decidable properties . . . 110
5.3.2 Combining predicates . . . 112

5.3.3 Semidecidable predicates . . . 113
5.4 The Halting Problem . . . 115
5.5 Universality of Computation . . . 117
5.5.1 The universal partial function . . . 120
5.5.2 The universal partial function is computable . . . 121
5.5.3 The Parameterization Theorem . . . 121
5.5.4 Functions of several variables . . . 123
5.6 The Church-Turing Thesis . . . 123
5.7 Summary . . . 124
5.8 Solutions to Exercises . . . 125

6 Mathematical notions and results **136**
6.1 The size of the set of functions from $\mathbb{N}$ to $\mathbb{N}$ . . . 136
6.2 Encoding data structures as natural numbers . . . 137
6.2.1 Integers . . . 138
6.2.2 Pairs of natural numbers . . . 138
6.2.3 More complex structures . . . 140
6.2.4 Encoding operations . . . 140
6.3 Encoding programs . . . 141
6.4 Computing the stepwise operational semantics . . . 143
6.5 Summary . . . 144
6.6 Solutions to Exercises . . . 145

A Mathematical background **148**
A.1 Numbers . . . 148
A.2 Functions . . . 149
A.3 Partial functions . . . 150
A.4 Binary relations . . . 151
A.5 Countable and uncountable sets . . . 152

About Part 2 of the course unit

Part 2 of COMP11212 *Fundamentals of Computation* looks at ways of understanding computation.

On completing this part of the units students will be able to

- formally investigate the behaviour of a program using the tools of operational semantics and axiomatic systems;
- define the notion of asymptotic complexity based on $O(-)$ and the corresponding complexity classes and evaluate the complexity of example programs and
- understand fundamental ideas about what is computable, and be able to explain why some questions cannot be solved with programs.

The methodology of learning is the same as for Part 1: These notes provide the foundation for the material; there's some support on Blackboard in the form of videos and some quizzes.

We strongly encourage students to use the weekly exercise sheets to ensure they obtain a good working understanding of the material, and to discuss their answers with GTAs in the example classes to address potential misconceptions and to deepen their understanding.

For 2024–25 there had to be a change of staff at fairly short notice, and we expect to further develop the available resources—for the current academic year the main effort has gone into re-writing the notes, with a particular emphasis on providing more examples, exercises, and solutions to those exercises. Part 2 is taught by Joe Razavi and Andrea Schalk.

The notes cover all the examinable material and there is no need to read anything else. Students who would like to see (some of) the material presented here from other sources are referred to the following:

M. Sipser. **Introduction to the Theory of Computation.** *PWS Publishing Company*, 1997. ISBN 0-534-94728-X.

This book covers the topics of decidability (see Chapter 5), but it uses a different abstract model of computation and it concentrates on examples from the theory of formal languages (as studied in Part 1). It also discusses time complexity (see Chapter 3) and introduces $O(-)$. It goes significantly beyond the content we teach here for those topics.

J.E. Hopcroft, R. Motwani, and J.D. Ullman. **Introduction to Automata Theory, Languages, and Computation.** *Addison Wesley*, second edition 2001. ISBN 0-201-44124-1.

This book covers the topic of undecidability (see Chapter 5), but it does so using a different abstract model of computation.

M. Huth and M. Ryan (2004). **Logic in Computer Science: Modelling and Reasoning About Systems**. *Cambridge University Press*, second edition 2004. ISBN 9780521543101.

This book looks at an axiomatic system such as the one we introduce in Chapter 4.

H. Nielson and F. Nielson. **Semantics with Applications: An Appetizer**. *Springer*, 2007. ISBN 1846286913.

This book has quite a bit of overlap with what we teach—it does use the While language (see Chapter 1), provides a small step and a big step operational semantics (see Chapter 2), and it also provides axiomatic systems (see Chapter 4) for these.

Acknowledgement. This part of the unit was originally designed by Dave Lester who wrote the first set of notes. These were variously adapted by Giles Reger and Gareth Henshall. They have been completely rewritten by Andrea Schalk. The structure and the order of the material has changed significantly, as have some of the systems described. What remains from the previous versions are the selection of topics, and some examples and exercises.

Introduction

In this part of the unit we aim for an understanding of what we mean by computation, how we may be able to make rigorous statements about programs, and what it means to be computable.

These topics are developed further in subsequent units such as COMP26120 *Algorithms and Data Structures*, COMP36111 *Advanced Algorithms and Complexity* and COMP31311 *Giving Meaning to Programs*.

If we want to use programs that run on real world computing devices we are limited to observations one run at a time. This raises questions about what happens if we run the same program again on the same computer, or if we run it on a different piece of hardware.

Abstract models of computation

This is why in order to gain an understanding of computation computer scientists look at *abstract notions of computation*. This requires setting up some kind of abstract device that allows a notion of carrying out a computation, so it's necessary to define what it means to *run a computation*.

The advantage of using an abstract notion is that we're not tied to individual circumstances (such as a real world computation device and it's potentially changing behaviour depending on outside factors such as the operating system or the load on the processor) that are hard to control for, and that we are able to make precise statements about these, and we're able at least in principle to rigorously prove our statements.

The second half of this course unit may be seen as a case study in a particular abstract notion of computation, introducing along the way concepts such as complexity and computability that have meaning beyond the particular notion chosen.

Students have already seen one abstract notion of computation in the form of finite state automata in the first half of this course unit: We may see such an automaton as an abstract computation device that takes as input a word and produces a yes/no answer. This then led us to consider derived notions such as the language defined by an automaton, and we were able to prove results, for example that every language recognized by a finite state automaton is regular.

As a notion of computation this is rather weak, and doesn't seem to have much in common with real world programming languages. For that reason we look at a different abstract notion in this part of the unit.

A number of such notions appear in the literature and we mention a few of them.

Turing machines. This is a famous abstract notion of computation named after Alan Turing who came up with the idea. He (in the same year as Alonzo Church) proved that there are mathematical problems with a yes/no answer which cannot be computed by any computer, and he used these machines for the proof.[1] Turing came up with his notion for this machine very carefully reasoning about what one might mean by 'computation', and students interested in this question are encouraged to read that part of the paper.

The λ-calculus. In the same year as Turing published his result described in the previous paragraph, Alonzo Church published his solution, using a formalism known as the λ-calculus.[2]

[1] See Alan Turing, *On Computable Numbers, with an Application to the Entscheidungsproblem*, Proceedings of the London Mathematical Society, Series 2, 42 (1936–7), pp 230–265. An online version is available at http://www.abelard.org/turpap2/tp2-ie.asp.

[2] See Alonzo Church, *A note on the Entscheidungsproblem*, Journal of Symbolic Logic, 1 (1936), pp 40-41 and

The λ-calculus appears as a stepping stone towards an abstract functional model of computation in COMP31311 *Giving Meaning to Programs*.

There a many more such models, *register machines* (which exist in several incarnations) and *cellular automata* being examples that interested students may want to have a look at.

The model we present here has commonality with register machines: Register machines are built on an abstract notion of data storage in the form of 'registers' , together with programming instructions that make changes to the values held in the registers.

Our model uses programming instructions such as while loops and conditional statements of the form **`if`** … **`then`** … **`else`** … , and it has support for boolean and arithmetic expressions. This makes it more similar to a real world programming language and it allows students to apply their intuition gained from programming to our abstract model.

Typically theoreticians like their abstract models to be *lean*, that is in this case, to have the smallest possible instruction sets and the fewest data types. The reason for this is that when proving properties of all programs, one typically has to carry out a proof by induction over all cases, so reducing the number of cases makes the proofs less laborious and shorter.

Programmers, on the other hand, would like their languages to make it easy to write their code, and for those purposes, having additional data structures and operations is useful. The model we look at here is something of a compromise in that it is not really as lean as a theoretician might want it to be while also not being very comfortable to program in.

Running a computation

Having described our language we turn to the question of what it should mean to run a computation for this particular setting. For an imperative language, a standard way to do so is to think of the computation proceeding step by step, where each step is concerned with comparing some values or manipulating what is stored in the variables, and we can monitor a computation by looking at how the values in the variables change over time. We introduce an *operational semantics* in Chapter 2 which formalizes this idea. It has the advantage that we do not need any additional infrastructure to describe it: Just a program, how the values of some variable change when we carry out a computation step, and the program that still has to be executed after that step. This is called a *small step semantics* because we follow each step—we also formally define a *big step semantics* which tries to jump straight to where we are when the program execution is complete.

How many steps?

A natural next question to ask is how many steps a computation takes before arriving at its final answer. As computer scientists, we are typically not interested in the precise number of steps required for particular inputs, but we want to understand how the number of steps grows as our input becomes bigger and bigger. This is known as *asymptotic analysis* and lays the foundation for the notion of (time) *complexity*, which we look at in Chapter 3. There we look at the formal definition of complexity classes in the $O(-)$ system, and practice determining the complexity of example programs.

Does it do what it should?

We then turn to the question of whether our abstract notion of computation allows us to prove that a particular program carries out the calculation we were aiming for in Chapter 4. This requires us to be able to state unambiguously what we were expecting to compute. For that we allow ourselves the use of logic, plus mathematics, without providing a specific syntax, but instead relying on our common understanding of these ideas as taught on other units. Our system then allows us to establish properties such as 'if the variable `n` held the value n at the start of the program `S` then

Alonzo Church, *An unsolvable problem of elementary number theory*, American Journal of Mathematics, 58 (1936), pp 345–363.

once the program has been executed, the variable x holds the value $n \operatorname{div} 2$.' Establishing such properties rigorously is challenging since we have to consider the behaviour of the program in a variety of situations, but it does allow us to ensure that we do

What can we hope to compute?

This may seem a very general question at first sight. But as we explain in Chapters 5 and 6 it turns out that it is sufficient to focus on the question which functions from $\mathbb{N}$ to $\mathbb{N}$ we are able to compute. Our first result is that there certainly are (lots of) functions that we cannot hope to compute, but without providing an example of an uncomputable function. We do provide an example of such a function in the form of the *Halting Problem*. We also address the question how general our considerations are, given that we look at just one abstract model of computation, and the answer is that this does not impose any restrictions on our ability to compute.

Chapter 1

The While Language

In this chapter we introduce part of our abstract notion of computation, namely the syntax for the programming instructions, plus we discuss the notion of data storage.

The programming languages Python and Java students have met in the two programming course units are *imperative* languages. These language have a notion of the current *state*, and that state is changed by programming instructions. The abstract notion of computation we introduce here follows the same principles.

Readers who prefer to see the syntax of the whole language before considering the constituent parts are encouraged to begin by looking at Section 1.5, and then revisiting the sections on variables, boolean and arithmetic expressions.

1.1 Variables and numerals

Our language has a notion of **variable** which we use to store data in the form of integers. We assume that there's a predefined **set of variables** Vars. While a given program can only mention a finite number of variable names we want to make sure we can never run out of new variables, so we assume that this set is infinite.

We are not concerned with how variable names are constructed, but for readability reasons we use letters from the alphabet to name them—however, this does *not* mean that we assume there are only 26 variables! One could instead assume that we have a variable x_i for each $i \in \mathbb{N}$, but that produces code that is less readable. In our language we assume that there is infrastructure that generates, and in the grammar below we use '`var`' for variables.

We assume that we have a notion of **numeral** that can be stored in our variables, and in our grammar we use '`num`' for these.

There is a choice here regarding which numerals one may want to allow. In programming languages it is customary to have *integers* while traditional material on computability is concerned with *natural numbers*. The former choice has the advantage that we don't have to worry about generating errors when forming expressions such as $4 - 5$, which are not defined in the natural numbers. For the time being, this is the choice we adopt.

Further, any real world programming language can only represent numbers up to some maximal and minimal values. This the requires dealing with *overflow* errors, when encountering an expression that goes above the largest (or below the smallest) value possible. Since our language is for an abstract notion of computation we avoid this issue by assuming that all values in $\mathbb{Z}$ have corresponding numerals.

1.2 Arithmetic expressions

In order for us to be able to compute something non-trivial we need some kind of infrastructure. We want to compute with numbers, and for that we have **arithmetic expressions**, that allow us to perform basic operations for these.

The syntax for arithmetic expressions, in Backus-Naur form (see Section 4.6 of the notes for Part 1), is

$$\begin{aligned}\langle \text{aexp} \rangle \quad ::= \; & \langle \text{num} \rangle \mid \langle \text{var} \rangle \mid \\ & \langle \text{aexp} \rangle + \langle \text{aexp} \rangle \mid \langle \text{aexp} \rangle - \langle \text{aexp} \rangle \mid \langle \text{aexp} \rangle \times \langle \text{aexp} \rangle\end{aligned}$$

In other words, an arithmetic expressions may be given by a variable, a numeral (representing an integer) from some set $\mathtt{Num}$, or the sum, the difference, or the product of two arithmetic expressions.

Note that strictly speaking we should always use brackets around arithmetic expressions as we create them, but we'll assume that we know, for example, how to evaluate $3 \times 5 + 4$ without explicitly bracketing $(3 \times 5) + 4$. We do allow ourselves to use brackets as needed.

The grammar above generates the set AExp of arithmetic expressions. We may think of this as being defined recursively, with the two base cases being given by integers and variables, and the step cases arising from adding, subtracting or multiplying two arithmetic expressions.

1.3 Boolean expressions

Our language has boolean expressions that allow us to formulate conditional statements of the form: If a boolean expression evaluates to true then something happens, else something happens.

The syntax for **boolean expressions**, in Backus-Naur form, is

$$\begin{aligned}\langle \text{bexp} \rangle \quad ::= \; & \mathtt{ff} \mid \mathtt{tt} \mid \langle \text{aexp} \rangle = \langle \text{aexp} \rangle \mid \langle \text{aexp} \rangle \leq \langle \text{aexp} \rangle \mid \\ & \neg \langle \text{bexp} \rangle \mid \langle \text{bexp} \rangle \wedge \langle \text{bexp} \rangle\end{aligned}$$

Here we use $\mathtt{ff}$ to mean falsehood, $\mathtt{tt}$ for truth, $\neg$ for the negation, and $\wedge$ for the conjunction operator from logic. We also expect to be able to evaluate whether one arithmetic expression describes a numeral that is less than or equal to that given by another.

Again we allow the use of brackets, using the operator precedence for logical connectives to avoid fully bracketing our expressions.

This grammar recursively generates the set BExp of boolean expressions for our language. Its base cases are given by $\mathbf{tt}$, $\mathbf{ff}$, equality of arithmetic expressions and inequality of arithmetic expressions, while the step cases are provided by negation and conjunction.

We note that we are not able to write $\mathtt{a<b}$, for example, but that we may achieve the same effect by writing $\mathtt{a} \leq \mathtt{b} \wedge \neg \mathtt{a} = \mathtt{b}$.

As mentioned in the introduction, a theoretical language is lean on purpose regarding its syntax. This makes it less comfortable for programming. The following exercise is here to show that we have not lost expressive power.

Exercise 1.
For the following, assume that $\mathtt{a}$ and $\mathtt{b}$ are boolean expressions.

(a) how can we represent $\mathtt{a} \vee \mathtt{b}$ in the While language?

(b) How can we represent $\mathtt{a} \rightarrow \mathtt{b}$ in the While language?

It's more practical if we allow ourselves the use of boolean expressions that are not formally part of the language and think of them as shortcuts along the lines of the examples given above. This makes the code more readable.

1.4 Statements

The programming instructions of While are known as **statements**. Their syntax is

$$\begin{aligned}\langle\text{stmt}\rangle \ ::= \ & \langle\text{var}\rangle := \langle\text{aexp}\rangle \mid \texttt{skip} \mid \langle\text{stmt}\rangle;\langle\text{stmt}\rangle \mid \\ & \texttt{if}\ \langle\text{bexp}\rangle\ \texttt{then}\ \langle\text{stmt}\rangle\ \texttt{else}\ \langle\text{stmt}\rangle \mid \texttt{while}\ \langle\text{bexp}\rangle\ \texttt{do}\ \langle\text{stmt}\rangle\end{aligned}$$

Four of the statements allow us to carry out computation steps:

- We may assign an arithmetic expression or a variable to a variable. This allows us to store data, both intermediate results and the eventual output.
- We have a conditional statement in a form that appears in many imperative languages: If a boolean expression evaluates to `tt` then we carry out some statement, otherwise we carry out a (typically different) statement.
- We have a loop constructor in the form of a 'while loop': For such instructions, it is checked whether the boolean expression evaluates to `tt`, and if that is the case, the body of the loop is carried out. If the boolean expression still evaluates to `tt`, the body of the loop is carried out again, and this continues unless the boolean expression evaluates to `ff` at some point.

But, of course, we want to be able to carry out more than one computation step, and this is possible since we know that if we have two statements we may combine them using the semicolon operator: This allows us to string together a finite number of statements to form another statement.

Finally there is the `skip` statement. This is there for theoretical reasons.

Strictly speaking we should have included a *bracketing mechanism* to allow us to understand, for example, where the body of a loop ends, and further instructions begin. Indeed, for Java braces are used to help the compiler decide, for example, which instructions should be carried out if a boolean expression evaluates to true.

For While we adopt the convention that we use *indentation* to indicate this—it is friendlier to humans reading code than bracketing. However, we do occasionally need to write a While programme in one line, and in that case we do use brackets where appropriate.

The grammar for statements defines a set Stmt of statements generated by the grammar. We frequently use the term While programs for this set.

1.5 The syntax of While

For convenience we summarize the definition of entire language.

$$\begin{aligned}\langle\text{aexp}\rangle \ ::= \ & \langle\text{num}\rangle \mid \langle\text{var}\rangle \mid \\ & \langle\text{aexp}\rangle + \langle\text{aexp}\rangle \mid \langle\text{aexp}\rangle - \langle\text{aexp}\rangle \mid \langle\text{aexp}\rangle \times \langle\text{aexp}\rangle \\ \langle\text{bexp}\rangle \ ::= \ & \texttt{ff} \mid \texttt{tt} \mid \langle\text{aexp}\rangle = \langle\text{aexp}\rangle \mid \langle\text{aexp}\rangle \leq \langle\text{aexp}\rangle \mid \\ & \neg\langle\text{bexp}\rangle \mid \langle\text{bexp}\rangle \wedge \langle\text{bexp}\rangle \\ \langle\text{stmt}\rangle \ ::= \ & \langle\text{var}\rangle := \langle\text{aexp}\rangle \mid \langle\text{var}\rangle := \langle\text{var}\rangle \mid \texttt{skip} \mid \langle\text{stmt}\rangle;\langle\text{stmt}\rangle \mid \\ & \texttt{if}\ \langle\text{bexp}\rangle\ \texttt{then}\ \langle\text{stmt}\rangle\ \texttt{else}\ \langle\text{stmt}\rangle \mid \texttt{while}\ \langle\text{bexp}\rangle\ \texttt{do}\ \langle\text{stmt}\rangle\end{aligned}$$

Exercise 2.
Why is the grammar for the while language *ambiguous*? Give one example of an ambiguous Arithmetic Expression, Boolean Expression, and Statement. You may need to revisit Section 4.4 of Part 1.

When we refer to a While **program** we mean a statement in this language.

The following exercises are somewhat abstract and readers may prefer to first look at those that appear towards the end of this chapter.

Exercise 3.
Different concepts for loops exist. Explain how we may implement a 'repeat until' loop in While.

Exercise 4.
Show that we don't need if statements in While.

This exercise tells us that we could make our language even smaller by dispensing with if statements entirely, and we could carry out the same computations as before, a claim that we substantiate with Exercise 21.

1.6 Input and output

The reader may have wondered what to do about input and output for this language. For imperative programs, we typically consider both, the inputs and the outputs, as being stored in the variables the program has access to.

In a real world programming language, there would be some kind of mechanism to provide inputs, for example from a terminal or from a file, and to produce output, for example by printing or writing to a file. For abstract imperative languages we assume that values stored in variables when the program begins execution provide inputs, and the values held in those variables once the program terminates provides outputs. When we discuss specific examples we often give instructions which variables to view as inputs, and which as outputs.

1.7 Example programs

We look at our first example program.

```
x := 1; y := 5;
while y>0 do
    x := x × 2; y := y − 1;
```

For readers wanting to confirm what that program is intended to do, the behaviour we are aiming for (and which we formally define in the following chapter) is much the same as for the following program in Python.[1]

```
x = 1; y = 5
while y>0:
  x = x × 2
  y = y − 1
print x,y
```

The While program given looks quite similar to this Python program—there are just some differences in the syntax. When the program has completed its calculation the value in the variable x will be $2^5 = 32$ and we could consider that what the program computes.

What if we want to compute m^n for general m and n? In that situation we assume that the input is provided as the value of the variables m and n when the program starts. It is also good practice not to destroy the input in the course of the computation, so we copy its value.

```
x := 1; y := n;
while y>0 do
    x := x × m; y := y − 1
```

[1] To run this program, type Python on the command line to get the interactive mode, at least on any School machine.

Now the variable x will contain the value m^n when the program finishes.
We show additional examples.

Example 1. Assume we want to write a program that for values m and n in $\mathbb{N}$ with $n \neq 0$ delivers the result of the division and the remainder in the variables d and r respectively, that is

$$d = m \operatorname{div} n \qquad \text{and} \qquad r = m \bmod n.$$

We assume that the values m and n are stored in the variables m and n respectively.

```
r := m;
d := 0;
while n ≤ r do
    d := d + 1; r := r − n
```

We consider what we expect to happen when we run this program with initial values of $m = 10$ and $n = 3$.

iteration	r	d
1	10	0
2	7	1
3	4	2
4	1	3

Once the program terminates we may see that since

$$10 = 3 \cdot 3 + 1,$$

the values d = 3 and r = 1 are the ones we expect.

Example 2. We give an example of a program that finds an integer solution (if one exists) to the quadratic equation

$$ax^2 + bx + c = 0,$$

for integer coefficients a, b and c. We assume that the values for a, b and c are stored in the variables a, b and c respectively.

```
x := 0; y := 0;
while ¬y = 1 do
    if a × x × x + b × x + c = 0 then
        y := 1
    else
        if a × x × x − b × x + c = 0 then
            y := 1
        else
            skip
    x := x + 1
```

If a solution is found it is the value of the variable x once the program has been executed.

Example 3. Here is a program to compute the *greatest common divisor* gcd of two numbers, that is a program which accepts arguments passed $m \in \mathbb{N}$ and $n \in \mathbb{N}$, given as the values of the variables m and n respectively and places the output in x = y. In order not to destroy the input we copy the values of these variables at the start, but that is not strictly necessary to solve the problem.

```
x := m; y := n;
while ¬x = y do
    if y<x then
        x := x − y
    else
        y := y − x;
```

It's not immediately obvious that this program does indeed compute the greatest common divisor of m and n, and we look at some sample computations in Exercise 14.

Example 4. We provide a program to decide whether a number is a prime number. We assume that the variable n contains the input value, and that this is a natural number.

Testing for primality is straightforward: we just look at all the possible divisors and check whether they divide n, compare Example 1. We use the variable z to record the answer, using 0 for false and 1 for true.

```
y := n − 1;
z := 1;
while y>1 ∧ z = 1 do
  r := n;
  while y ≤ r do
      r := r − y;
      if r = 0 then
          z := 0;
      else
          skip ;
   y := y − 1;
```

Example 5. We provide an integer square root program, *i.e.* a program which accepts an argument passed in in variable $x \in \mathbb{N}$, and calculates

$$z = \lfloor \sqrt{(x)} \rfloor \qquad \text{as well as} \qquad r = x - z^2.$$

This means that z is the biggest integer below the square root of x, and r is the difference between x and the square of z.

There are lots of very clever ways to perform this calculation, but we'll choose the very simplest.

```
z := 0;
while (z + 1) × (z + 1) ≤ x do
    z := z + 1;
r := x − z × z
```

In effect we start with an initial guess that the square root is 0 and increment the guess at each step until a number is found that is "too big". If this program is confusing then do what we did before and write out the table of intermediate values.

Example 6. Here is a program to compute the nth Fibonacci number. We assume that variable $x \in \mathbb{N}$ contains the value n and place the result in variable z.

This is a standard programming task but here we do not have access to many of the language features we may be used to.

```
y := 1;
z := 0;
while x>0 do
    t := z;
    z := y;
    y := t + y;
    x := x − 1
```

Running this for $x = 4$ will lead to the variables holding the following values at the end of the loop:

iteration	x	y	z	t
1	4	1	0	0
2	3	1	1	0
3	2	2	1	1
4	1	3	2	1
5	0	5	3	2

i.e. we use y to remember the previous value in the sequence.

Exercise 5.
Write a While program that takes as input an integer n in the variable n and determines whether n is even or odd.

Use your solution so far to provide a program that computes $n \bmod m$ for integers m and n.

Exercise 6.
Let m and n be natural numbers. Write a program that finds the number of natural numbers x with the property that x is a cube and $m \leq x \leq n$. Assume that the inputs are given as the values of the variables m, and n.

Exercise 7.
Let m and n be integers. Write a program that finds a solution in the natural numbers to the following equation

$$mx + ny = k$$

if one exists. Assume that the inputs are given as the values of the variables m, n and k, and ensure that the answer your program finds is stored in the variables x and y.

Explain how you might extend your program so that it will find an integer solution if one exists.

Exercise 8.
For the functions β and β' in Section 6.2.1 give While programs that implement them. Assume that the input is given in the variable m, and that the output is the value of the variable x. You may assume that the input for β' is in $\mathbb{N}$.

Exercise 9.
For the functions ϕ and its inverse from Section 6.2.2 give While programs that implement them.

For ϕ assume that the input is given in the variables m and n, and that the output is the value of the variable x.

For ϕ' assume that the input is given in the variable `n` and that the output is the value of the variables `x` and `y`.

Exercise 10.
Let m and n be natural numbers. Write a program that implements Euclid's algorithm to find the greatest common divisor of m and n. Assume that the inputs are given in the variables `m` and `n`, and that the output is given in variable `x`.

If you are taking COMP111 you'll find Euclid's algorithm in Example 6.44. Alternatively, you'll see it described here https://www.cut-the-knot.org/blue/Euclid.shtml.

1.8 Summary

This chapter provides the syntax for our abstract model of computation. This syntax has analogies with real world programming languages that lets us anticipate what a program in this language might compute. We make the notion of computation precise in the next chapter.

1.9 Solutions to Exercises

Solution 1.
We look at both cases.

(a) A possible answer to represent $a \vee b$ in the While language is as

```
¬(¬a∧¬b)
```

We may insert this expression anywhere we use boolean expressions, that is for if statements or loops.

(b) For $a \to b$, we may use

```
¬(a∧¬b)
```

Note that in Exercise 11 we justify this choice by checking that these boolean expressions have the intended interpretation.

Solution 2.
The main reason the language as defined in the notes is ambiguous is that our grammar does not insist on using brackets to ensure that every While expression or statement can be uniquely parsed.

Introducing brackets everywhere makes it hard for humans to read expressions or statements, so in practice most languages have conventions that allow us to drop some of the brackets while maintaining an unambiguous interpretation.

You are familiar with such bracketing conventions for arithmetic expressions, for example since addition of integers is associative, we may write $4 + 5 + 6$ without the ambiguity causing issues, but also there is a convention that we interpret $0 - 0 - 1$ as $(0 - 0) - 1$ rather than $0 - (0 - 1)$, and these two expressions do not describe the same number.

Ambiguous Arithmetic Expression: Consider the expression $z - y - x$. There are two ways of generating this string using the grammar. We note that depending on the values of z, y and x, the expected value of the expression may differ for different ways of bracketing this expression, for example if $x = 1$, $y = 0$ and $z = 0$ then

$$(0 - 0) - 1 = -1 \qquad \text{whereas} \qquad 0 - (0 - 1) = 0 - (-1) = 1.$$

Ambiguous Boolean Expression: Similarly the expression $a \wedge b \wedge c$ is ambiguous for given boolean expressions a, b and c for much the same reasons with the same remedy. (We note that in this case the interpretations of the expressions $(a \wedge b) \wedge c$ and $a \wedge (b \wedge c)$ agree (see the following chapter for a precise definition), so in this case the ambiguity does not cause different behaviours for different ways of parsing the expression.)

Ambiguous Statement: Finally, for statements S, S' and S'' the statement S; S'; S'' is ambiguous, and again one might use brackets to resolve this. See Exercise 24 for the question whether the two different ways of parsing this statement lead to different results when executing it.

Solution 3.
A 'repeat until' loop carries out the body of a loop before checking the condition for the first time. It then keeps carrying out the body of the loop until the given condition is true.

Hence possible answer is as follows:

```
S;
while ¬b do
    S
```

Solution 4.
We show how to replace the statement `if b then S else S'` without using if statements. We assume that x is a variable not used anywhere in the if statement. (We can always find a suitable variable since the statement can only mention finitely many variables, so there's always a fresh variable available.)

```
x := 0;
while b∧x=0 do
    x := 1; S;
while x=0 do
    x := 1; S'
```

See Exercise 21 for a discussion of the behaviour of this program.

Solution 5.
We provide a program that solves the first problem. It's a good idea to think ahead to the second problem we want to solve. The output of our program is the value of the variable x after termination, with a value of 0 indicating that the input is even.

```
x := n;
if 0≤n then
    while 2≤x do
        x := x−2;
else
    while x≤−2 do
        x := x+2
```

We now tackle the more general case.

```
x := n;
if 0≤m then
    y := m;
else
    y := −m;
if 0≤n then
    while m≤x do
        x := x−y;
else
    while x≤−m do
        x := x+y
```

Again the desired output may be found by looking at the value held by the variable x once the program terminates.

Solution 6.
We provide a suitable program.

```
x := 0;  y := 0;
while x×x×x<m do
    x := x+1;
while x×x×x≤n do
    x := x+1;
    y := y+1;
```

The desired number is the value of y when the program terminates.

Solution 7.
We begin with giving a program that searches for a solution through the natural numbers. We may think of solutions as coming in the form of pairs (x, y), and we need to look at all possible such pairs in the natural numbers to ensure we find a solution if it exists.

We increase x in the outer loop, and y in the inner loop, but only until we get up to x (otherwise this would create an infinite loop and we would only look at solutions where $x = 1$). This means we look at pairs $(0, 0)$, $(1, 0)$, $(1, 1)$, $(2, 0)$, $(2, 1)$, $(2, 2)$,.... But there's one problem with this idea: We only look at potential solutions where $x \geq y$. We can fix this by checking whether for a given pair (x, y) we obtain a solution if we swap the two values, which we do in an if statement.

```
x := 0;  y := 0;
while ¬(m×x+n×y=k) do
    x := x+1;  y := 0;
        while ¬(m×x+n×y=k)∧y<x do
            if m×y+n×x=k then
                z := x;  x := y;  y := z
            else
            y := y+1
```

To find all existing integer solutions we also have to test whether any combination of ±x or ±y solves the problem, and we have to do this also for the situation where we swap these values. This may be done using a lot of if statements.

Solution 8.
Here is a program that implements β:

```
if 0≤n then
    x := 2×n
else
    x := −2×n−1
```

For the program that implements β' we need to use a loop.

```
r := n;  x := 0;
while 2≤ r do
```

```
        x:=x+1; r=r−2;
    if r=1 then
        x:=−x−1;
    else
        skip
```

Solution 9.
Here is a program that implements ϕ:

```
y:=1; x:=1;
while y≤m do
    x:=2×x;
    y:=y+1;
x:=x×(2×n+1)
```

For the program that implements ϕ's inverse we first check for the highest power of 2 that is smaller than 1 added to the input n, given in n. We begin by determining the highest power of 2 that is less than or equal to $n+1$. This is stored in p and the corresponding exponent in e. We then check whether this, or a lower power of 2, divides the input, and this gives the output x.

We then divide the input by that number, subtract 1, divide by 2, and that gives the output y.

```
p:=1; e:=0;
while p≤n+1 do
    p:=p×2; e:=e+1;
e:=e−1;
while 1<e do
    m=n+1; x:=e; p:=1; d:=1;
    while 0<x do
        p:=p×2; x=x−1;
    while p<m do
        m:=m−p; d:=d+1
    if m=0 then
        e:=0; x:=d;
    else
        e:=e−1; x:=0;
m:=m−1;y:=0
while 2≤m do
    m:=m−2; y:=y+1
```

Solution 10.
We provide a suitable program.

```
while ¬n=0 do
    while m<n do
        n:=n−m;
    x:=n; n:=m; m:=n
```

Chapter 2

Computing with While programs

In the overview we state that abstract models of computation allow us to make rigorous the notion of a computation. However, so far our understanding of example programs has been entirely based on the *intended meaning* of our syntax, relying on our intuition based on real world programming languages.

In this section we make good on the claim that we may use our theoretical programming language to rigorously describe how to compute with it.

2.1 States

As stated in the overview, a typical imperative programming language is based on having a notion of the state of a program, by which we mean the values stored in the variables. That state is manipulated via programming instructions, which may change the values stored in the variables mentioned in the program. This idea also applies to how we define computation for While programs.

As explored in the previous chapter, we may think of the values held by the variables to serve a number of purposes:

- We may think of the *input* to the program being given by the values held by chosen variables at the start of the computation.
- We may think of the *output* of the program as being given by the values held by chosen variables once the computation has ended.
- We can see that manipulating the values of variables is also what allows us to construct data along the way to finding the desired answer.

We think of a state as being given by the assignment of a value to every variable. There is an issue with this idea that we have to resolve: A program can only mention a finite number of variables. But we also don't want to assume that there's a given finite number of variables available to use—we always want to be able to find a new variable we may use if we want to expand our program.

As stated in Section 1.1 we assume that there's an infinite set Vars of variables. One might expect these variables to be named as $\mathtt{x}_0$, $\mathtt{x}_1$, x_2 and so on, but that would make our programs less readable, so we allow ourselves to use convenient one letter names in our example programs. One might instead decide to allow variable names that are more sophisticated, but for the small programs we look at we do not see a need for that.

Having to have a notion of 'the variables the program knows something about' and keeping track of their values is tedious to deal with—instead we assume that we may as well think of a state as having finitely many variables with a non-zero integer value, and we take 0 as the value stored in all the other variables. This allows us to define a notion of state that works for every While program.

Definition 1. A **state** is a function

$$\sigma : \mathsf{Vars} \longrightarrow \mathbb{Z}$$

such that there are only finitely many variables $x \in \mathsf{Vars}$ with $\sigma x \neq 0$. We use St for the set of all states.

We use Greek letters to name states so that we don't confuse them with anything else.

Given a state σ we may query what the value of the variable x is in that state by applying σ to x, that is, the value of the variable x in the state σ is σx.

2.1.1 Describing states

We need to be able to describe specific states in order to talk about how a While statement changes a state when it is carried out. Given that we know that all but finitely many values held by our variables are 0, it's possible to give a finite description of every state.

The program

```
r :=m;
d:= 0;
while n≤r do
    d:=d+1;  r:=r−n
```

from Example 1 knows about variables m, n (the inputs) as well as d and r (the outputs). When we call the program with inputs m=10 and n = 3, we use

$$\{\mathtt{m} \mapsto 10, \mathtt{n} \mapsto 3\}$$

to describe the state where m holds the value 10, n holds the value 3, and all other variables hold the value 0. Once we have run the program from that state we expect to be in the state

$$\{\mathtt{m} \mapsto 10, \mathtt{n} \mapsto 3, \mathtt{d} \mapsto 3, \mathtt{r} \mapsto 1\}.$$

Note that to describe a state, all the variables mentioned in the description have to be distinct. This is in contrast with the notion of *updating a state* in the following section.

2.1.2 Manipulating states

As we carry out the statements from a program we expect to change the state, so we need notation that allows us to do this.

Given a state σ, we would like to define a new state where the variable x has been updated to hold the value n. We use

$$\sigma[x \mapsto n]$$

for this purpose. We define this idea formally by saying what the new state gives as the value of a variabley.

$$(\sigma[x \mapsto n])\,\mathtt{y} = \begin{cases} n & \mathtt{y} = \mathtt{x} \\ \sigma\mathtt{y} & \text{else.} \end{cases}$$

We may see that indeed, unless the variable y is equal to x, we obtain the value of y in the state σ, and otherwise we obtain the updated value n.

Note that we may want to update the value of a variable x with the value held in the variable y, in which case we apply the update

$$\sigma[x \mapsto \sigma\mathtt{y}].$$

If we want to update more than on variable then using the notation repeatedly is unwieldy:

For the program discussed above, if

$$\sigma = \{\mathtt{x} \mapsto 10, \mathtt{y} \mapsto 3\},$$

is the state we are in at the start then after running the program is the updated state may be described as

$$\big(\sigma[\mathtt{d} \mapsto 3]\big)[\mathtt{r} \mapsto 1].$$

This becomes tedious to describe when we want to change several variables, and so we adopt the following notation as a shortcut:

$$\sigma[\mathtt{d} \mapsto r, \mathtt{r} \mapsto 1].$$

Note that

$$\sigma[\mathtt{x} \mapsto 1, \mathtt{x} \mapsto 2]$$

is a valid update, while

$$\{x \mapsto 1, x \mapsto 2\}$$

is *not* a valid description of a state.

We also note that if we are updating the same variable more than once it is the final update that sticks, so for example

$$\sigma[\mathtt{x} \mapsto 1, \mathtt{x} \mapsto 2] = \sigma[\mathtt{x} \mapsto 2].$$

2.2 Extended example: Computation via updating states

Readers who prefer to see the general definition rather than first working through an example are directed to read Section 2.3.4 first, or Section 2.3 if they would like to see the general rules explained.

We return to the sample program

```
r := m;
d := 0;
while n≤r do
    d:=d+1;  r:=r−n
```

from Example 1. Assume we start in the state

$$\sigma = \{m \mapsto 10, n \mapsto 3\}.$$

When the first statement, $\mathtt{r} := \mathtt{m}$, has been carried out we expect to be in the state

$$\sigma[\mathtt{r} \mapsto \sigma\mathtt{m}],$$

and after the second statement, $\mathtt{d} := 0$, has been carried out in the state

$$\sigma' = \sigma[\mathtt{r} \mapsto \sigma\mathtt{m}, \mathtt{d} \mapsto 0] = \{\mathtt{m} \mapsto 10, n \mapsto 3, \mathtt{r} \mapsto 10, \mathtt{d} \mapsto 0\}.$$

So far so good, but now we come to the loop and life becomes a bit more complicated.

We first of all have to decide whether we carry out the body of the loop at all, which means we have to evaluate the boolean expression $\mathtt{n} \leq \mathtt{r}$ is true.

We are going to use boolean values

$$\mathbb{B} = \{\mathbf{tt}, \mathbf{ff}\}$$

when we evaluate such expressions.

We know that $\sigma'\mathtt{n} = 3$ and $\sigma'\mathtt{r} = 0$, so $\mathtt{n} \leq \mathtt{r}$ evaluates to the boolean value $\mathbf{tt}$ in the state σ'. Hence we do carry out the body of the loop, and when we have done this we expect to be in the state

$$\sigma'' = \sigma'[\mathtt{d} \mapsto \sigma'(\mathtt{d}) + 1, \mathtt{r} \mapsto \sigma'(\mathtt{r}) - \sigma'(\mathtt{n})] = \{\mathtt{m} \mapsto 10, \mathtt{n} \mapsto 3, \mathtt{r} \mapsto 7, \mathtt{d} \mapsto 1\}.$$

We have to check what has happened to the boolean expression $\mathtt{n} \leq \mathtt{r}$. We have $\sigma''\mathtt{n} = 3$ and $\sigma''\mathtt{r} = 7$, so this still evaluates to **tt** and we have to carry out the body of the loop again.

This takes us to the state

$$\tau = \sigma''[\mathtt{d} \mapsto \sigma''(\mathtt{d}) + 1, \mathtt{r} \mapsto \sigma''(\mathtt{r}) - \sigma''(\mathtt{n})] = \{\mathtt{m} \mapsto 10, \mathtt{n} \mapsto 3, \mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2\}.$$

We evaluate the boolean expression $\mathtt{n} \leq \mathtt{r}$ in the state τ, and since $\tau\mathtt{n} = \tau\mathtt{r}$ and $\tau\mathtt{r} = 4$, it still evaluates to **tt** and we have to carry out the body of the loop once again, and we expect to end up in the state

$$\rho = \tau[\mathtt{d} \mapsto \tau(\mathtt{d}) + 1, \mathtt{r} \mapsto \tau(\mathtt{r}) - \tau\mathtt{n}] = \{\mathtt{m} \mapsto 10, \mathtt{n} \mapsto 3, \mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3\}.$$

In the state ρ we have $\rho\mathtt{n} = 3$ and $\rho\mathtt{r} = 1$, so in this state the boolean expression $\mathtt{n} \leq \mathtt{r}$ evaluates to **ff**, and we are done with the while loop. This is the end of the program.

So if we start our program in the state σ we expect to be in the state ρ after the program has finished.

We now want to formalize this idea so that we may apply it to arbitrary While programs for arbitrary states. That creates a fair amount of notation that looks intimidating at first sight. We aim to formalize the above considerations, and we show how the formal system introduced in the following section may be used on this example in Section 2.4.

2.3 Computation via updating states step by step

We want to define in general terms how executing a While programme changes the current state. From the example we see that in order to do that we need to carry out the following tasks:

- Define the evaluation of an arithmetic expression to an integer so that we may evaluate boolean expressions.
- Define the evaluation of a boolean expression to a truth value in $\mathbb{B} = \{\mathbf{tt}, \mathbf{ff}\}$ so that we may work out the action of a loop, or a conditional statement.
- Define how each kind of statement changes the current state.

It should come as no surprise that we use a *recursive definition* using the recursive definition of a program.

2.3.1 Arithmetic expressions

Given an arithmetic expression and a state, we expect to be able to determine an integer. This requires first of all that the numerals that are part of our language (and which give us the the base case of an arithmetic expression) has an interpretation, so we expect to have a function

$$\mathcal{N} : \mathtt{Num} \longrightarrow \mathbb{Z},$$

which, given a numeral $n \in \mathtt{Num}$ returns the integer which the numeral represents.

This now allows us to recursively define a function $\mathcal{A}$ which takes as input an arithmetic expression and a state and which returns an integer.

bc num $\quad \mathcal{A}\ \mathtt{n}\ \sigma = \mathcal{N}\mathtt{n}.$

bc var $\quad \mathcal{A}\ \mathtt{x}\ \sigma = \sigma\mathtt{x}.$

sc + $\quad \mathcal{A}\,(\mathtt{a} + \mathtt{a'})\,\sigma = (\mathcal{A}\ \mathtt{a}\,\sigma) + (\mathcal{A}\ \mathtt{a'}\ \sigma).$

sc − $\quad \mathcal{A}\,(\mathtt{a} - \mathtt{a'})\,\sigma = (\mathcal{A}\ \mathtt{a}\,\sigma) - (\mathcal{A}\ \mathtt{a'}\ \sigma).$

sc $\times$ $\quad \mathcal{A}\,(\mathtt{a} \times \mathtt{a'})\,\sigma = (\mathcal{A}\ \mathtt{a}\,\sigma) \cdot (\mathcal{A}\ \mathtt{a'}\ \sigma).$

In our extended example above, Section 2.2, we used this implicitly when evaluating the boolean expression controlling the loop.

Note that for a given state it is always possible to evaluate a given arithmetic expression to a number.

Formally $\mathcal{A}$ is a function

$$\text{from } \mathsf{AExp} \times \mathsf{St} \qquad \text{to } \mathbb{Z},$$

where AExp is the set of arithmetic expressions. We would usually write such a function applied to its arguments as $\mathcal{A}(a, \sigma)$, but in computer science we sometimes prefer to supply arguments one by one, writing $\mathcal{A}\ a\,\sigma$ instead.

2.3.2 Boolean expressions

Now that we have a way of interpreting arithmetic expressions as integers we may interpret boolean expressions as boolean values. Again we expect this to depend on the current state, so we define a function $\mathcal{B}$ which takes as input a boolean expression and a state and returns a value in $\mathbb{B}$.

bc false $\quad \mathcal{B}\ \mathtt{ff}\ \sigma = \mathbf{ff}.$

bc true $\quad \mathcal{B}\ \mathtt{tt}\ \sigma = \mathbf{tt}.$

sc $=$ $\quad \mathcal{B}\,(\mathtt{a} = \mathtt{a'})\,\sigma = \begin{cases} \mathbf{tt} & \mathcal{A}\ \mathtt{a}\,\sigma = \mathcal{A}\ \mathtt{a'}\ \sigma \\ \mathbf{ff} & \text{else.} \end{cases}$

sc $\leq$ $\quad \mathcal{B}\,(\mathtt{a} \leq \mathtt{a'})\,\sigma = \begin{cases} \mathbf{tt} & \mathcal{A}\ \mathtt{a}\,\sigma \leq \mathcal{A}\ \mathtt{a'}\ \sigma \\ \mathbf{ff} & \text{else.} \end{cases}$

sc $\neg$ $\quad \mathcal{B}\,(\neg\ \mathtt{b})\,\sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{ff} \\ \mathbf{ff} & \text{else.} \end{cases}$

sc $\wedge$ $\quad \mathcal{B}\,(\mathtt{b} \wedge \mathtt{b'})\,\sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{tt} \text{ and } \mathcal{B}\ \mathtt{b'}\ \sigma = \mathbf{tt} \\ \mathbf{ff} & \text{else.} \end{cases}$

We used this idea informally in the extended example in Section 2.2 above. Formally, $\mathcal{B}$ is a function from the set

Note that given a state, every boolean expression evaluates to either **tt** or **ff**.

Formally $\mathcal{B}$ is a function

$$\text{from } \mathsf{BExp} \times \mathsf{St} \qquad \text{to } \mathbb{B},$$

where BExp is the set of boolean expressions. We would usually write such a function applied to its arguments as $\mathcal{B}(\mathtt{b}, \sigma)$, but in computer science we sometimes prefer to supply arguments one by one, writing $\mathcal{B}\ \mathtt{b}\,\sigma$ instead.

Exercise 11.
Take your solutions to Exercise 1 and show that the interpretation of the boolean expressions you give have the expected evaluation.

2.3.3 Statements

We are now ready to look at statements. Given a statement and a state, we expect to be able to potentially carry out one step of computation and move to the remaining statement and an updated state. The following rules, defined by recursion over the statement, set out how this works. The computation is complete when we are left with a skip statement (and nothing else) and a state.

bc := This rules asks us to update the state as expected when assigning an arithmetic expression a to a variable x:

$$\langle \mathtt{x} := \mathtt{a}, \sigma\rangle \Rightarrow \langle \mathbf{skip}, \sigma[x \mapsto \mathcal{A}\ a\,\sigma]\rangle$$

bc skip The skip command does not come with any computation steps to be carried out and there is no rule here.

sc ; If carrying out a step of statement S leads to the remaining statement S' (still to be carried out) with an updated state, we want to be able to do carry out this step when S is part of a bigger statement.

$$\frac{\langle \mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S'}, \sigma'\rangle}{\langle \mathtt{S;S''}, \sigma\rangle \Rightarrow \langle \mathtt{S';S''}, \sigma'\rangle}$$

If we have a skip statement preceding a computation we may move to the next step of the computation discarding that statement, and keeping the same state.

$$\langle \mathbf{skip}\,;\mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S}, \sigma\rangle$$

sc if If in the state σ the boolean expression b evaluates to **tt** we execute S and otherwise S'.

$$\langle\, \mathbf{if}\ \ \mathtt{b}\ \mathbf{then}\ \mathtt{S}\ \ \mathbf{else}\ \ \mathtt{S'}, \sigma\rangle \Rightarrow \begin{cases} \langle \mathtt{S}, \sigma\rangle & \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{tt} \\ \langle \mathtt{S'}, \sigma\rangle & \text{else} \end{cases}$$

sc while We may approach the semantics for loops by checking whether the boolean expression governing it evaluates to **tt** in the current state, or not.

$$\langle \mathbf{while}\ \ \mathtt{b}\ \ \mathbf{do}\ \mathtt{S}, \sigma\rangle \Rightarrow \begin{cases} \langle \mathtt{S};\mathbf{while}\ \ \mathtt{b}\ \ \mathbf{do}\ \mathtt{S}, \sigma\rangle & \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{tt} \\ \langle \mathbf{skip}, \sigma\rangle & \text{else.} \end{cases}$$

We note that whenever we are given a statement S and a state σ then unless S=**skip**, there is always a transition to a pair consisting of a statement and an updated state.

We further note that if we have

$$\langle \mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S'}, \rho\rangle \qquad \text{as well as} \qquad \langle \mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S'}, \tau\rangle,$$

then

$$\rho = \tau.$$

In other words: if starting in the state σ we have carried out some of the instructions of S with the same statement, S', left to be executed we have reached a uniquely determined state.

Exercise 12.
Justify the claim preceding this exercise.

This is a semantics using a *transition system*: We have nodes (confusingly called states in the language of transition systems) labelled with a statement and a state, and these are connected if carrying out a computation step of the statements takes us to the corresponding remaining statement and updated state. We may think of this system as a very large graph, with nodes given by the set $\mathsf{Stmt} \times \mathsf{St}$, and edges given by $\Rightarrow$. When we carry out a stepwise computation we start at the node given by the While program under consideration and the start state. We then follow this computation step by step, and for each step we follow the corresponding edge in our graph. This graph is much too large to draw, so we only look at a very small selection of nodes and edges at a time.

We are following the computation step by step, and this formal model of computation is called a **small step operational semantics**.

2.3.4 The stepwise operational semantics of While

For convenience we give the whole system on one page, leaving the labels of the recursive definitions implicit.

$$\mathcal{A}\ \mathtt{n}\,\sigma = \mathcal{N}\mathtt{n} \qquad \mathcal{B}\ \mathbf{ff}\ \sigma = \mathbf{ff} \qquad \mathcal{B}\ \mathbf{tt}\ \sigma = \mathbf{tt}$$

$$\mathcal{A}\ \mathtt{x}\,\sigma = \sigma\mathtt{x} \qquad \mathcal{B}(a = a')\,\sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ a\,\sigma = \mathcal{B}\ a'\ \sigma \\ \mathbf{ff} & \text{else.} \end{cases}$$

$$\mathcal{A}(a + a')\,\sigma = (\mathcal{A}\ a\,\sigma) + (\mathcal{A}\ a'\ \sigma) \qquad \mathcal{B}(a \leq a')\,\sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ a\,\sigma \leq \mathcal{B}\ a'\ \sigma \\ \mathbf{ff} & \text{else.} \end{cases}$$

$$\mathcal{A}(a - a')\,\sigma = (\mathcal{A}\ a\,\sigma) - (\mathcal{A}\ a'\ \sigma) \qquad \mathcal{B}(\neg a)\,\sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ a\,\sigma = \mathbf{ff} \\ \mathbf{ff} & \text{else.} \end{cases}$$

$$\mathcal{A}(a \times a')\,\sigma = (\mathcal{A}\ a\,\sigma) \cdot (\mathcal{A}\ a'\ \sigma) \qquad \mathcal{B}(a \wedge a')\,\sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ a\,\sigma = \mathbf{tt} \text{ and } \mathcal{B}\ a'\ \sigma = \mathbf{tt} \\ \mathbf{ff} & \text{else.} \end{cases}$$

$$\langle \mathtt{x} := \mathtt{a}, \sigma\rangle \Rightarrow \langle \mathbf{skip}, \sigma[x \mapsto \mathcal{A}\ a\,\sigma]\rangle \qquad \langle \mathbf{skip}\,;\mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S}, \sigma\rangle$$

$$\langle \mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S'}, \sigma'\rangle \qquad \text{implies} \qquad \langle \mathtt{S;S''}, \sigma\rangle \Rightarrow \langle \mathtt{S';S''}, \sigma'\rangle$$

$$\langle \mathbf{if}\ \mathtt{b}\ \mathbf{then}\ \mathtt{S}\ \mathbf{else}\ \mathtt{S'}, \sigma\rangle \Rightarrow \begin{cases} \langle \mathtt{S}, \sigma\rangle & \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{tt} \\ \langle \mathtt{S'}, \sigma\rangle & \text{else} \end{cases}$$

$$\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{S}, \sigma\rangle \Rightarrow \begin{cases} \langle \mathtt{S};\mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{S}, \sigma\rangle & \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{tt} \\ \langle \mathbf{skip}, \sigma\rangle & \text{else.} \end{cases}$$

2.4 Example: A formal stepwise computation

We return to the example from Section 2.2, where we begin in the state $\sigma = \{\mathtt{m} \mapsto 10, \mathtt{n} \mapsto 3\}$. We again use L as a name for the loop part of the programme, that is L is

```
while n≤r do
   d:=d+1;  r:=r−n
```

We are now able to formally say which state we end up in when executing our program from start state σ:

We note that we have to invoke the general case of the ; rule all the time and we do not mention that explicitly to save a little space.

We use B for the body of the loop, that is

$$\mathtt{B} \text{ is } \mathtt{d{:}d} + 1;\mathtt{r} := \mathtt{r} - \mathtt{n}$$

$\langle \mathrm{r} := \mathrm{m}; \mathrm{d} := 0; \mathrm{L}, \sigma \rangle$

$\Rightarrow \langle \textbf{skip}; \mathrm{d} := 0; \mathrm{L}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10\} \rangle$

$:=$ rule since $\{\mathrm{m} \mapsto 0, \mathrm{n} \mapsto 3\}[\mathrm{r} \mapsto \sigma \mathrm{m}] = \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10\}$

$\Rightarrow \langle \mathrm{d} := 0; \mathrm{L}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10\} \rangle$

by the ; rule for **skip**

$\Rightarrow \langle \textbf{skip}; \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10, \mathrm{d} \mapsto 0\} \rangle$

$:=$ rule since $\{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10\}[\mathrm{d} \mapsto 0] = \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10, \mathrm{d} \mapsto 0\}$

$\Rightarrow \langle \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10, \mathrm{d} \mapsto 0\} \rangle$

by the ; rule for **skip**

$\Rightarrow \langle \mathrm{B}; \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10, \mathrm{d} \mapsto 0\} \rangle$

by the while rule since $\mathcal{B}(\mathrm{n} \leq \mathrm{r})\{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10, \mathrm{d} \mapsto 0\} = \mathbf{tt}$

$\Rightarrow \langle \textbf{skip}; \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 7, \mathrm{d} \mapsto 1\} \rangle$

$\langle \mathrm{d}{:}\mathrm{d} + 1; \mathrm{r} := \mathrm{r} - \mathrm{n}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10, \mathrm{d} \mapsto 0\} \rangle \Rightarrow \langle \textbf{skip}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 7, \mathrm{d} \mapsto 1\} \rangle$

$\Rightarrow \langle \mathrm{B};\ \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 7, \mathrm{d} \mapsto 1\} \rangle$

by the while rule since $\mathcal{B}(\mathrm{n} \leq \mathrm{r})\{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 7, \mathrm{d} \mapsto 1\} = \mathbf{tt}$

$\Rightarrow \langle \textbf{skip}; \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 4, \mathrm{d} \mapsto 2\} \rangle$

$\langle \mathrm{d}{:}\mathrm{d} + 1; \mathrm{r} := \mathrm{r} - \mathrm{n}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 7, \mathrm{d} \mapsto 1\} \rangle \Rightarrow \langle \textbf{skip}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 4, \mathrm{d} \mapsto 2\} \rangle$

$\Rightarrow \langle \mathrm{B};\ \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 4, \mathrm{d} \mapsto 2\} \rangle$

by the while rule since $\mathcal{B}(\mathrm{n} \leq \mathrm{r})\{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 4, \mathrm{d} \mapsto 2\} = \mathbf{tt}$

$\Rightarrow \langle \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } (\mathrm{d}{:}\mathrm{d} + 1; \mathrm{r} := \mathrm{r} - \mathrm{n}), \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 1, \mathrm{d} \mapsto 3\} \rangle$

$\langle \mathrm{d}{:}\mathrm{d} + 1; \mathrm{r} := \mathrm{r} - \mathrm{n}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 4, \mathrm{d} \mapsto 2\} \rangle \Rightarrow \langle \textbf{skip}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 1, \mathrm{d} \mapsto 3\} \rangle$

$\Rightarrow \langle \textbf{skip}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 1, \mathrm{d} \mapsto 3\} \rangle$

by the while rule since $\mathcal{B}(\mathrm{n} \leq \mathrm{r})\{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 1, \mathrm{d} \mapsto 3\} = \mathbf{ff}$

We have reached the end of the computation when the only instruction left to be carried out is **skip**, which we may think of as standing in for the empty program.

Typing out the full justification is still quite an effort, and typically we shorten our calculations of such computations, see the examples following.

Example 7. We provide a more compact presentation of the computation from above of the kind that we expect students to give. We allow ourselves to carry out several assignments in one step (but please note that we indicate that this requires more than one transition!), we typically skip over removing the **skip** statement and we don't provide justifications as detailed as those above.

Note that we again use B for the body of the loop.

$\langle \mathrm{r} := \mathrm{m}; \mathrm{d} := 0; \textbf{while } \mathrm{r} \leq \mathrm{n} \textbf{ do } \mathrm{B}, \sigma \rangle$

$\Rightarrow^4 \langle \textbf{while } \mathrm{r} \leq \mathrm{n} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10, \mathrm{d} \mapsto 0\} \rangle$	$:=$ rule
$\Rightarrow \langle \mathrm{B}; \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 10, \mathrm{d} \mapsto 0\} \rangle$	while rule **tt**
$\Rightarrow^4 \langle \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 7, \mathrm{d} \mapsto 1\} \rangle$	$:=$ rule
$\Rightarrow \langle \mathrm{B}; \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 7, \mathrm{d} \mapsto 1\} \rangle$	while rule **tt**
$\Rightarrow^4 \langle \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 4, \mathrm{d} \mapsto 2\} \rangle$	$:=$ rule
$\Rightarrow \langle \mathrm{B}; \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 4, \mathrm{d} \mapsto 2\} \rangle$	while rule **tt**
$\Rightarrow^4 \langle \textbf{while } \mathrm{n} \leq \mathrm{r} \textbf{ do } \mathrm{B}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 1, \mathrm{d} \mapsto 3\} \rangle$	$:=$ rule
$\Rightarrow \langle \textbf{skip}, \{\mathrm{m} \mapsto 10, \mathrm{n} \mapsto 3, \mathrm{r} \mapsto 1, \mathrm{d} \mapsto 3\} \rangle$	while rule **ff**

Example 8. Sometimes we ask students to track the execution of a programme, and in that situation the following table is what we are looking for: It keeps track of how the values of the variables change, and it looks at conditions governing if statements or loops. It is fine to omit variables that never change, such as m and n in the example from above. In this example the conditioning governing the loop $n \leq r$ is easy to keep track off and does not require a separate column.

We repeat the program for convenience.

```
r :=m;
d := 0;
while n≤r do
    d := d+1;  r := r−n
```

We assume that at the start the value in m is 10, and that in n is 3—this is the value that we check for the loop condition.

	d	r
start	0	10
loop	1	7
loop	2	4
loop	3	1

If readers struggle with the following exercises a this stage they may want to look at additional examples first.

Exercise 13.
Reason that if S is a statement that does not mention the variable x then given a state σ we have that $\langle \mathtt{S}, \sigma \rangle \Rightarrow \langle \mathtt{S'}, \sigma' \rangle$ implies that $\sigma x = \sigma' x$.

What can you say about variables whose values do change as we execute the given statement?

Exercise 14.
For the program determining the greatest common divisor gcd of two inputs held in m and n from Example 3, compute all its transitions in the small step semantics for a state σ with $\sigma \mathtt{m} = 12$ and $\sigma \mathtt{n} = 30$. Make sure to justify each step.

2.5 Non-termination

It is clear that the While language allows us to write programs that do not terminate, such as

```
x := 0;
while 0≤x do
    x := x+1
```

Readers may think that this program is obviously not a serious attempt at answering a question and so there's no need to worry about this. But consider Example 2 from the previous section: It's not clear how many steps it might take to find a solution, or whether a solution exists at all. Indeed, the question of whether or not a program terminates is very much non-trivial, and we encounter it also in Chapters 4 and 5, and if we try to limit ourselves to terminating programs there are many problems we cannot solve, as is discussed in Chapter 5.

We try to work out what happens when we compute the states we encounter for a non-terminating program.

Example 9. We begin with the program from above.

```
x := 0;
while 0≤x do
    x := x+1
```

We get the following

$$
\begin{aligned}
&\langle \mathtt{x} := 0;\textbf{while } 0<\mathtt{x} \textbf{ do } \mathtt{x} := \mathtt{x}+1, \sigma\rangle \\
&\quad \Rightarrow^2 \langle \textbf{while } 0 \leq \mathtt{x} \textbf{ do } \mathtt{x} := \mathtt{x}+1, \sigma[x \mapsto 0]\rangle && := \text{ rule} \\
&\quad \Rightarrow \langle \mathtt{x} := \mathtt{x}+1;\textbf{while } 0 \leq \mathtt{x} \textbf{ do } \mathtt{x} := \mathtt{x}+1, \sigma[x \mapsto 0]\rangle && \text{while rule } \mathbf{tt} \\
&\quad \Rightarrow^2 \langle \textbf{while } 0 \leq \mathtt{x} \textbf{ do } \mathtt{x} := \mathtt{x}+1, \sigma[x \mapsto 1]\rangle && := \text{ rule} \\
&\quad \Rightarrow \langle \mathtt{x} := \mathtt{x}+1;\textbf{while } 0 \leq \mathtt{x} \textbf{ do } \mathtt{x} := \mathtt{x}+1, \sigma[x \mapsto 1]\rangle && \text{while rule } \mathbf{tt} \\
&\quad \Rightarrow^2 \langle \textbf{while } 0 \leq \mathtt{x} \textbf{ do } \mathtt{x} := \mathtt{x}+1, \sigma[x \mapsto 2]\rangle && := \text{ rule} \\
&\quad \Rightarrow \langle \mathtt{x} := \mathtt{x}+1;\textbf{while } 0 \leq \mathtt{x} \textbf{ do } \mathtt{x} := \mathtt{x}+1, \sigma[x \mapsto 2]\rangle && \text{while rule } \mathbf{tt} \\
&\quad \Rightarrow^2 \langle \textbf{while } 0 \leq \mathtt{x} \textbf{ do } \mathtt{x} := \mathtt{x}+1, \sigma[x \mapsto 3]\rangle && := \text{ rule} \\
&\quad \Rightarrow \ldots
\end{aligned}
$$

We may see that this will keep going forever without us arriving at a final state, which is the behaviour one might expect.

The example above makes it easy to detect that when executing the program there is always a transition and this process never ends. But in general, this is not the case. We look at another example.

Example 10. Recall the program from Example 2 that searches for integer solution to a quadratic equation.

```
x := 0;  y := 0;
while ¬(y=1) do
    if a×x×x + b×x + c = 0 then
        y := 1
    else
        if a×x×x − b×x + c = 0 then
            y := 1
        else
            skip
    x := x+1
```

Assume we have a state σ with $\sigma\mathtt{a} = 1$, $\sigma\mathtt{b} = -2$ and $\sigma\mathtt{c} = 2$. We work through the change of states as the program is being executed. We use `I` and `I'` for the two if statements and `L` for the loop.

$$
\begin{aligned}
&\langle \mathtt{x} := 0; \mathtt{y} := 0; \textbf{while } \neg(\mathtt{y} = 1) \textbf{ do } \mathtt{I}, \sigma\rangle \\
&\Rightarrow^3 \langle \textbf{while } \neg(\mathtt{y} = 1) \textbf{ do } \mathtt{I}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle && := \text{ rule} \\
&\Rightarrow \langle \mathtt{I}; \mathtt{x} := \mathtt{x} + 1; \mathtt{L}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle && \text{while rule } \mathbf{tt} \\
&\Rightarrow \langle \mathtt{I}'; \mathtt{x} := \mathtt{x} + 1; \mathtt{L}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle && \text{if rule } \mathbf{ff} \\
&\Rightarrow \langle \textbf{skip}; \mathtt{x} := \mathtt{x} + 1; \mathtt{L}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle && \text{if rule } \mathbf{ff} \\
&\Rightarrow \langle \mathtt{x} := \mathtt{x} + 1; \mathtt{L}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle && \text{skip rule} \\
&\Rightarrow^2 \langle \mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle && := \text{ rule} \\
&\Rightarrow \langle \mathtt{I}; \mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle && \text{while rule } \mathbf{tt} \\
&\Rightarrow \langle \mathtt{I}'; \mathtt{x} := \mathtt{x} + 1; \mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle && \text{if rule } \mathbf{ff} \\
&\Rightarrow \langle \textbf{skip}; \mathtt{x} := \mathtt{x} + 1; \mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle && \text{if rule } \mathbf{ff} \\
&\Rightarrow \langle \mathtt{x} := \mathtt{x} + 1; \mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle && \text{skip rule} \\
&\Rightarrow^2 \langle \mathtt{L}, \sigma[\mathtt{x} \mapsto 2, \mathtt{y} \mapsto 0]\rangle && := \text{ rule}
\end{aligned}
$$

We can see that this may go on for some time, and we look at how we may understand the situation more generally. The variable x is incremented by 1 each time we execute the body of the loop, and we check two arithmetic expressions. We look provide a table that monitors the behaviour.

x	$x \cdot x + 2x + 2$	$x \cdot x - 2x + 2$
0	2	2
1	5	1
2	10	2
3	17	5
4	26	10

This allows us to see that both expressions are growing, and our condition is asking for them to be 0. Indeed we note that the first expression is positive for all $x \in \mathbb{N}$ since it is a sum of non-negative numbers including 2, and that the second is also positive for all $x \geq 2$ since that implies $x \cdot x \geq 2x$ and so $x \cdot x - 2x + 2 \geq 0$, and so a solution cannot exist, the program does not terminate, and it keeps updating the state.

Note that for different values of a, b and c the program will behave quite differently and it's not easy to treat a general case.

We may see that our system for executing While programs matches the behaviour we expect to see in cases of non-terminating programs as well.

2.6 Following an execution

We provide one more example of looking at a program and following its states. In particular note how we use a table to keep track of the value of key variables and expressions—the following exercises ask readers to perform similar tasks.

Example 11. Recall the program computing the greatest common divisor of inputs held in m and n from Example 3, returning its output in m=n.

```
while ¬(m=n) do
   if n<m then
      m:=m−n
   else
      n:=n−m;
```

Assume we carry out this program for initial values m = 48 and n = 210.

n < m	m	n
	48	210
ff	48	162
ff	48	114
ff	48	66
ff	48	18
tt	30	18
tt	12	18
ff	12	6
tt	6	6

For more complicated programs we suggest adding an extra column to keep track of which instruction is under consideration, in particular if there are nested loops.

Exercise 15.
For your solution to Exercise 7 for natural numbers

(a) Use the small step semantics for your solution from the previous part for the state $\sigma = \{m \mapsto 3, n \mapsto 4, k \mapsto 7\}$, see Example 7, until you get to a result of the form $\langle \mathbf{skip}, \sigma' \rangle$. You may carry out several transitions at once for the := rule. *Hint: To avoid writing a lot of the code you may want to devise names for various statements that appear in your program.*

(b) Track how executing the program changes the values held in the various variables given the following input states.

(i) $\sigma = \{m \mapsto -6, n \mapsto 8, k \mapsto 4\}$

(ii) $\sigma = \{m \mapsto -6, n \mapsto 9, k \mapsto 2\}$

For both cases give the state after the program terminates, or argue that it does not terminate.

2.7 Computing final states

An alternative way of defining a semantics for While programs is to focus on the final state that is reached upon termination of the program. This provides us with less detail regarding the execution of the program, but allows us to concentrate on what the program computes (rather than how it does so).

We provide a system for this purpose, which is also known as **big step operational semantics**. We provide statements of the form

$$\langle \mathtt{S}, \sigma \rangle \to \Downarrow \sigma'$$

to mean that if we execute the programme S in state σ then after termination we are in state σ'.

Some of the rules we provide look similar to the small step operational semantics.

bc := This rules says that an assignment always terminates with the expected update to the state we are in when we start.

$$\frac{}{\langle \mathtt{x} := \mathtt{a}, \sigma \rangle \to \Downarrow \sigma[x \mapsto \mathcal{A}\ a\ \sigma]}$$

bc skip The skip command terminates without changing the state.

$$\frac{}{\langle \mathbf{skip}, \sigma \rangle \to \Downarrow \sigma}$$

sc ; If executing statement S in the state σ terminates in stateσ', and executing statement S' in that state terminates in the state σ'' then executing statement S;S' in the state σ terminates in the state σ''.

$$\frac{\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma' \quad \langle \mathtt{S'}, \sigma'\rangle \to \Downarrow \sigma''}{\langle \mathtt{S;S'}, \sigma\rangle \to \Downarrow \sigma''}$$

sc if If in the state σ the boolean expression b evaluates to **tt** we execute S and otherwise S', and that allows us to say the following about executing this if statement:

$$\frac{\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma'}{\langle \textbf{if}\ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \textbf{else}\ \mathtt{S'}, \sigma\rangle \to \Downarrow \sigma'}\ \text{if } \mathcal{B}\ \mathtt{b}\ \sigma = \mathbf{tt} \qquad \frac{\langle \mathtt{S'}, \sigma\rangle \to \Downarrow \sigma''}{\langle \textbf{if}\ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \textbf{else}\ \mathtt{S'}, \sigma\rangle \to \Downarrow \sigma''}\ \text{if } \mathcal{B}\ \mathtt{b}\ \sigma = \mathbf{ff}$$

sc while If in the state σ the boolean expression b evaluates to **tt** then if executing the body of the loop terminates and updates the state to σ', and if executing the loop in the state σ' terminates in the state σ'' then executing the loop in the state σ also terminates in the state σ''.

If in the state σ the boolean expression b evaluates to **ff** then the loop has been executed and our program terminates without changing the state.

$$\frac{\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma' \quad \langle \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma'\rangle \to \Downarrow \sigma''}{\langle \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma\rangle \to \Downarrow \sigma''}\ \text{if } \mathcal{B}\ \mathtt{b}\ \sigma = \mathbf{tt} \qquad \frac{}{\langle \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma\rangle \to \Downarrow \sigma}\ \text{if } \mathcal{B}\ \mathtt{b}\ \sigma = \mathbf{ff}$$

Note that when we reason about while loops we reason *backwards*: If we already know that when executing the loop in state σ it terminates in state σ'', and carrying out the body of the loop in state σ terminates in state σ', then executing the loop in state σ also terminates in state σ''. See this in action on page 33.

Exercise 16.
Use the big step semantics to argue that a program that is built from assignment, skip, and if statements using composition terminates for every start state.

2.7.1 The big step operational semantics of While

For convenience we give the whole system on one page, leaving the labels of the recursive definitions implicit.

$$\mathcal{A}\ \mathtt{n}\ \sigma = \mathcal{N}\mathtt{n} \qquad \mathcal{B}\ \mathbf{ff}\ \sigma = \mathbf{ff} \qquad \mathcal{B}\ \mathbf{tt}\ \sigma = \mathbf{tt}$$

$$\mathcal{A}\ \mathtt{x}\ \sigma = \sigma\mathtt{x} \qquad \mathcal{B}(a = a')\ \sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ a\ \sigma = \mathcal{B}\ a'\ \sigma \\ \mathbf{ff} & \text{else.} \end{cases}$$

$$\mathcal{A}(a + a')\ \sigma = (\mathcal{A}\ a\ \sigma) + (\mathcal{A}\ a'\ \sigma) \qquad \mathcal{B}(a \leq a')\ \sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ a\ \sigma \leq \mathcal{B}\ a'\ \sigma \\ \mathbf{ff} & \text{else.} \end{cases}$$

$$\mathcal{A}(a - a')\ \sigma = (\mathcal{A}\ a\ \sigma) - (\mathcal{A}\ a'\ \sigma) \qquad \mathcal{B}(\neg a)\ \sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ a\ \sigma = \mathbf{ff} \\ \mathbf{ff} & \text{else.} \end{cases}$$

$$\mathcal{A}(a \times a')\ \sigma = (\mathcal{A}\ a\ \sigma) \cdot (\mathcal{A}\ a'\ \sigma) \qquad \mathcal{B}(a \wedge a')\ \sigma = \begin{cases} \mathbf{tt} & \mathcal{B}\ a\ \sigma = \mathbf{tt} \text{ and } \mathcal{B}\ a'\ \sigma = \mathbf{tt} \\ \mathbf{ff} & \text{else.} \end{cases}$$

$$\frac{}{\langle \mathtt{x} := \mathtt{a}, \sigma\rangle \to \Downarrow \sigma[x \mapsto \mathcal{A}\ a\ \sigma]} \qquad \frac{}{\langle \textbf{skip}, \sigma\rangle \to \Downarrow \sigma}$$

$$\frac{\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma' \quad \langle \mathtt{S'}, \sigma'\rangle \to \Downarrow \sigma''}{\langle \mathtt{S;S'}, \sigma\rangle \to \Downarrow \sigma''}$$

$$\frac{\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma'}{\langle \textbf{if}\ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \textbf{else}\ \mathtt{S'}, \sigma\rangle \to \Downarrow \sigma'}\ \text{if } \mathcal{B}\ \mathtt{b}\,\sigma{=}\mathbf{tt} \qquad \frac{\langle \mathtt{S'}, \sigma\rangle \to \Downarrow \sigma''}{\langle \textbf{if}\ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \textbf{else}\ \mathtt{S'}, \sigma\rangle \to \Downarrow \sigma''}\ \text{if } \mathcal{B}\ \mathtt{b}\,\sigma{=}\mathbf{ff}$$

$$\frac{\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma' \quad \langle \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma'\rangle \to \Downarrow \sigma''}{\langle \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma\rangle \to \Downarrow \sigma''}\ \text{if } \mathcal{B}\ \mathtt{b}\,\sigma{=}\mathbf{tt} \qquad \frac{}{\langle \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma\rangle \to \Downarrow \sigma}\ \text{if } \mathcal{B}\ \mathtt{b}\sigma{=}\mathbf{ff}$$

2.8 Example: A formal big step computation

Revisiting the extended example from Section 2.2 from above, we may now provide a formal derivation of its behaviour in terms of showing how one would show that after termination, a particular final state is reached.

We use some abbreviations in what follows, namely we use L as a name for the loop part of the programme, that is L is

```
while y≤r do
    d:=d+1;  r:=r−y
```

and B for the body of the loop, that is $\mathtt{d} := \mathtt{d} + 1; \mathtt{r} := \mathtt{r} + 1$. We further use ρ for the state $\{\mathtt{x} \mapsto 10, \mathtt{y} \mapsto 3\}$, and since the two variables x and y do not change as we execute the program we may express every state involved in the computation as an update of ρ where we change the values of the variables r and d, which saves a bit of space.

$$
\dfrac{\langle \mathtt{r} := \mathtt{x}; \mathtt{d} := 0, \sigma\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 10, \mathtt{d} \mapsto 0] \qquad \dfrac{\langle \mathtt{B}, \rho[\mathtt{r} \mapsto 10, \mathtt{d} \mapsto 0]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 7, \mathtt{d} \mapsto 1] \qquad \dfrac{\langle \mathtt{B}, \rho[\mathtt{r} \mapsto 7, \mathtt{d} \mapsto 1]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2] \qquad \dfrac{\langle \mathtt{B}, \rho[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3] \qquad \dfrac{}{\langle \mathtt{L}, \rho[\mathtt{r} \mapsto 3, \mathtt{d} \mapsto 1]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]}}{\langle \mathtt{L}, \rho[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]}}{\langle \mathtt{L}, \rho[\mathtt{r} \mapsto 7, \mathtt{d} \mapsto 1]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]}}{\langle \mathtt{L}, \rho[\mathtt{r} \mapsto 10, \mathtt{d} \mapsto 0]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]}}{\langle \mathtt{r} := \mathtt{x}; \mathtt{d} := 0; \mathtt{L}, \sigma\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]}
$$

To complete the above derivation we would have to apply the composition and assignment rules to the branches that do not begin with an axiom rule, but then the resulting derivation would definitely be too large to display.

For the sake of completeness we show what that would look like for the top-most of these:

$$
\dfrac{\dfrac{}{\langle \mathtt{d} := \mathtt{d} + 1, \rho[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 3]} \qquad \dfrac{}{\langle \mathtt{r} := \mathtt{r} - \mathtt{y}, \rho[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 3]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]}}{\langle \mathtt{d} := \mathtt{d} + 1; \mathtt{r} := \mathtt{r} - \mathtt{y}, \rho[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2]\rangle \to \Downarrow \rho[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]}
$$

As we may see from the previous page, representing the big step semantics via formal derivations is not a very convenient format. Students who have taken COMP11120 *Mathematical Techniques for Computer Science* have seen examples of turning a formal derivation in a tree structure into a linear list, and the same idea works here.

We again look at our running example from Section 2.2, we once more start in the state

$$\sigma = \{m \mapsto 10, n \mapsto 3\},$$

and we again use $\mathtt{B}$ for the body of the loop.

1. $\langle \mathtt{r} := \mathtt{m}, \sigma\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 10]$ by the := rule.
2. $\langle \mathtt{d} := 0, \sigma[\mathtt{r} \mapsto 10]\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 10, \mathtt{d} \mapsto 0]$ by the := rule.
3. $\langle \textbf{while } \mathtt{n} \leq \mathtt{r} \textbf{ do } \mathtt{B}, \sigma[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]$ by the while rule **ff**.
4. $\langle \mathtt{d} := \mathtt{d} + 1, \mathtt{r} := \mathtt{r} - \mathtt{n}, \sigma[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2]\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]$ by the := rule.
5. $\langle \textbf{while } \mathtt{n} \leq \mathtt{r} \textbf{ do } \mathtt{B}, \sigma[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2]\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]$ by the while rule **tt**, 3 and 4.
6. $\langle \mathtt{d} := \mathtt{d} + 1, \mathtt{r} := \mathtt{r} - \mathtt{n}, \sigma[\mathtt{r} \mapsto 7, \mathtt{d} \mapsto 1]\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 4, \mathtt{d} \mapsto 2]$ by the := rule.
7. $\langle \textbf{while } \mathtt{n} \leq \mathtt{r} \textbf{ do } \mathtt{B}, \sigma[\mathtt{r} \mapsto 7, \mathtt{d} \mapsto 1]\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]$ by the while rule **tt**, 5 and 6.
8. $\langle \mathtt{d} := \mathtt{d} + 1, \mathtt{r} := \mathtt{r} - \mathtt{n}, \sigma[\mathtt{r} \mapsto 10, \mathtt{d} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 7, \mathtt{d} \mapsto 1]$ by the := rule.
9. $\langle \textbf{while } \mathtt{n} \leq \mathtt{r} \textbf{ do } \mathtt{B}, \sigma[\mathtt{r} \mapsto 10, \mathtt{d} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]$ by the while rule **tt**, 7 and 8.
10. $\langle \mathtt{r} := \mathtt{m}; \mathtt{d} := 0 \textbf{ while } \mathtt{n} \leq \mathtt{r} \textbf{ do } \mathtt{B}, \sigma\rangle \to \Downarrow \sigma[\mathtt{r} \mapsto 1, \mathtt{d} \mapsto 3]$ by the ; rule applied to 1, 2, and 9.

We recommend comparing the above account of the big step semantics with the small step semantics given in Section 2.4. In order to reason about the while loop in the big step semantics we need to know which state the program will be in when the loop terminates, and that information is readily available from the small step transitions, and has to be worked out going forward, rather than going backwards as is required for the big step semantics.‘

Exercise 17.
For your solution to Exercise 7 for natural numbers find the final state when the start state is $\sigma = \{m \mapsto= 3, n \mapsto 4, k \mapsto 7\}$ by using the big step semantics. Compare your answer to that from part (a) of Exercise 15

Exercise 18.
For the program determining the greatest common divisor gcd of two inputs held in m and n from Example 3, compute the result using the big step semantics for a state σ with $\sigma\mathtt{m} = 12$ and $\sigma\mathtt{n} = 30$. Make sure to justify each step. Compare your answer to that for Exercise 14.

Exercise 19.
Assume that σ is a state such that $\sigma\,\mathtt{i} \geq 1$. For the following two programs

```
y := 1;  a := 0;
while y≤i do
    a := a+y;
    y := y+1;
```

```
x := i×(i+1);  y := 0;
while 0≤x do
    x := x−2;
    y := y+1
```

carry out the following tasks:

(a) Carry out some steps using the small step operational semantics from the state σ until you can see what the programs do.

(b) Sketch how we could use the big step semantics to find a final state for the program on the left when started in the state σ.

(c) The two programs may be thought of as carrying out the same computation. Formalize this via a statement using the big step semantics when starting from the state σ both times.

Exercise 20.
Describe the circumstances under which we may have

$$\langle \textbf{while } \mathtt{b} \textbf{ do } \mathtt{S}, \sigma\rangle \to \Downarrow \tau$$

for a boolean expression b, a statement S and states σ and τ.

Exercise 21.
Argue that your solution to Exercise 4 behaves in the same way as the original if statement

(a) using the small step semantics;[a]

(b) using the big step semantics.

[a] Note that this task is not straightforward.

Exercise 22.
In Exercise 3 we looked at how one would implement a 'repeat until' loop in While. Imagine now the grammar of While was changed to include a statement of the form `repeat S until b` rather than while loops.

(a) What should rules for a stepwise operational semantics for this statement look like?

(b) What should the rules for a big step operational semantics for this statement look like?

(c) Argue that your solution to Exercise 3 has the same behaviour in the big step semantics as the repeat until loop under your rule for the previous part. *Hint: This needs to be an induction over the number of times the body of the loop is executed.*

We note that for every statement S and every state σ there exists at most one state σ' such that

$$\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma'.$$

Exercise 23.
Justify the claim preceding this exercise.

Note that strictly speaking, we *anticipated* that the composition operator ; is associative, since we didn't introduce brackets for our syntax. We need to ensure that our notion of computation behaves appropriately. This is not necessary for the small step semantics since that enforces carrying out one step at a time.

Exercise 24.
Show that the composition of statements is associative for our notion of computation. Begin by thinking about how you may phrase this statement using the big step operational semantics and then show it. *Hint: You'll need brackets.*

You may wonder whether there is a connection between the small and the big step semantics. The answer is yes, see Exercise 26.

For us to form a connection between the two we first of all have to form the *reflexive transitive closure* of the relation $\Rightarrow$, and establish a property.

Exercise 25.
Show that if $\langle \mathtt{S;S'}, \sigma\rangle \Rightarrow^n \langle \mathbf{skip}, \sigma'\rangle$ then there exist natural numbers m and m' with

$$n = m + m' + 1$$

as well as a state τ such that

$$\langle \mathtt{S}, \sigma\rangle \Rightarrow^m \langle \mathbf{skip}, \tau\rangle \qquad \text{and} \qquad \langle \mathtt{S'}, \tau\rangle \Rightarrow^{m'} \langle \mathbf{skip}, \sigma'\rangle.$$

The following exercise provides a statement that connects the small step with the big step semantics formally: It says that if when starting in state σ a While program S terminates in state σ', then when we carry out the small step semantics for S from the state σ we eventually end up in the state σ' with no further instructions to be carried out.

Exercise 26.
Give an argument that, given a statement S and two states σ and σ', we have

$$\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma'$$

if and only if for the reflexive transitive closure $\Rightarrow^*$ of the transition relation $\Rightarrow$ we have that

$$\langle \mathtt{S}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle.$$

2.9 Moving beyond individual states

Above we provide an abstract notion of computation based on following each step of the computation: A program in the presence of a state transitions to a the remainder of the program after some steps have been carried out, together with the updated state. This kind of semantics is sometimes referred to as an **operational semantics**.

A reasonably obvious thought is the following: We may think of a While program as taking a given state, and producing at most one (but possibly none) state that reflects how the values held in variables have changed as the program was executed.

This suggests the idea that we may think of a While program S as describing a partial function from St to St. Given an input $\sigma \in \mathsf{St}$ this partial function

- outputs σ' if $\langle \mathtt{S}, \sigma \rangle \Rightarrow \sigma'$ and
- is undefined otherwise.

It is indeed possible to make this idea a rigorous definition. This gives rise to what is called a **denotational semantics** for our language, that is, a way of assigning a mathematical object to each program in such a way that reasoning about the mathematical objects in question allows us to draw conclusions about programs. Unfortunately making this precise requires a mathematical machinery that we don't have the scope to set up.

In order to describe the interpretation of a loop requires us to be able to compute the state we are in when the loop terminates, and it is this that creates mathematical issues that need to be overcome.

A nice property of the denotational semantics is that statement composition is interpreted by composition of partial functions, that is the interpretation of `S;S'` is the interpretation of `S` followed by that of `S'`.

The advantage of an *operational semantics* is that we may define it without going outside of the language, with minimal infrastructure (here in the form of states), and which still allows us to prove properties of programs in our language. In contrast, a *denotational semantics* typically requires significant mathematical machinery. However, it then allows us to reason about mathematical objects instead of syntactic ones, and typically allows us to prove more powerful theorems about our language. The idea of a denotational semantics is pursued in COMP31311 *Giving Meaning to Programs* for functional languages.

In Chapter 4 we extend our analysis of the input/output behaviour by finding an alternative method of moving away from individual states.

2.10 Summary

This chapter provides a formal notion of computation for While programs. It does so based on the idea that a computation manipulates the state, that is the values stored by the available variables. This makes it possible to make rigorous statements about the input/output behaviour of programs. A number of case studies show that this notion of computation matches the intuition gained from real world programs.

2.11 Solutions to Exercises

Solution 11.
We look at both cases.

(a) Assume that σ is a state. We check $\mathcal{B}$ applied to $\neg(\neg\mathtt{a}\&\neg\mathtt{b})$ in the state σ.

$$\begin{aligned}
\mathcal{B}\,(\neg(\neg\mathtt{a}\&\neg\mathtt{b})\,)\,\sigma &= \begin{cases} \mathbf{tt} & \mathcal{B}\ (\neg\mathtt{a} \wedge \neg\mathtt{b})\,\sigma = \mathbf{ff} \\ \mathbf{ff} & \text{else} \end{cases} \\
&= \begin{cases} \mathbf{ff} & \mathcal{B}\ \neg\mathtt{a}\ \ \sigma = \mathbf{tt} \text{ and } \mathcal{B}\ \neg\mathtt{b}\ \ \sigma = \mathbf{tt} \\ \mathbf{tt} & \text{else} \end{cases} \\
&= \begin{cases} \mathbf{ff} & \mathcal{B}\ \mathtt{a}\,\sigma = \mathbf{ff} \text{ and } \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{ff} \\ \mathbf{tt} & \text{else} \end{cases} \\
&= \begin{cases} \mathbf{tt} & \mathcal{B}\ \mathtt{a}\,\sigma = \mathbf{tt} \text{ or } \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{tt} \\ \mathbf{ff} & \text{else} \end{cases}
\end{aligned}$$

which is the behaviour we expect for the disjunction operator.

(b) Again we check what happens when we evaluate $\neg(\mathtt{a}\&\neg\mathtt{b})$ for a state σ.

$$\begin{aligned}
\mathcal{B}\,(\neg(\mathtt{a}\&\neg\mathtt{b})\,)\,\sigma &= \begin{cases} \mathbf{tt} & \mathcal{B}\,(\mathtt{a} \wedge \neg\mathtt{b})\,\sigma = \mathbf{ff} \\ \mathbf{ff} & \text{else} \end{cases} \\
&= \begin{cases} \mathbf{ff} & \mathcal{B}\ \mathtt{a}\,\sigma = \mathbf{tt} \text{ and } \mathcal{B}\ \neg\mathtt{b}\ \ \sigma = \mathbf{tt} \\ \mathbf{tt} & \text{else} \end{cases} \\
&= \begin{cases} \mathbf{ff} & \mathcal{B}\ \mathtt{a}\,\sigma = \mathbf{tt} \text{ and } \mathcal{B}\ \mathtt{b}\,\sigma = \mathbf{ff} \\ \mathbf{tt} & \text{else} \end{cases}
\end{aligned}$$

This reflects the truth table for implication as required.

Solution 12.
Looking at the rules we may see that we obtain a unique state for rules $:=$, if and while, and this is also true for the ; rule for skip.

That leaves the other ; rule. We may now carry out a proof by induction over the number of times this rule is invoked.

In the base case, the rule is invoked once and the reason why $\langle \mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S'}, \sigma'\rangle$ is the use of one of the other rules, in which case we have already argued that σ' is uniquely determined.

The induction hypothesis is that the state is unique when we invoke this rule n times.

When we invoke this rule the $(n+1)$th time, we are relying on having

$$\langle \mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S'}, \sigma'\rangle$$

for suitable statements S and S', and by the induction hypothesis, σ' is uniquely determined.

Hence the transition we make, which is of the form

$$\langle \mathtt{S;S''}, \sigma\rangle \Rightarrow \langle \mathtt{S';S''}, \sigma'\rangle$$

takes us to the uniquely determined state σ'.

Solution 13.
We may see that only one rule tells us to change the value of a variable, namely the $:=$ rule. Hence only the values of variables mentioned in the program can be changed as we update our state.

A formal proof of this statement is a proof by induction over the derivation that witnesses the assumption that $\langle \mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S'}, \sigma'\rangle$.

We provide all the required cases.

bc $:=$ This rules asks us to update the state as expected when assigning an arithmetic expression a to a variable y:

$$\langle \mathtt{y} := \mathtt{a}, \sigma\rangle \Rightarrow \langle \textbf{skip}, \sigma[y \mapsto \mathcal{A}\ a\,\sigma]\rangle$$

By assumption we know that $\mathtt{y} \neq \mathtt{x}$ and

$$(\sigma[\mathtt{y} \mapsto \mathcal{A}\ a\,\sigma])\,\mathtt{x} = \sigma\mathtt{x},$$

confirming that the value of the updated state at x does not change.

bc skip The skip command does not come with any computation steps to be carried out and there is no rule here.

sc ; The rule tells us that

$$\frac{\langle \mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S'}, \sigma'\rangle}{\langle \mathtt{S;S''}, \sigma\rangle \Rightarrow \langle \mathtt{S';S''}, \sigma'\rangle}$$

and we have that $\sigma'\mathtt{x} = \sigma\mathtt{x}$ by the induction hypothesis.

The corresponding rule tells us that

$$\langle \textbf{skip}\,;\mathtt{S}, \sigma\rangle \Rightarrow \langle \mathtt{S}, \sigma\rangle$$

and since the state remains the same, the value of the variable x remains the same as well.

sc if The rule tells us that

$$\langle\, \textbf{if}\ \ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \ \textbf{else}\ \ \mathtt{S'}, \sigma\rangle \Rightarrow \begin{cases} \langle \mathtt{S}, \sigma\rangle & \mathcal{B}\ \mathtt{b}\,\sigma = \textbf{tt} \\ \langle \mathtt{S'}, \sigma\rangle & \text{else,} \end{cases}$$

and since the state does not change, the value of the variable x remains the same.

sc while The rule for loops is

$$\langle \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma\rangle \Rightarrow \begin{cases} \langle \mathtt{S};\textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma\rangle & \mathcal{B}\ \mathtt{b}\,\sigma = \textbf{tt} \\ \langle \textbf{skip}, \sigma\rangle & \text{else,} \end{cases}$$

and again there is no change to the state.

Note that the induction hypothesis is only required for the composition rule, since the other transitions change the statement rather than the state.

Solution 14.
We assume we have a state σ with $\sigma\mathtt{m} = 12$ and $\sigma\mathtt{n} = 30$. We use L for the loop and I for the if statement.

$$
\begin{array}{ll}
\langle \mathtt{x} := \mathtt{m};\ \mathtt{y} := \mathtt{n};\ \mathtt{L}, \sigma\rangle & \\
\quad \Rightarrow^2 \langle \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 30]\rangle & := \text{rule} \\
\quad \Rightarrow \langle \mathtt{I};\ \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 30]\rangle & \text{while rule } \textbf{tt} \\
\quad \Rightarrow \langle \mathtt{y} := \mathtt{y} - \mathtt{x};\ \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 30]\rangle & \text{if rule } \textbf{ff} \\
\quad \Rightarrow \langle \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 18]\rangle & := \text{rule} \\
\quad \Rightarrow \langle \mathtt{I};\ \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 18]\rangle & \text{while rule } \textbf{tt} \\
\quad \Rightarrow \langle \mathtt{y} := \mathtt{y} - \mathtt{x};\ \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 18]\rangle & \text{if rule } \textbf{ff} \\
\quad \Rightarrow \langle \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 6]\rangle & := \text{rule} \\
\quad \Rightarrow \langle \mathtt{I};\ \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 6]\rangle & \text{while rule } \textbf{tt} \\
\quad \Rightarrow \langle \mathtt{x} := \mathtt{x} - \mathtt{y};\ \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 6]\rangle & \text{if rule } \textbf{tt} \\
\quad \Rightarrow \langle \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]\rangle & := \text{rule} \\
\quad \Rightarrow \langle \textbf{skip}, \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]\rangle & \text{while rule } \textbf{ff}
\end{array}
$$

Solution 15.
We solve each part.

For convenience we repeat the program here.

```
x := 0;  y := 0;
while ¬(m×x+n×y=k) do
    x := x+1;  y := 0;
        while ¬(m×x+n×y=k)∧y<x do
            if m×y+n×x=k then
                z := x;  x := y;  y := z
            else
                y := y+1
```

(a) We carry out the steps as indicated. We use L for the outer and L' for the inner loop, and I for the if statement which forms the body of the inner loop.

$$
\begin{array}{ll}
\langle \mathtt{x} := 0;\ \mathtt{y} := 0;\ \mathtt{L}, \sigma\rangle \Rightarrow^2 \langle \mathtt{y} := 0;\mathtt{L}, \sigma[\mathtt{x} \mapsto 0]\rangle & := \text{rule} \\
\quad \Rightarrow^2 \langle \mathtt{L}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle & := \text{rule} \\
\quad \Rightarrow \langle \mathtt{x} := \mathtt{x} + 1;\ \mathtt{y} := 0;\mathtt{L'};\mathtt{L}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle & \text{while rule } \textbf{tt} \\
\quad \Rightarrow^4 \langle \mathtt{L'};\mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle & := \text{rule} \\
\quad \Rightarrow \langle \mathtt{I};\mathtt{L'};\mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle & \text{while rule } \textbf{tt} \\
\quad \Rightarrow \langle \mathtt{y} := \mathtt{y} + 1;\mathtt{L};\mathtt{L'}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]\rangle & \text{if rule } \textbf{ff} \\
\quad \Rightarrow^2 \langle \mathtt{L'};\mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]\rangle & := \text{rule} \\
\quad \Rightarrow \langle \textbf{skip};\mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]\rangle & \text{while rule } \textbf{ff} \\
\quad \Rightarrow \langle \textbf{skip}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]\rangle & \text{while rule } \textbf{ff}
\end{array}
$$

(b) We follow the instructions, using the same names as in the previous part.

1. $\langle \mathtt{x} := 0;\ \mathtt{y} := 0, \sigma\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]$ by the := rule used twice.
2. $\langle \mathtt{L'}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by the **ff** version of the while rule.
3. $\langle \mathtt{y} := \mathtt{y} + 1, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by the := rule.
4. $\langle \mathtt{I}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by the **ff** version of the if rule.
5. $\langle \mathtt{L'}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by 2, 3 and the **tt** version of the while rule.
6. $\langle \mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by the **ff** version of the while rule.
7. $\langle \mathtt{x} := \mathtt{x} + 1;\ \mathtt{y} := 0, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]$ by the := rule twice.
8. $\langle \mathtt{x} := \mathtt{x} + 1;\ \mathtt{y} := 0;\ \mathtt{L'}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by 6, 5 and the ; rule.
9. $\langle \mathtt{L}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by 7, 5 and the **tt** version of the while rule.
10. $\langle \mathtt{x} := 0;\ \mathtt{y} := 0;\ \mathtt{L}, \sigma\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by 1, 8 and the ; rule.

(c) The two situations unfold in different ways:

(i)

We may summarize what happens by keeping track of the variables x and y as well as what happens to the boolean expressions $-6 \times \mathtt{x} + 8 \times \mathtt{y}$ and $-6 \times \mathtt{y} + 8 \times \mathtt{x}$, one of which needs to become 4 for the program to terminate. In each line we first check the boolean expression that governs the loop or if statement, and we then update the variables as the program determines.

	$-6 \times \mathtt{x} + 8 \times \mathtt{y}$	$-6 \times \mathtt{y} + 8 \times \mathtt{x}$	x	y
start			0	0
outer loop	0		1	
inner loop	-6			
if statement		8		1
outer loop	2		2	0
inner loop	-12			
if statement		16		1
inner loop	-4			
if statement		10		2
inner loop		4		

The final state is $\{\mathtt{m} \mapsto -6, \mathtt{n} \mapsto 8, \mathtt{k} \mapsto 4, \mathtt{x} \mapsto 2, \mathtt{y} \mapsto 2\}$, and indeed $-6 \cdot 2 + 8 \cdot 2 = -12 + 16 = 4$.

(ii) The program starts much as in the previous part, and the critical condition is now concerned with $-6 \cdot 0 + 9 \cdot 0$, and as we keep increasing the values of the variables x and y we want to reach the number 2. We give a table that keeps track of the variables x and y and the boolean condition similar to the previous part.

	$-6 \times x + 9 \times y$	$-6 \times y + 9 \times x$	x	y
start			0	0
outer loop	0		1	
inner loop	−6			
if statement		9	1	
outer loop	3		2	0
inner loop	−12			
if statement		18		1
inner loop	3			
if statement		3		2
outer loop	12		3	0
inner loop	−18			
if statement		27		1
inner loop	−9			
if statement		21		2
inner loop	0			
if statement		15		3
outer loop	9		4	0
inner loop	−24			
if statement		36		1
inner loop	−15			
if statement		30		2
inner loop	−6			
if statement		24		3
inner loop	3			
if statement		18		4
outer loop	12		5	0
inner loop	−30			
if statement		45		1
inner loop	−21			
if statement		39		2
inner loop	−12			
if statement		33		3
inner loop	−3			
if statement		27		4
inner loop	6			
if statement		21		5

This is getting a bit tedious, and if the program does not terminate we could keep going forever with this! We look instead at whether there's something we may learn from the work carried out so far. A few observations seem useful:

- The arithmetic expression $-6 \times x + 9 \times y$ goes up by 9 each time the value of y is incremented, and it goes down by 6 when x is incremented if we compare the two for y being 0.
- The arithmetic expression $-6 \times y + 9 \times x$ goes down by 6 each time the value of y is incremented, and it goes up by 9 each time when the value of x is incremented if we compare the two for y being 0.
- We may use these observation to note that all numbers the arithmetic expressions evaluate to are multiples of 3: Indeed, 3 divides both, -6 and 9, and so it divides $-6 \cdot x + 9 \cdot y$ for all $x, y \in \mathbb{N}$. 3 does not divide 2, and so the equation has no solution, which means our program keeps going forever.

Solution 16.
Inspecting the rules for the big step semantics we observe the following:

- assignment and skip statements terminate for every start state;
- since a boolean expression always evaluates to either **tt** or **ff**, an if statement terminates for every start state.

This leaves us with composition of statements. We may prove the desired result by induction over the number of composition symbols used.

If there are no composition symbols then our program has to be one of the statements examined above, and these statement terminate for every start state.

Assume there are $n+1$ many composition symbols. Then we know the statement resulting from the first n many composition statements terminates for every start state. But the final statement must now again be one of those examined above, which means it terminates for whatever state the first n statements take us to.

Solution 17.
For convenience we repeat the program here.

```
x := 0;  y := 0;
while ¬(m×x+n×y=k) do
    x := x+1;  y := 0;
    while ¬(m×x+n×y=k)∧y≤x do
            if m×y+n×x=k then
                z := x;  x := y;  y := z
            else
                y := y+1
```

In the following we use L for the outer and L' for the inner loop and I for the if statement.

1. $\langle \mathtt{x} := 0;\ \mathtt{y} := 0, \sigma\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]$ by the := rule used twice.
2. $\langle \mathtt{L'}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by the **ff** version of the while rule.
3. $\langle \mathtt{y} := \mathtt{y} + 1, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by the := rule.
4. $\langle \mathtt{I}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by the **ff** version of the if rule.
5. $\langle \mathtt{L'}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by 2, 3 and the **tt** version of the while rule.
6. $\langle \mathtt{L}, \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by the **ff** version of the while rule.
7. $\langle \mathtt{x} := \mathtt{x} + 1;\ \mathtt{y} := 0, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 0]$ by the := rule twice.
8. $\langle \mathtt{x} := \mathtt{x} + 1;\ \mathtt{y} := 0;\ \mathtt{L'}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by 6, 5 and the ; rule.
9. $\langle \mathtt{L}, \sigma[\mathtt{x} \mapsto 0, \mathtt{y} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by 7, 5 and the **tt** version of the while rule.
10. $\langle \mathtt{x} := 0;\ \mathtt{y} := 0;\ \mathtt{L}, \sigma\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 1, \mathtt{y} \mapsto 1]$ by 1, 8 and the ; rule.

We note that the final state is the same as that reached in part (a) of Exercise 15.

Solution 18.
We assume we have a state σ with $\sigma\mathtt{m} = 12$ and $\sigma\mathtt{n} = 30$. We use L for the loop and I for the if statement.

1. $\langle \mathtt{x} := \mathtt{m};\ \mathtt{y} := \mathtt{n}, \sigma\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 30]$ by the := rule.

2. $\langle \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]$ by the **ff** version of the while rule.

3. $\langle \mathtt{x} := \mathtt{x} - \mathtt{y}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 6]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]$ by the := rule.

4. $\langle \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 6]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]$ by 3 and the **tt** version of the if rule.

5. $\langle \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 6]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]$ by 4, 2 and the **tt** version of the while rule.

6. $\langle \mathtt{y} := \mathtt{y} - \mathtt{x}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 18]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 6]$ by the := rule.

7. $\langle \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 18]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 6]$ by 6 and the **ff** version of the if rule.

8. $\langle \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 18]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]$ by 5, 7 and the **tt** version of the while rule.

9. $\langle \mathtt{y} := \mathtt{y} - \mathtt{x}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 30]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 18]$ by the := rule.

10. $\langle \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 30]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 18]$ by 9 and the **ff** version of the if rule.

11. $\langle \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}, \sigma[\mathtt{x} \mapsto 12, \mathtt{y} \mapsto 30]\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto 6, \mathtt{y} \mapsto 6]$ by 8, 10 and the **tt** version of the while rule.

We note that the final state is the same as that we reached in Exercise 14.

Solution 19.
We think of the variable $\mathtt{i}$ as providing an input to the program.

(a) We look at the transitions for the program on the left. We use $\mathtt{L}$ for the loop and $\mathtt{B}$ for its body. Assume we start in state σ.

$$
\begin{aligned}
&\langle \mathtt{y} := 1;\ \mathtt{a} := 0;\mathtt{L}, \sigma\rangle \\
&\quad \Rightarrow^3 \langle \mathtt{L}, \sigma[\mathtt{a} \mapsto 0, \mathtt{y} \mapsto 1]\rangle && := \text{ rule} \\
&\quad \Rightarrow \langle \mathtt{B};\textbf{while}\ \mathtt{y} \le \mathtt{i}\ \textbf{do}\ \mathtt{B}, \sigma[\mathtt{a} \mapsto 1, \mathtt{y} \mapsto 2]\rangle && \text{while rule } \textbf{tt}, :=\text{rule} \\
&\quad \Rightarrow^3 \langle \textbf{while}\ \mathtt{y} \le \mathtt{i}\ \textbf{do}\ \mathtt{B}, \sigma[\mathtt{a} \mapsto 3, \mathtt{y} \mapsto 3]\rangle && := \text{ rule}
\end{aligned}
$$

We may see that potentially the condition governing the loop will be false now. More generally, the loop keeps running until $\mathtt{y}=\mathtt{i}$, and when that happens, $\mathtt{a}$ will be equal to

$$\sum_{y=1}^{\mathtt{i}} y,$$

that is, when the program terminates a will be the sum of the first i positive integers and y will be equal to i.

For the program on the right, using L' for the loop and B' for its body, we note that

$$\begin{aligned}
&\langle \mathtt{x} := \mathtt{i} \times (\mathtt{i}+1); \mathtt{y} := 0; \mathtt{L'}, \sigma\rangle \\
&\quad \Rightarrow^3 \langle \mathtt{L'}, \sigma[\mathtt{x} \mapsto i \cdot (i+1), \mathtt{y} \mapsto 0]\rangle && := \text{rule} \\
&\quad \Rightarrow \langle \mathtt{B'}; \mathtt{L'}, \sigma[\mathtt{x} \mapsto i \cdot (i+1), \mathtt{y} \mapsto 0]\rangle && \text{while rule } \mathbf{tt}, := \text{rule} \\
&\quad \Rightarrow^3 L'\sigma[\mathtt{x} \mapsto x-2, \mathtt{y} \mapsto 1] && \text{while rule } \mathbf{tt}, := \text{rule}
\end{aligned}$$

Again we note that the condition governing the loop may potentially be false now. The loop is repeated $i \cdot (i+1)/2$ many times, and then the value in y will be equal to $i \cdot (i+1)/2$ while the value in x will be 0. This is also the sum over the first i many integers, so in this sense we may think of the two programs as carrying out the same computation (although the output is reported in different variables).

(b) In the big step semantics one would argue as follows:

1. $\langle \mathtt{y} := 1;\ \mathtt{a} := 0, \sigma\rangle \to \Downarrow \sigma[\mathtt{y} \mapsto 1, \mathtt{a} \mapsto 0]$ by the := rule.
2. $\langle \textbf{while } \mathtt{y} \leq \mathtt{i} \textbf{ do } \mathtt{B}, \sigma\rangle \to \Downarrow \sigma[\mathtt{y} \mapsto \sigma\mathtt{i}+1, \mathtt{a} \mapsto \sum_{n=1}^{\sigma\mathtt{i}} n]\sigma[\mathtt{y} \mapsto \sigma\mathtt{i}+1, \mathtt{a} \mapsto \sum_{n=1}^{\sigma\mathtt{i}} n]$ by the **ff** version of the while rule.
3. $\langle \mathtt{B}, \sigma[\mathtt{y} \mapsto \sigma\mathtt{i}, \mathtt{a} \mapsto \sum_{n=1}^{\sigma\mathtt{i}-1}]\rangle \to \Downarrow \sigma[\mathtt{y} \mapsto \sigma\mathtt{i}+1, \mathtt{a} \mapsto \sum_{n=1}^{\sigma\mathtt{i}} n]$ by the := rule.
4. $\langle \textbf{while } \mathtt{y} \leq \mathtt{i} \textbf{ do } \mathtt{B}, \sigma[\mathtt{y} \mapsto \sigma\mathtt{i}, \mathtt{a} \mapsto \sum_{n=1}^{\sigma\mathtt{i}-1} n]\rangle \to \Downarrow \sigma[\mathtt{y} \mapsto \sigma\mathtt{i}+1, \mathtt{a} \mapsto \sum_{n=1}^{\sigma\mathtt{i}} n]$ by the 2, 3 and the **tt** version of the while rule.

One now has to keep generating statements like 2, 3 and 4 until one reaches

$\langle \textbf{while } \mathtt{y} \leq \mathtt{i} \textbf{ do } \mathtt{B}, \sigma[\mathtt{y} \mapsto 1, \mathtt{a} \mapsto 0]\rangle \to \Downarrow \sigma[\mathtt{y} \mapsto \sigma\mathtt{i}+1, \mathtt{a} \mapsto \sum_{n=1}^{\sigma\mathtt{i}} n]$ and it is possible to argue using the composition rule for this statement and 1 to obtain the result

$\langle \mathtt{y} := 1;\ \mathtt{a} := 0;\ \textbf{while } \mathtt{y} \leq \mathtt{i} \textbf{ do } \mathtt{B}, \sigma\rangle \to \Downarrow \sigma[\mathtt{y} \mapsto \sigma\mathtt{i}+1, \mathtt{a} \mapsto \sum_{n=1}^{\sigma\mathtt{i}} n]$

(c) We may make this observation formal by stating that P is the program on the left, P' is the program on the right, and if σ, σ' and σ'' are states then if

$$\langle \mathtt{P}, \sigma\rangle \to \Downarrow \sigma' \qquad \text{and} \qquad \langle \mathtt{P'}, \sigma\rangle \to \Downarrow \sigma''$$

then $\sigma'\mathtt{a} = \sigma''\mathtt{y}$.

Solution 20.

We may see that only the final two rules apply in this situation. The second of these tells us that the loop does not change the current state if the boolean expression b evaluates to **ff** in that state.

The first rule seems almost circular at first sight! But what the two rules together say is that we get the transition given if and only if there is a number $n \in \mathbb{N}$ such that $\sigma_0 = \sigma$, $\langle \mathtt{S}, \sigma_i\rangle \to \Downarrow \sigma_{i+1}$ for $0 \leq i < n$, $\sigma_n = \tau$ and $\mathcal{B}\ b\ \sigma_n = \mathbf{ff}$.

In other words, the body of the loop is executed a finite number of times (given by $n \in \mathbb{N}$ in the formal description above), and as this happens, the state keeps getting updated until the condition governing the loop evaluates to false, at which point the state we have reached must be equal to τ.

Solution 21.
The question doesn't make it precise what is required to carry out the task, and part of this exercise is for students to think about what they should establish.

(a) We first look at the small step semantics.

We know that a statement of the form **if** b **then** S **else** S' when executed in the state σ behaves as follows:

$$\langle \textbf{if}\ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \textbf{else}\ \mathtt{S'}, \sigma\rangle \Rightarrow \begin{cases} \langle \mathtt{S}, \sigma\rangle & \mathcal{B}\ b\ \sigma = \mathbf{tt} \\ \langle \mathtt{S'}, \sigma\rangle & \text{else} \end{cases}$$

We expect to be able to establish that our code shows a similar behaviour, but potentially requiring more than one transition.

Referring to the solution for that exercise, we use the following names for code:

- L for **while** $\mathtt{b} \wedge \mathtt{x} = 0$ **do** $\mathtt{x} := 1$; S and
- L' for **while** $\neg\mathtt{b} \wedge \mathtt{x} = 0$ **do** $\mathtt{x} := 1$; S'.

For our program we now calculate

$$\langle \mathtt{x} := 0;\mathtt{L};\mathtt{L'}, \sigma\rangle \Rightarrow \langle \mathtt{L};\mathtt{L'}, \sigma[x \mapsto 0]\rangle \qquad \text{; rule}$$

There is now a case distinction.

We first look at the case where $\mathcal{A}\ \mathtt{b}\ \sigma = \mathbf{tt}$. In that situation we have

$$\mathcal{A}\,(\mathtt{b} \wedge x = 0)\,(\sigma[x \mapsto 0]) = \mathbf{tt},$$

so

$$\begin{aligned} \langle \mathtt{L};\mathtt{L'}, \sigma[x \mapsto 0]\rangle &\Rightarrow \langle \mathtt{x} := 1;\mathtt{S};\mathtt{L};\mathtt{L'}, \sigma[x \mapsto 0]\rangle && \text{while rule} \\ &\Rightarrow \langle \mathtt{S};\mathtt{L};\mathtt{L'}, \sigma[x \mapsto 1]\rangle && := \text{ rule} \end{aligned}$$

We further not the following:

- We know that the variable x is not mentioned in S, so the execution of this statement will proceed as it would do without having updated this variable;
- the value of the variable x is not changed during this execution;
- if the execution of S does not terminate that is the same behaviour as we would have seen in the original if statement;
- otherwise the loop conditions for both, L and L' evaluate to $\mathbf{ff}$ when these statements are reached in the execution since the value in x is 1.

We now look at the else case, that is $\mathcal{A}\ \mathtt{b}\ \sigma = \mathbf{ff}$, which means that

$$\mathcal{A}\,(\mathtt{b} \wedge x = 0)\,(\sigma[x \mapsto 0]) = \mathbf{ff}.$$

In that situation again by the while rule we have

$$\begin{aligned}
\langle \texttt{L;L'}, \sigma[x \mapsto 0]\rangle &\Rightarrow \langle \textbf{skip}\,;\texttt{L'}, \sigma[x \mapsto 0]\rangle && \text{while rule}\\
&\Rightarrow \langle \textbf{while}\ \neg\texttt{b} \wedge \texttt{x} = 0\ \textbf{do}\ \texttt{x} := \texttt{x} + 1;\texttt{S'}, \sigma[x \mapsto 0]\rangle && \text{skip ; rule}\\
&\Rightarrow \langle \texttt{x} := 1;\texttt{S'};\texttt{L'}, \sigma[x \mapsto 0]\rangle && \text{while rule}\\
&\Rightarrow \langle \texttt{S'};\texttt{L'}, \sigma[x \mapsto 1]\rangle
\end{aligned}$$

We may make similar observations to before: We further not the following:

- We know that the variable x is not mentioned in S', so the execution of this statement will proceed as it would do without having updated this variable;
- the value of the variable x is not changed during this execution;
- if the execution of S' does not terminate that is the same behaviour as we would have seen in the original if statement;
- otherwise the loop condition L' evaluates to **ff** when these statements are reached in the execution since the value in x is 1.

Hence in both situations the computation is proceeding as it would for the original if statement, with the exception of the value of the variable x.

(b) We now look at the big step semantics. Let σ be an arbitrary state. We know that **if** b **then** S **else** S' shows the following behaviour:

If $\mathcal{B}\ \texttt{b}\ \sigma = \textbf{tt}$ then $\langle$ **if** b **then** S **else** S', $\sigma\rangle \to\Downarrow \sigma'$ precisely when $\langle \texttt{S}, \sigma\rangle \to\Downarrow \sigma'$, and if $\mathcal{B}\ \texttt{b}\ \sigma = \textbf{ff}$ then $\langle$ **if** b **then** S **else** S', $\sigma\rangle \to\Downarrow \sigma'$ precisely when $\langle \texttt{S'}, \sigma\rangle \to\Downarrow \sigma'$.

We now examine the behaviour or our proposed program. We use

- L for the loop **while** $\texttt{b} \wedge \texttt{x} = 0$ **do** $\texttt{x} := 1$; S and
- L' for the loop **while** $\neg\texttt{b} \wedge \texttt{x} = 0$ **do** $\texttt{x} := 1$;S'.

Recall that x is a variable not mentioned in **if** b **then** S **else** S'.

We first look at the case where $\mathcal{B}\ \texttt{b}\ \sigma = \textbf{tt}$, and we assume that $\langle \texttt{S}, \sigma\rangle \to\Downarrow \sigma'$. We have to show that in this situation for our program P we have $\langle \texttt{P}, \sigma\rangle \to\Downarrow \sigma''$, where σ'' is a state that is equal to σ' for all variables other than x.

1. $\langle \texttt{x} := 0, \sigma\rangle \to\Downarrow \sigma[x \mapsto 0]$ by the := rule.
2. $\langle$**while** $\texttt{b} \wedge \texttt{x} = 0$ **do** $\texttt{x} := 1$; S, $\sigma'[x \mapsto 1]\rangle \to\Downarrow \sigma'[x \mapsto 1]$ by the **ff** instance of the while loop.
3. $\langle \texttt{x} := 1;\texttt{S}, \sigma[x \mapsto 0]\rangle \to\Downarrow \sigma'[x \mapsto 1]$ by the assumption about S, the := rule, and the fact that we know that S does not mention x.
4. $\langle$**while** $\texttt{b} \wedge \texttt{x} = 0$ **do** $\texttt{x} := 1$; S, $\sigma[x \mapsto 0]\rangle \to\Downarrow \sigma'[x \mapsto 1]$ by 2, 3 and the **tt** instance of the while rule.
5. $\langle$**while** $\texttt{x} = 0$ **do** $\texttt{x} := 1$; S', $\sigma'[x \mapsto 1]\rangle \to\Downarrow \sigma'[x \mapsto 1]$ by the **ff** instance of the while rule.
6. $\langle \texttt{x} := 0$; L; L', $\sigma\rangle \to\Downarrow \sigma'[x \mapsto 1]$ by the ; rule from 1, 4 and 5.

We may further see from 3 and 4 that in the situation where $\mathcal{B}\ \texttt{b}\ sigma = \textbf{tt}$ if we have that $\langle \texttt{x} := 0$; L; L', $\sigma\rangle \to\Downarrow \sigma''$ it must be the case that $\langle \texttt{S}, \sigma\rangle \to\Downarrow \rho$ where ρ is a state that differs σ'' only by its value for x.

On the other hand, if $\mathcal{B}\ \mathtt{b}\ \sigma = \mathbf{ff}$ and $\langle \mathtt{S'}, \sigma\rangle \to \Downarrow \sigma'$ then we have to show that $\langle \mathtt{P}, \sigma\rangle \to \Downarrow \sigma''$, where σ'' is a state that is equal to σ' for all variables other than $\mathtt{x}$.

1. $\langle \mathtt{x} := 0, \sigma\rangle \to \Downarrow \sigma[x \mapsto 0]$ by the := rule.
2. $\langle \textbf{while}\ \mathtt{b} \wedge \mathtt{x} = 0\ \textbf{do}\ \mathtt{x} := 1;\ \mathtt{S}, \sigma[x \mapsto 0]\rangle \to \Downarrow \sigma[x \mapsto 0]$ by the **ff** instance of the while rule.
3. $\langle \textbf{while}\ \mathtt{x} = 0\ \textbf{do}\ \mathtt{x} := 1;\ \mathtt{S'}, \sigma'[x \mapsto 1]\rangle \to \Downarrow \sigma'[x \mapsto 1]$ by the **ff** instance of the while rule.
4. $\langle \mathtt{x} := 1;\ \mathtt{S'}, \sigma[x \mapsto 0]\rangle \to \Downarrow \sigma'[x \mapsto 1]$ by the assumption about S', the := rule, and the fact that we know that S' does not mention x.
5. $\langle \textbf{while}\ \mathtt{b} \wedge \mathtt{x} = 0\ \textbf{do}\ \mathtt{x} := 1;\ \mathtt{S'}, \sigma[x \mapsto 0]\rangle \to \Downarrow \sigma'[x \mapsto 1]$ by 3, 4 and the **tt** instance of the while rule.
6. $\langle \mathtt{x} := 0;\ \mathtt{L};\ \mathtt{L'}, \sigma\rangle \to \Downarrow \sigma'[x \mapsto 1]$ by the ; rule from 1, 4 and 5.

Again we may further see from 4 and 5 that in the situation where $\mathcal{B}\ \mathtt{b}\ \sigma = \mathbf{ff}$ if we have that $\langle \mathtt{x} := 0;\ \mathtt{L};\ \mathtt{L'}, \sigma\rangle \to \Downarrow \sigma''$ it must be the case that $\langle \mathtt{S'}, \sigma\rangle \to \Downarrow \rho$ where ρ is a state that differs σ'' only by its value for x.

Solution 22.
We look at each part.

(a) We use the subscript r to indicate transitions and rules in the new system. We expect the body of the loop to be carried out before the condition is checked, but we can't just use a rule that says

$$\langle \textbf{repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}, \sigma\rangle \Rightarrow_r \langle \mathtt{S};\ \textbf{repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}, \sigma\rangle,$$

because then we would just keep carrying out S without ever checking the condition.

Instead we need to do something more sophisticated:

$$\langle \textbf{repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}, \sigma\rangle \Rightarrow_r \langle \mathtt{S};\ \textbf{if}\ \mathtt{b}\ \textbf{then skip else repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}, \sigma\rangle.$$

(b) We need two cases, just as we did for while

$$\frac{\langle \mathtt{S}, \sigma\rangle \to_r \Downarrow_r \sigma' \quad \langle \textbf{repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}, \sigma'\rangle \to_r \Downarrow_r \sigma''}{\langle \textbf{repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}, \sigma\rangle \to_r \Downarrow_r \sigma''}\ \text{if } \mathcal{B}\ \mathtt{b}\ \sigma' = \mathbf{ff}$$

and

$$\frac{\langle \mathtt{S}, \sigma\rangle \to_r \Downarrow_r \sigma'}{\langle \textbf{repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}, \sigma\rangle \to_r \Downarrow_r \sigma'}\ \text{if } \mathcal{B}\ b\ \sigma' = \mathbf{tt}$$

(c) Our solution to Exercise 3 is

```
S ;
while ¬b do
    S
```

The big steps semantics for While says that

$$\langle \mathtt{S};\ \textbf{while}\ \neg\mathtt{b}\ \textbf{do}\ \mathtt{S}, \sigma\rangle \sigma''$$

if and only if there exists a (necessarily unique) state σ' with

$$\langle \mathtt{S}, \sigma\rangle \to \Downarrow \sigma' \qquad \text{and} \qquad \langle \textbf{while } \neg \mathtt{b} \textbf{ do } \mathtt{S}, \sigma'\rangle \to \Downarrow \sigma''.$$

We know that the body of the loop is carried out at least once. In the base case it is carried out precisely once, which happens if and only if $\mathcal{B}\ b\ \sigma' = \mathbf{tt}$. In this situation

$$\langle \textbf{while } \neg \mathtt{b} \textbf{ do } \mathtt{S}, \sigma'\rangle \to \Downarrow \sigma'$$

by the **ff** version of the while rule and

$$\langle \textbf{repeat } \mathtt{S} \textbf{ until } \mathtt{b}, \sigma\rangle \to_r \Downarrow_r \sigma'$$

by our second rule.

If $\mathcal{B}\ b\ \sigma' = \mathbf{ff}$ we are in the step case, and the induction hypothesis allows us to use that

$$\langle \textbf{repeat } \mathtt{S} \textbf{ until } \mathtt{b}, \sigma'\rangle \to_r \Downarrow_r \sigma'' \qquad \text{if and only if} \qquad \langle \textbf{while } \neg \mathtt{b} \textbf{ do } \mathtt{S}, \sigma'\rangle \to \Downarrow \sigma''$$

to obtain the desired $\langle \textbf{repeat } \mathtt{S} \textbf{ until } \mathtt{b}, \sigma\rangle \to_r \Downarrow_r \sigma''$ from the first rule given in the previous part.

Solution 23.

This is a proof by induction over the derivation that $\langle \mathtt{S}, \sigma\rangle \Rightarrow \sigma'$.

In the two base cases, there is precisely one state that we transition to.

The induction hypothesis is that for statements S and S' the claim holds.

There are three step cases. For composition, the induction hypothesis gives us that there is a at most one state σ' with $\langle \mathtt{S}, \sigma\rangle \Rightarrow \sigma'$, and also that there is at most one state σ'' with $\langle \mathtt{S'}, \sigma'\rangle \Rightarrow \sigma''$, and σ'' is the only state for which we may derive $\langle \mathtt{S;S'}, \sigma\rangle\sigma''$.

For conditional statements, we note that for the state σ, the boolean expression will evaluate to precisely one of **tt** or **ff**, so precisely one of the two rules applies, and then the claim follows from the induction hypothesis.

For while loops, again we note that that for the state σ, the boolean expression will evaluate to precisely one of **tt** or **ff**, and so again precisely one of the two rules applies. In the **ff** case, it is clear that there is a unique state in the form of σ again.

In the **tt** case, by the induction hypothesis we know that σ' is uniquely determined if it exists, and the remaining claim follows from Exercise 20, where we note that each state constructed along the way is uniquely determined.

Solution 24.

We want to show that for all states σ and σ' as well as statements S, S' and S'' it is the case that

$$\langle \mathtt{(S;S')}; \mathtt{S''}, \sigma\rangle \to \Downarrow \sigma' \qquad \text{if and only if} \qquad \langle \mathtt{S;(S';S'')}, \sigma\rangle \to \Downarrow \sigma'.$$

We know from the definition of $\to$ that $\langle \mathtt{(S;S')}; \mathtt{S''}, \sigma\rangle \to \Downarrow \sigma'$ if and only there is a necessarily unique state (see Exercise 23) ρ with

$$\langle \mathtt{S;S'}, \sigma\rangle \to \Downarrow \rho \qquad \text{as well as} \qquad \langle \mathtt{S''}, \rho\rangle \to \Downarrow \sigma'.$$

Using the same idea again, there must be a necessarily unique state τ with

$$\langle \mathtt{S}, \sigma\rangle \to \Downarrow \tau \qquad \text{and} \qquad \langle \mathtt{S'}, \tau\rangle \to \Downarrow \rho.$$

So now we know that $\langle(\mathtt{S};\mathtt{S'});\mathtt{S''},\sigma\rangle \to \Downarrow \sigma'$ if and only if there are uniquely determined states τ and ρ such that

$$\langle \mathtt{S},\sigma\rangle \to \Downarrow \tau \qquad \langle \mathtt{S'},\tau\rangle \to \Downarrow \rho \qquad \langle \mathtt{S''},\rho\rangle \to \Downarrow \sigma',$$

and this is the case if and only if $\langle \mathtt{S};(\mathtt{S'};\mathtt{S''}),\sigma\rangle \to \Downarrow \sigma'$ using much the same reasoning.

Solution 25.
This is a proof by induction over n.

For the base case assume that we know that $\langle \mathtt{S};\mathtt{S'},\sigma\rangle \Rightarrow \langle \mathbf{skip},\sigma'\rangle$. The only way in which this is possible is if we are using the **skip** version of the ; rule, and that requires that both S and S' are equal to **skip**, and σ is equal to σ'. In this situation we have that

$$\langle \mathtt{S},\sigma\rangle \Rightarrow^0 \langle \mathbf{skip},\sigma\rangle \qquad \text{and} \qquad \langle \mathtt{S'},\sigma\rangle \Rightarrow^0 \langle \mathbf{skip},\sigma\rangle$$

as required.

For the step case, assume that $\langle \mathtt{S};\mathtt{S'},\sigma\rangle \Rightarrow^{n+1} \langle \mathbf{skip},\sigma'\rangle$.

The only rule that may be applied in this situation is the general ; rule, so there exists a While program T and a state τ with

$$\langle \mathtt{S};\mathtt{S'},\sigma\rangle \Rightarrow \langle \mathtt{T};\mathtt{S'},\tau\rangle \Rightarrow^n \langle \mathbf{skip},\sigma'\rangle$$

as well as

$$\langle \mathtt{S},\sigma\rangle \Rightarrow \langle \mathtt{T},\sigma'\rangle.$$

By the induction hypothesis there exist m and m' with $n = m + m' + 1$ and a state τ' with

$$\langle \mathtt{T},\tau\rangle \Rightarrow^m \langle \mathbf{skip},\tau'\rangle \qquad \text{and} \qquad \langle \mathtt{S'},\tau\rangle \Rightarrow^{m'} \langle \mathbf{skip},\sigma'\rangle.$$

Hence we have that

$$\langle \mathtt{S},\sigma\rangle \Rightarrow^{m+1} \langle \mathbf{skip},\tau'\rangle \qquad \text{and} \qquad \langle \mathtt{S'},\tau'\rangle \Rightarrow^{m'} \langle \mathbf{skip},\sigma'\rangle,$$

and since $n + 1 = m + m' + 1 + 1 = (m + 1) + m' + 1$ we have established the required property for the step case.

Solution 26.
We split our proof into two, for the two directions of the bi-implication.

First assume we know that $\langle \mathtt{S},\sigma\rangle \to \Downarrow \sigma'$. We carry out a proof by induction over the derivation of this transition.

There are two base cases.

- If we use the **skip** rule the we have $\langle \mathbf{skip},\sigma\rangle \Rightarrow^* \langle \mathbf{skip},\sigma\rangle$ by reflexivity of the relation.
- If we use the assignment rule we know that S is of the form $\mathtt{x} := \mathtt{a}$ for some variable x and arithmetic expression a, and $\sigma' = \sigma[\mathtt{x} \mapsto \mathcal{A}\ a\ \sigma]$. In this situation we have

  $$\langle \mathtt{x} := \mathtt{a},\sigma\rangle \Rightarrow^* \langle \mathbf{skip},\sigma[\mathtt{x} \mapsto \mathcal{A}\ a\ \sigma]\rangle$$

  by the small step assignment rule.

We next address the step cases.

- If we have that S is of the form T;T' for statements T and T' and $\langle \mathtt{T}, \sigma\rangle \to \Downarrow \tau$ as well as $\langle \mathtt{T'}, \tau\rangle \to \Downarrow \sigma'$ then by the induction hypothesis we know that we have

$$\langle \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \tau\rangle \qquad \text{as well as} \qquad \langle \mathtt{T'}, \tau\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle,$$

  so by the composition rule we have

$$\langle \mathtt{T;T'}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}\,;\mathtt{T'}, \tau\rangle \Rightarrow \langle \mathtt{T'}, \tau\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle,$$

  which gives the required

$$\langle \mathtt{T;T'}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle.$$

- If S is of the form **if** b **then** T **else** T' there are two cases. If $\mathcal{B}\ b\ \sigma = \mathbf{tt}$ then for

$$\langle \mathbf{if}\ \mathtt{b}\ \mathbf{then}\ \mathtt{T}\ \mathbf{else}\ \mathtt{T'}, \sigma\rangle \to \Downarrow \sigma'$$

  to hold we must have

$$\langle \mathtt{T}, \sigma\rangle \to \Downarrow \sigma'.$$

  Applying the induction hypothesis we have $\langle \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle$ as required, and the proof for the case where the boolean expression evaluates to **ff** is much the same.

- If S is of the form **while** b **do** T, we also have two cases. If $\mathcal{B}\ b\ \sigma = \mathbf{tt}$ then for

$$\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \to \Downarrow \sigma'$$

  there must be a state τ such that both

$$\langle \mathtt{T}, \sigma\rangle \to \Downarrow \tau \qquad \text{and} \qquad \langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \tau\rangle \to \Downarrow \sigma'.$$

  By the induction hypothesis we know that

$$\langle \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \tau\rangle \qquad \text{and} \qquad \langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \tau\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle,$$

  and so we obtain

$$\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathtt{T};\mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \tau\rangle \Rightarrow \langle \mathbf{skip}, \sigma'\rangle$$

  as required.

  If the loop condition is false we know that $\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \to \Downarrow \sigma$. In this situation the **ff** version of the while rule for $\Rightarrow$ tells us that $\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \Rightarrow \langle \mathbf{skip}, \sigma\rangle$ as required.

Now assume that we know that $\langle \mathtt{S}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle$.
This is a proof by induction over the number of $\Rightarrow$ transitions required.

- If there are 0 transitions then S must be **skip** and σ must be σ', and in that situation we have

$$\langle \mathbf{skip}, \sigma\rangle \to \Downarrow \sigma$$

  as required.

- If there are $n+1$ many transitions required, this may be for the following reasons.

- S is of the form x := a for some variable x and arithmetic expression a, and the transition is
$$\langle \mathtt{x} := \mathtt{a}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma[\mathtt{x} \mapsto \mathcal{A}\ a\,\sigma]\rangle.$$
In this situation we have
$$\langle \mathtt{x} := \mathtt{a}, \sigma\rangle \to \Downarrow \sigma[\mathtt{x} \mapsto \mathcal{A}\ a\,\sigma]$$
by the := rule for the big step semantics.
- If S is of the form T;T' we know that
$$\langle \mathtt{T;T'}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle$$
must occur because there exists a state τ with
$$\langle \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \tau\rangle$$
and the transition to a final state is provided by
$$\langle \mathtt{T;T'}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}\,;\mathtt{T'}, \tau\rangle \Rightarrow \langle \mathtt{T'}, \tau\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma\rangle.$$
The induction hypothesis gives us
$$\langle \mathtt{T}, \sigma\rangle \to \Downarrow \tau \qquad \text{and} \qquad \langle \mathtt{T'}, \tau\rangle \to \Downarrow \sigma',$$
and by the ; rule for the big step semantics this implies
$$\langle \mathtt{T;T'}, \sigma\rangle \to \Downarrow \sigma'$$
as required.
- If S is of the form **if** b **then** T **else** T' we make a case distinction. If $\mathcal{B}\ b\ \sigma = \mathbf{tt}$ then the reason we have
$$\langle \mathbf{if}\ \mathtt{b}\ \mathbf{then}\ \mathtt{T}\ \mathbf{else}\ \mathtt{T'}, \sigma\rangle \Rightarrow \langle \mathbf{skip}, \sigma'\rangle$$
must be that we have both,
$$\langle \mathbf{if}\ \mathtt{b}\ \mathbf{then}\ \mathtt{T}\ \mathbf{else}\ \mathtt{T'}, \sigma\rangle \Rightarrow^* \langle \mathtt{T}, \sigma\rangle \qquad \text{and} \qquad \langle \mathtt{T}, \sigma\rangle \Rightarrow \langle \mathbf{skip}, \sigma'\rangle.$$
By the induction hypothesis we know that
$$\langle \mathtt{T}, \sigma\rangle \to \Downarrow \sigma',$$
and so
$$\langle \mathbf{if}\ \mathtt{b}\ \mathbf{then}\ \mathtt{T}\ \mathbf{else}\ \mathtt{T'}, \sigma\rangle \to \Downarrow \sigma'$$
by the **tt** version of the if rule for the big step semantics.
The case where $\mathcal{B}\ b\ \sigma = \mathbf{ff}$ is much the same.
- If S is of the form **while** b **do** T we again have two cases. If $\mathcal{B}\ b\ \sigma = \mathbf{ff}$ then
$$\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \Rightarrow \langle \mathbf{skip}, \sigma\rangle,$$
and we obtain
$$\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \to \Downarrow \sigma$$
by the **ff** version of the while rule.

If $\mathcal{B}\ b\ \sigma = \mathbf{tt}$ then the reason we have

$$\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle$$

must be because

$$\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \Rightarrow \langle \mathtt{T};\ \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle,$$

which in turn requires the existence of a state τ with

$$\langle \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}, \tau\rangle$$

so that

$$\langle \mathtt{T};\ \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \Rightarrow^* \langle \mathbf{skip}\,; \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \tau\rangle \Rightarrow \langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \tau\rangle \Rightarrow^* \langle \mathbf{skip}, \sigma'\rangle.$$

The induction hypothesis now gives us that

$$\langle \mathtt{T}, \sigma\rangle \rightarrow \Downarrow \tau \qquad \text{and} \qquad \langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \tau\rangle \rightarrow \Downarrow \sigma',$$

The **tt** version of the while rule for the big step semantics now ensures that we have the required

$$\langle \mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{T}, \sigma\rangle \rightarrow \Downarrow \sigma'.$$

Chapter 3

Complexity and Asymptotic Analysis

In the previous chapter we provide an abstract notion of computation, and that provides us with the ability to analyse the input/output behaviour of programs.

In this chapter we are concerned with looking at more fine-grained behaviour of programs. The material provided here gives an introduction to the idea of the complexity of programs, which is studied in more detail in COMP26120 *Algorithms and Data Structures*.

At occasion in this chapter we refer to algorithms that we can't implement in While without encoding the problem—this is due to the fact that our only data structure is that of a number. Nonetheless it seems sensible to refer to such algorithms in the interest of making connections with material students see on other course units.

3.1 General ideas

The analysis of the complexity of programs (and more generally, algorithms) relies on some basic ideas:

- We need a measure of complexity and
- we need a notion of 'size' of the problem (or input).

This allows us to describe the complexity of a program as a function from the size of the problem to the complexity of executing the program for inputs of that size. This should then allow us to compare the behaviour of two programs that solve the same problem.

3.1.1 Time and space complexity

Typical measures of complexity are known as **time** and **space**.

Space complexity is concerned with the amount of memory a program uses. It's not unusual to be able to make trade-offs between space and time complexity: If more memory is used then intermediate results can be saved, which may allow our algorithm to reach the result with fewer steps. When looking at the While language the only available memory is in the form of the number of variables used, and one might imagine writing a program that uses fewer variables but is slower because it needs to re-calculate some values. In Example 6 the first few Fibonacci numbers are calculated, and we keep track of the current value in the variable y and the previous value in the variable z, which allows us to calculate the new current value as the sum of the two—if we had to reduce the number of variables we would have to use another loop to recalculate the previous value.

Counter-intuitively, time complexity is not measured by running a program and measuring how long it takes—that would rely on the hardware used, how busy the processor is, and other unknown quantities. Instead, one measures the complexity by *counting steps*.

3.1.2 Counting steps

Which steps to count is an interesting question: One could try to count all individual instructions in the code, but that is typically rather laborious to do as a function of the size of the input. Also, as we see below, we are typically interested in the **asymptotic behaviour** of a program—that is, we want to know what happens if the input size becomes very large, and we are only interested in doing this up to the asymptotic growth of the underlying function. Hence counting all instructions would lead to a level of detail that is subsequently ignored.

Typically for different problems different steps might be counted, for example:

- For a sorting algorithm one often counts the number of comparisons being carried out.
- For a search algorithm one might count the number of look-ups required.
- For an algorithm that operates on matrices we might count the number of times it has to access an entry in a matrix, or to write to a matrix.

In general one needs to make sure that what one counts is representative of the way the algorithm works, potentially taking into account the architecture it runs on: Some operations may be much more time expensive than others. However, for modern architectures this is usually not a big issue.

3.1.3 Best, worst, average case

A further issue to consider is how we count steps based on the size of the problem - typically there are many different variations of a problem with the same size, for example searching for a specific entry among n many entries, and for different inputs the number of steps taken will vary even if the problem size (the number of entries among which we search) is the same. One has to make a decision here whether

- One wants to look at the worst case or
- one wants to determine an 'average case' or
- one wants to look at the best case.

For application purposes the final choice is not much use—yes, if we search for an entry among 10,000 we may be lucky on the first try, so the best case is one look-up, but that isn't very likely to happen. Typically one either wants to guard against what can happen in the worst case, or form some kind of average. In the COMP1120 notes an example of forming such an average is covered in Section 4.5, which gives some idea of the complexity involved in analysing the average rigorously.

For the purposes of these notes we will be mostly concerned with the **worst case** and concentrate on asymptotic complexity, see Section 3.2.

3.1.4 Examples

Consider the following two While programs which take as input a number $n \in \mathbb{N}$ in the variable n and produce an answer in the variable a such that when the program has been executed

$$\mathsf{a} = \begin{cases} 1 & n \text{ is even} \\ 0 & \text{else.} \end{cases}$$

```
while ¬n≤1
    n:=n−2;
a:=1−n
```

```
y:=0;
while y≤n−1
    y:=y+2;
if y=n
    a:=1
else
    a:=0
```

The second program is longer. Does that mean it's less efficient than the first one? Does the difference matter when the input is very large?

Let's count the number of steps these programs take, counting

- assignments of values to variables and
- checks of boolean expressions.

We carry out the task for the two programs.

This program consists of a loop followed by an assignment. Hence there is on instruction, plus the loop. We may also see that each time the loop is carried out, there are 2 instructions (one comparison to check whether the loop should be carried out, and one assignment), plus one more comparison when the condition is false.
In order to work out how many times the loop is executed, we check the condition, which tells us the loop stops when $\mathtt{n} \leq 1$, and that 2 is subtracted from n each time the loop is executed. So the loop is carried out $\lfloor n/2 \rfloor$ many times, and the total number of steps taken is (ignoring the $\lfloor\rfloor$ function)

$$1 + 2 \cdot \frac{n}{2} + 1 = n + 2.$$

This program has one instruction at the start, a loop, and then an if statement, with one instruction in each branch, so that makes for 2 steps (one to check whether the condition is true and one assignment).
The number of times the loop is carried out is $\lfloor n - 1/2 \rfloor$, so the total number of steps is (ignoring the $\lfloor\rfloor$ function)

$$3 + 2\frac{n-1}{2} + 1 = n + 3.$$

Hence the number of instructions carried out for both programs are almost the same, and there is no significant difference in their efficiency. They both have *linear complexity* because the function describing the number of steps taken is a linear function. We explain in more detail what that means in Section 3.4. We note that both programs fail to terminate if the input fails to be a natural number.

Exercise 27.
For the two programs from Exercise 19 we know that they compute the same value for the input variable i, where the program on the left computes its output in the variable x, and the one on the right in the variable x.

```
y:=1;  a:=0;
while y≤i
    a:=a+y;
    y:=y+1;
```

```
x:=i×(i+1);  y:=0;
while 0≤x
    x:=x−2;
    y:=y+1
```

Count the number of steps they take, again counting assignments and comparisons.

Exercise 28.
For the program from Example 4 count the number of steps it takes as a function of the input held in the variable `n`. Recall that we are interested in the worst case. Count checks of boolean conditions and assignments. There is no need to simplify the expression you obtain.

3.2 Asymptotic analysis—rates of growth

There are practical reasons why we are concerned with the complexity of algorithms. Executing a particular program will take time and the amount of time may matter.

We might be running a safety-critical application, and the plane may have crashed by the time the computer has told the auto-pilot what to do. Or we might be running the program on huge data sets, and we have to wait days (or even years) for the answer, or we might be running our program often on smaller data sets and we need to get the answer promptly for our work flow. For all those applications we want to be able to tell what to expect.

Complexity analysis is concerned with the question of how our programs and algorithms perform when we have a large input size: whatever function governs the number of steps taken by a programme or algorithm, the effects are much more visible when the input is really large. That's what the name 'asymptotic analysis' refers to.

3.2.1 Basic assumptions

The basis of our mathematical treatment is that we have a function that describes the complexity of an algorithm as a function of the size of the input. We want to be able to *compare* such functions for different algorithms or programs, so we expect these functions to output to numbers, where we have the less then or equal relation.

One might expect that it should be possible to describe the complexity of an algorithm as a function

$$\mathbb{N} \longrightarrow \mathbb{N},$$

since it makes sense to measure the size of the problem as a natural number, and if we count steps (or locations in memory) we should also be able to express this as a natural number. But it is customary to instead think of the underlying assignments as functions

$$\mathbb{R}^+ \longrightarrow \mathbb{R}$$

instead, where

$$R^+ = \{x \in \mathbb{R} \mid x \geq 0\}.$$

That also allows us to graph them, and reason without having to worry about integer inputs producing integer outputs. The definitions we give below are more general, but for the purposes of this course unit it's fine if we think of all functions of interest as being from the non-negative real numbers to the real numbers. You may be surprised that we allow negative outputs given that functions we are interested in are concerned with counting instructions. However, we only approximate the number of instructions, so for small inputs we may end up with negative values in our approximation. We want to be able to talk about assignments such as $x \longmapsto x^2 - x$, and we can study these more easily if functions with negative outputs are part of what we study.

Given two such functions we would like to answer the question of which them has a *lower rate of growth* since that is the function that describes the behaviour of an algorithm that is less computationally expensive when the problem size grows to be large.

3.2.2 Comparing functions

For students who did not take COMP11120 *Mathematical Techniques for Computer Science* we review formal definitions that are relevant to our considerations and follow this by introducing the formal definition of the concept known as 'big O'.

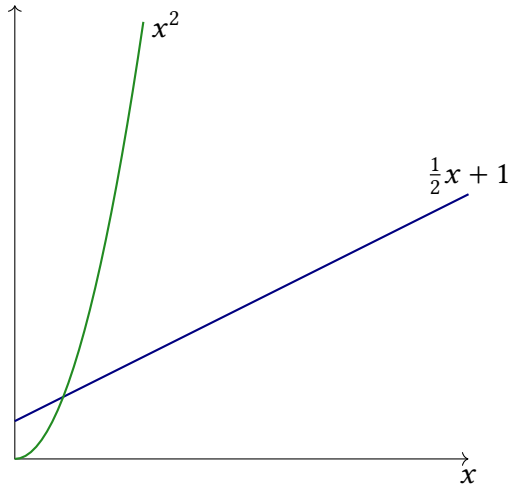
The simplest notion of comparing two functions is the following:

Definition 2. Let N be a set of numbers, $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ or $\mathbb{R}$, and let f and g be two functions from a set S to N. We say that f **dominates** g (or f **is above** g) if and only if

$$\text{for all } s \in S \text{ it is the case that } fs \geq gs.$$

Clearly, if f and g are functions describing the behaviour of two algorithms we want to use the one corresponding to g—it is better behaved for all inputs. But often comparing two algorithms doesn't give such a clear-cut answer.

Consider the functions below, going from $\mathbb{R}^+$ to $\mathbb{R}$.



Neither function dominates the other. But clearly if the problem size is large (that is, we move to the right in the graph) then the function

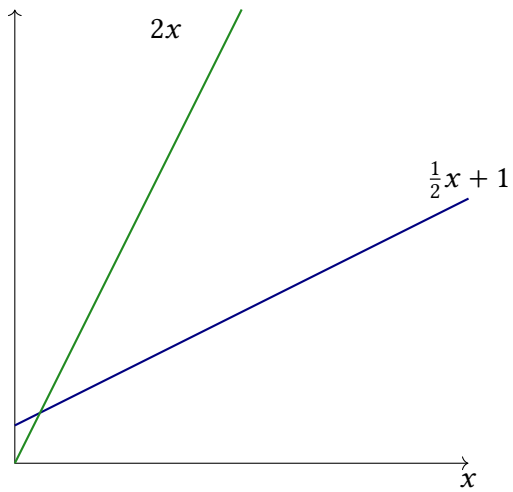
$$x \longmapsto \tfrac{1}{2}x + 1$$

offers a much preferable solution. This idea is encapsulated by the following definition.

Definition 3. Let N and N' be sets of numbers from $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ or $\mathbb{R}$, and let f and g be two functions from N to N'. We say that f **eventually dominates** g if and only if

$$\begin{aligned} &\text{there exists } k \in \mathbb{R} \\ &\qquad \text{such that} \qquad \text{for all } x \in N \text{ with } x \geq k \text{ we have } fx \geq gx. \end{aligned}$$

However, when computer scientists compare the complexity of algorithms, they make a further generalization.



Both functions given here are *linear*, and while one shows a distinct improvement on the other as far as the rate of growth is concerned, this can be described using a *constant factor*. In order to obtain a clear separation of different behaviours we have to rule out such constant factors as being the only difference in the growth behaviour of two functions.

Definition 4. Let N be a set of numbers, $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ or $\mathbb{R}$, and let f and g be two functions from a set S to N. We say that g is $O(f)$ if and only if there exists $c \in \mathbb{R}^+$ such that $c \cdot f$ eventually dominates $|g|$.

This is typically pronounced as 'g is big O of f'. We may think of f determining a collection of functions whose asymptotic growth is at most that of f, and these are the functions that are $O(f)$.

3.2.3 Investigating $O(-)$

The two functions given above,

$$\begin{aligned} f : \mathbb{R}^+ &\longrightarrow \mathbb{R} \\ x &\longmapsto \tfrac{1}{2}x + 1 \end{aligned} \qquad \text{and} \qquad \begin{aligned} g : \mathbb{R}^+ &\longrightarrow \mathbb{R} \\ x &\longmapsto 2x \end{aligned}$$

satisfy both,

$$f \text{ is } O(g) \qquad \text{and} \qquad g \text{ is } O(f).$$

Since g eventually dominates f it is clear (set $c = 1$) that f is $O(g)$. To see that we also have that g is $O(f)$ we note that for $c = 4$ we have that for all $x \in \mathbb{R}^+$,

$$|gx| = 2x \leq 4 \cdot (\frac{1}{2}x + 1) = 2x + 2 = fx.$$

And indeed, it is not difficult to show that all linear functions, that is functions of the form $x \longmapsto mx + b$ are $O(h)$, where h is the function

$$\begin{aligned} h : \mathbb{R}^+ &\longrightarrow \mathbb{R} \\ x &\longmapsto x \end{aligned}$$

which maps each element x of the source set to itself.

However, the function h'

$$\begin{aligned} h' : \mathbb{R}^+ &\longrightarrow \mathbb{R} \\ x &\longmapsto x^2 \end{aligned}$$

from $\mathbb{R}^+$ to $\mathbb{R}$ which maps x to x^2 is not $O(h)$ for this h:

Assume we had constants $c, k \in \mathbb{R}^+$ such that for all $x \in \mathbb{R}^+$ we know that

$$|h'(x+k)| = (x+k)^2 \leq c(x+k),$$

which is the condition required to obtain that h' is $O(h)$ since it says that for all inputs greater than or equal to k we have that h' is below $c \cdot h$. But we may see that whatever c is, we may choose x to be larger than c, and then the equality cannot possibly hold. Hence such a c cannot exist, and so the function h' is not $O(h)$.

See Section 3.4 of a discussion of complexity classes such as 'linear' and 'quadratic'.

3.2.4 Notation for $O(-)$

In the computer science literature, particular functions in a particular notation are typically chosen as representatives of a given behaviour.

You will people talk about $O(1)$, $O(n)$, $O(n^2)$, $O(n \log n)$ and similar.

Here 1 is notation for the constant function returning the output 1 for each input, and n is the notation for the function that returns its argument and n^2 is the notation for the quadratic function h' from the previous section.

Fixing the names for these functions in this way is convenient since it makes it easier to communicate with others. In what follows we treat notation such as n^2 as naming the appropriate function given by the assignment $x \longmapsto x^2$.

In the following section we justify the choice of these particular functions.

3.3 Properties of $O(-)$

Computer scientists are used to speaking about complexity classes informally, but for our usage to make sense we need to make sure that our notion behaves in the way we expect.

We have a binary relation on the set of functions from $\mathbb{R}^+$ to $\mathbb{R}$, where f is related to g if f is $O(g)$. We may think of this relation as 'f grows at most as fast as g'.

Exercise 29.
Show that the 'grows at most as fast as' relation is neither symmetric nor anti-symmetric.

Proposition 5. *The 'grows at most as fast as' relation is reflexive and transitive.*

Exercise 30.
Show that the 'grows at most as fast as' relation (see Section 3.3) is reflexive and transitive for functions from $\mathbb{R}^+$ to $\mathbb{R}^+$.

A relation that is reflexive and transitive is also known as a *pre-order*, which means it shares some properties with a partial order.

We find it convenient to write

$$O(f) = \{g : \mathbb{R}^+ \longrightarrow \mathbb{R} \mid g \text{ is } O(f)\}$$

since that allows us to use notation such as $g \in O(f)$. Hence $O(f)$ is the set of all those functions that grow at most as fast as f.

Proposition 6. *Let f and g be functions from $\mathbb{R}^+$ to $\mathbb{R}$. We have*

$$f \in O(g) \qquad \text{if and only if} \qquad O(f) \subseteq O(g).$$

Proof: For the only if direction this follow from transitivity of the grows at most as fast as relation: If h is a function $\mathbb{R}^+ \longrightarrow \mathbb{R}$ with $h \in O(f)$ then $f \in O(g)$ implies $h \in O(g)$.

For the if direction, if $O(f) \subseteq O(g)$ then $f \in O(f) \subseteq O(g)$ shows that $f \in O(g)$. ∎

We may see immediately that if we add functions in the same growth class, or multiply them with a constant, we do not leave that class:

Proposition 7. *Let f, g and h all be functions from $\mathbb{R}^+$ to $\mathbb{R}$.*

(i) If $f \in O(h)$ then for all $r \in \mathbb{R}^+$, $r \cdot f \in O(h)$.

(ii) For $r \in \mathbb{R}^+ \setminus \{0\}$ we have $O(f) = O(r \cdot f)$.

(iii) If $f, g \in O(h)$ then $f + g \in O(h)$.

(iv) If $g \in O(f)$ then $O(f + g) = O(f)$.

Proof: We show each statement.

(i) Since f is $O(h)$ we may find constants k and c in $\mathbb{R}^+$ such that for all $x \geq k$ we have $|fx| \leq c \cdot hx$. Then for $r \in \mathbb{R}^+$, $|r \cdot fx| = r \cdot |fx| \leq r \cdot c \cdot hx$ so a new constant $r \cdot c$ establishes that $r \cdot f$ is also $O(h)$.

(ii) The second statement follows from the first. It implies that $O(r \cdot f) \subseteq O(f)$. But the same result gives us for $r \in \mathbb{R}^+ \setminus \{0\}$ that $f = 1/r \cdot (r \cdot f) \in O(r \cdot f)$, which implies $O(f) \subseteq O(r \cdot f)$.

(iii) If f and g are $O(h)$ we may find constants k, k', c and c' in $\mathbb{R}^+$ such that for all $x \geq k$ we have $|fx| \leq c \cdot hx$ and for all $y \geq k'$, $|gx| \leq c' \cdot hy$. If we choose $k'' = \max\{k, k'\}$ and $c'' = c + c'$ we get for all $x \geq k''$

$$|(f + g)x| = |fx + gx| \leq |fx| + |gx| \leq c \cdot hx + c' \cdot hx = (c + c') \cdot hx = c'' \cdot hx,$$

which justifies the third claim.

(iv) The third statement gives us that $g \in O(f)$ implies $f + g \in O(f)$, which in turn implies $O(f + g) \subseteq O(f)$. To see that $f \in O(f + g)$ we observe that $g \in O(f)$ implies that we may find c and k in $\mathbb{R}^+$ such that for all $x \geq k$ we have $|gx| \leq c \cdot fx$. Note in particular that this means $0 \leq fx$ for all such x and so $|fx| = fx$ for $x \geq k$. Hence for $c' = 1 + 2c$ we have for all $x \geq k$ that

$$|fx| = fx + gx - gx \leq fx + |gx| + |gx| \leq fx + c \cdot fx + c \cdot fx = (1 + 2c) \cdot fx = c' \cdot fx,$$

and so we also get $O(f) \subseteq O(f + g)$.

∎

Recall that a **polynomial function** from a set of numbers N to itself is given by a function

$$\sum_{i=0}^{m} a_i x^i = a_m x^m + a_{m-1} x^{m-1} + \cdots + a_1 x + a_0,$$

where $m \in \mathbb{N}$ and $a_0, a_1, \ldots, a_m$ in N. If $a_m \neq 0$ we say that the polynomial has **degree** m, and a_m is the **lead coefficient**.

Note in particular that if $m = 0$ then the corresponding polynomial function is constant.

Proposition 8. *For m, m' in $\mathbb{N}$ with $m < m'$ we have that $O(n^m) \subsetneq O(n^{m'})$.*

Proof: We have for all $x \geq 1$ that $x^m \leq x^{m'}$, and so n^m is $O(n^{m'})$. By transitivity of the grows at most as fast as relation this means that if f is $O(n^m)$ then f is also $O(n^{m'})$, and so $O(n^m) \subseteq O(n^{m'})$.

We claim that $n^{m'}$ is not $O(n^m)$. Otherwise we would require c and k in $\mathbb{R}^+$ such that for all $x \geq k$, $x^{m'} \leq c \cdot x^m$, but this is false when $x \geq c$ as long as x is also greater than or equal to 1, so such c and k cannot exist and $n^{m'} \notin O(n^m)$, which establishes the claim. ∎

Proposition 9. *Let f be a polynomial function of degree m. Then*

$$O(f) = O(n^m).$$

Proof: Assume that the coefficient of f are given by $a_0, a_1, \ldots, a_m$. We know that for all $0 \leq i \leq m$ we have $n^i \in O(n^m)$. We also have that both $a_i \cdot n^i \in O(n^m)$ and $-a_i \cdot n^i \in O(n^m)$: If $a_i \in \mathbb{R}^+ \setminus \{0\}$ the claim follows from Proposition 7, and otherwise it follows from the fact that for all $x \geq 1$ we have $a_i \cdot x^i \leq x^i \leq x^m$. Applying the final statement in Proposition 7 we obtain that $O(f) = O(a_m \cdot n^m)$, and $O(a_m \cdot n^m) = O(n^m)$ by the second statement of Proposition 7. ∎

3.4 Complexity Classes

From the results of the previous section we may deduce that we have complexity classes with an increasing rate of growth in the form of

$$O(1) \subsetneq O(n) \subsetneq O(n^2) \subsetneq O(n^3) \cdots \subsetneq O(n^m) \subsetneq \cdots$$

We refer to functions in the first few of these classes as having constant, linear, quadratic, cubic respectively.

There are three additional classes that commonly appear when it comes to describing the complexity of common algorithms.

3.4.1 Logarithmic growth

We begin by remarking that we don't have to worry about which base we choose for our logarithm: since we know that for all $a, b \in \mathbb{R}^+ \setminus \{0\}$ we have

$$\log_b x = \frac{1}{\log_a b} \log_a x,$$

and $1/\log_a b \in \mathbb{R}^+ \setminus \{0\}$, all logarithm functions are in the same $O(-)$ growth collection no matter which base we use, so we use $\log$ without providing one.

Logarithmic growth gives us a complexity class between constant and linear growth.

Note that $\log n$ is a function with source set $\mathbb{R}^+ \setminus \{0\}$, rather than $\mathbb{R}^+$, but we ignore that here.

Proposition 10. *We have that*

$$O(1) \subsetneq O(\log n) \subsetneq O(n).$$

Proof: Let r be the base of the logarithm. For $x \geq r$ we have that $1 \leq \log x$, which establishes that $O(1) \subseteq O(\log n)$. If $\log n$ were an element of $O(1)$ we would have to be able to find c and k in $\mathbb{R}^+$ such that for all $x \geq k$, $\log x \leq c$, which is equivalent to $x \leq r^c$. Such a c cannot exist, so the inclusion is strict.

We know that for all $x \geq 1$ we have $\log x \leq x$, so $O(\log n) \subseteq O(n)$. If n were an element of $O(\log n)$ we would have to be able to find c and k in $\mathbb{R}^+$ such that for all $x \geq k$, $x \leq c \cdot \log x$, which is equivalent to $r^x \leq r^{c \cdot \log x} = (r^{\log x})^c = x^c$. Such a c cannot exist, but we don't formally show that here—see the comments below on exponential growth. ∎

You may also encounter the complexity class $O(n \log n)$, which satisfies

$$O(n) \subsetneq O(n \log n) \subsetneq O(n^2).$$

This is the complexity class of effective sorting algorithms.

Exercise 31.
For the following functions from $\mathbb{R}^+$ to $\mathbb{R}$, determine the smallest complexity class they belong to, then put them in rising order of their corresponding complexity class.

(a) $x \longmapsto 2x + 10,000$

(b) $x \longmapsto \log x - 1000$

(c) $x \longmapsto x^2 - x$

(d) $x \longmapsto 2^x$

(e) $x \longmapsto 500$

(f) $x \longmapsto \log x + x^2$

(g) $x \longmapsto 200x - \log x$

3.4.2 Polynomial growth and eyond

The complexity classes described above are often thought of, with some exceptions such as $O(\log n)$, as being given by polynomial functions with increasing degrees.

We sometimes have reason to form the union of all these to cover **polynomial growth**. A function f grows polynomially if and only if there exists an $m \in \mathbb{N}$ such that $f \in O(n^m)$. We may therefore think of such functions as being in the union of all the polynomial classes, that is in

$$\bigcup_{m \in \mathbb{N}} O(n^m).$$

But there are functions that grow even faster, and those are exponential functions. We won't look at those in detail here, but because of their rapid growth computer scientists avoid algorithms with that behaviour if at all possible. It is customary to talk of **exponential growth** without distinguishing the base.

Once we move beyond polynomial to exponential complexity there is an obvious question of whether there are functions that grow even faster than typical exponential functions such as $x \longmapsto 2^x$.

Different authors introduce different terminology and we do not look at their ideas in more detail. The **Ackermann function** is a function that grows faster than anything we discuss, and readers who are interested in an example of this behaviour are encouraged to look for material about this function.

Computer scientists often make a somewhat pragmatic distinction between problems that have polynomial complexity and those that cannot be solved with an algorithm of polynomial complexity.

Problems that have a complexity that can be expressed as a polynomial are considered **tractable**, and others are considered **intractable**. Of course, problems which are, say, exponential, can still be solved in practice for small input sizes, and problems that are polynomial for a large degree may be difficult to solve in practice even for small input sizes. In COMP26120 *Algorithms and Data Structures* these ideas are discussed in more detail, including the famous 'P vs NP' problem.

We address the question whether there are problems that cannot be solved by an algorithm at all in Chapter 5.

3.5 Determining complexity classes: examples

Consider the following While program which calculates $m \operatorname{div} 2$ if we view m as the input to the program and x as its output. We work on the assumption that the value of this variable is a natural number m. We count assignments and comparisons.

```
x := 0;
while 1<m do
   m:=m-2;  x := x+1;
```

We calculate the number of steps carried out when this program is executed. There is one step at the start, and then there is a comparison to check whether the body of the loop should be executed. The loop is then carried out $\lfloor m/2 \rfloor$ many times, with two assignments being carried out each time. Therefore the number of steps is

$$1 + \left\lfloor \frac{m}{2} \right\rfloor \cdot 3 + 1 = 3\left\lfloor \frac{m}{2} \right\rfloor + 2.$$

where the final step comes from checking whether the loop condition still holds. Hence this is a *linear* algorithm.

It's quite easy to make 'off by one' errors, so we carry out a quick check for $m = 5$. We keep track of the values in the variables m and x as well as the loop condition in the following table:

	x	m
start	0	5
loop	1	3
	2	1

The body of the loop is executed twice, which agrees with $\lfloor 5/2 \rfloor = \lfloor 2.5 \rfloor = 2$. A quick check confirms that if instead we choose an even input 4, the body of the loop is also executed twice, which agrees with $\lfloor 4/2 \rfloor = \lfloor 2 \rfloor = 2$.

When we want to count steps we have to be concerned about this kind of error. But if we merely want to determine the smallest complexity class that describes the behaviour of our program, we do not have to consider this level of detail.

To determine that this program is $O(n)$ it is sufficient to note that there's one loop (with a fixed number of instructions on each iteration), and that this loop is carried out a number of times that is linear in m, so the number of steps is of the form $c \cdot m + k$, where c and k are constants, and this is a function that is linear in m which we used as the input to our program.

It is important to note that the instructions available in our language have an impact on the complexity of a program when counting steps—for example, in While we do not have (integer) division as a primitive operation, so every time we want to divide one number by another we have to carry out a loop.

We provide another example: Imagine we restricted the syntax of While so that we are only able to subtract or add 1 to a variable, and we are only able to compare variables to 0. This language is just as powerful as While—we may carry out the same computations—but they become more laborious, and their complexity increases. For example, whereas adding two natural numbers has constant complexity in While since it may be carried out by the statement `x := m + n`, in our restricted language we have to implement addition using a loop such as

```
x:=m;
while ¬n=0 do
    n:=n−1; x:=x+1
```

which has linear complexity in the argument given in the variable `n`.

In order to multiply two numbers we now have to run two nested loops:

```
x:=m; y:=n; z:=m;
while ¬y=0 do
    while ¬x=0 do
        z:=z+1; x:=x−1;
    y:=y−1; x:=m
```

We may now count the number of steps this program takes, based on the number of assignments made: There are three instructions before the outer loop starts, the body of the outer loop is carried out n many times and each time the number of steps carried out is that of the inner loop, plus 2. The body of the inner loop is carried out m many times, and there are two steps each time. So the number of steps as a function of m and n is equal to

$$3 + n \cdot (2 + 2 \cdot m) = 3 + 2mn + 2n.$$

If we set $m = n$ this program computes n^2, and its complexity is quadratic since the function given above is a quadratic polynomial $2n^2 + 2n + 3$ in that case.

Exercise 32.
For $n \in \mathbb{N}$ consider

$$\sum_{i=1}^{n} \sum_{j=1}^{i} j.$$

(a) Write a While program that carries out this calculation. Assume that the input is given in the variable `n` and that the output is the value of the variable `x`.

(b) What is the number of steps your program takes as a function of n, counting assignments?

(c) What is the smallest complexity class containing your function from the previous part?

Exercise 33.
For each of the following While programs:

(a) State with a brief justification their time complexity.

(b) Place them in order of their asymptotic complexity from smallest to largest.

(i)

```
d:= 0;
while y≤r do
    s:=r;
    c:=y;
```

```
while c>0 do
    c := c−1;
    s := s−1;
if ¬r=s then
    d := d+1;
    r := s
else
    skip
```

(ii)

```
z := 1;
while 1≤n do
    z := z×2;
    n := n−1;
z := z×(2×m+1)−1
```

(iii)

```
z := 1;
r := 0;
s := 3;
while r≤s+1 do
    if r<s then
        r := r+1;
        s := s−1;
        z := r;
    else
        z := r−s;
        s := r−2
```

(iv)

```
n := 1000;
c := 0;
a := 1;
while a≤ n do
    a := a+2;
    c := c+1
```

3.6 Complexity of problems versus the complexity of algorithms

In computer science we are interested in the complexity of the solution we apply to a particular problem. This raises the obvious question of whether our solution is efficient, or whether better solutions exist.

3.6.1 Complexity of algorithms

Above we look at programs and their complexity, but in computer science we typically talk about the complexity of *algorithms*. In COMP26120 *Algorithmsand Data Structures* algorithms described using *pseudocode* which is not tied to any particular programming language.

When we look at the number of steps carried out, the programming language in question does not matter—this is a property of the structure of the program, such as its loops and how they are nested, and the actual language of the program does not matter. It should be straightforward to transfer material about While programs to this setting.

3.6.2 Complexity of problems versus the complexity of algorithms

When we have a program (or algorithm) we may ask how many steps it takes as a function of the size of its input, and think of this as a measure of its (time) complexity.

There is also a notion of the complexity of a *problem*. For example, we might ask ourselves what the complexity of 'sorting' is. By this we mean: What is the minimum complexity of an algorithm solving this problem?

This is why people sometimes say that 'sorting has complexity $n \log n$', or that 'searching has complexity $\log n$', although for the latter statement we need to have a set-up that allows us to carry out binary search.

Determining the complexity of an algorithm or program requires us to be able to count (and to form averages if we are interested in average complexity). Determining the complexity of a problem is a more challenging task: If we know that Problem A can be solved by Algorithm 1, and that Algorithm 1 has quadratic complexity, we know that Problem A has at most quadratic complexity. But maybe there's a more efficient way of solving it, and its actual complexity is below quadratic.

Sometimes we are able to apply reasoning to deduce something about the complexity of a problem. For example, if we need to sort k entities, one may make an argument that it is not possible to do so without looking at each entity at least once, and so we can say with confidence that the complexity of sorting is at least linear.

In this situation we would like to say that the function describing the complexity of a sorting algorithm must grow at least as fast as the function which maps an input n to itself. There is notation used for this which says that for functions $f, g : \mathbb{R}^+ \longrightarrow R$ we have that

$$f \text{ is } \Omega(g) \qquad \text{if and only if} \qquad g \text{ is } O(f).$$

Hence in our example we would say that a function f describing the complexity of a sorting algorithm must be $\Omega(n)$, pronounced 'big omega of n'. This notion goes back to Donald Knuth who contributed significantly to the rigorous analysis of the computational complexity of algorithms. He is also the creator of the TEX typesetting system, which is the predecessor of the system used to typeset these notes. We should note that this notion does not agree with a different definition of Ω used in areas of mathematics such as analytic number theory.

Exercise 34.
Show that f is $\Omega(g)$ if and only if there exist $k \in \mathbb{R}^+$ and $c \in \mathbb{R}^+ \setminus \{0\}$ such that for all $x \geq k$ we have $fx \geq c \cdot |gx|$.

We may then say that sorting is $\Omega(n)$—indeed, this notion makes more sense for problems rather than for algorithms.

But knowing that a problem is $\Omega(n)$ does not tell us whether we may indeed find an algorithm with linear complexity to solve it. If we know that a problem is $\Omega(f)$, but the best currently known algorithm is not $O(f)$ we don't know whether the problem is more complex and we can't find an algorithm that is $O(f)$ or whether it's just a question of finding a better solution.

On the other hand, if we know that a problem is $\Omega(f)$ and we have found

There is a final notion connecting Ω and O: We can ask ourselves when $O(f)$ and $O(g)$ give the same set. The relation $f \sim g$ if and only if $O(f) = O(g)$ is an equivalence relation.

Exercise 35.
Show the preceding claim that we have an equivalence relation.

When $O(f) = O(g)$ we say that f **is** $\Theta(g)$ (and then g is also $\Theta(f)$), pronounced 'big theta of f'.

Both, Ω and Θ appear in the complexity part of COMP26120 *Algorithms and Data Structures*.

Exercise 36.
Let $f, g : \mathbb{R}^+ \longrightarrow \mathbb{R}$. Show that f is Θg if and only if there are c, c' and k in R^+ such that for all $x \geq k$

$$c \cdot |gx| \leq fx \leq c' \cdot gx.$$

Exercise 37.
Consider the list of elements i of $\mathbb{N}$ with the property that

- i is divisible by 2
- **or** i is divisible by 3
- **and** i is not divisible by both,

in rising order.

Now consider the problem of, given an input n, finding the nth entry of this list.

What can you say about the complexity of this problem? Aim to identify a complexity class $O(f)$ where you are confident that the problem is in this class, and also aim to identify a class $\Omega(g)$ for this problem. Now consider whether you can think of ways of narrowing the gap.

3.7 Summary

This chapter discusses the idea that we may measure the complexity of a program by counting the number of steps it takes depending on the size of its input. Since there may be more than one input of a given size and the number of steps for these may differ, one has to decide whether to aim to consider an average, or the highest possible number of steps (the worst case). Computer scientists are typically interested in what happens if the inputs become very large, which is known as *asymptotic analysis*. There are suitable mathematical notions available to provide classes of complexity, with different degrees of polynomials providing suitable notions for many algorithms. We may also define the complexity of a problem by looking for the most efficient algorithm that solves it.

3.8 Solutions to Exercises

Solution 27.

We have two steps at the start.

We then have to look at the loop. There is a counter, so the body of the loop is being carried out n many times. The body of the loop has three steps (the comparison each time and the two assignments), and then there's a final comparison that establishes the condition for the loop is now false.

Hence there are

$$2 + n \cdot 3 + 1 = 3n + 3$$

many steps.

Again there are two steps at the start. For the loop we again have a counter, but it's slightly less straightforward than the first one: The loop is being carried out $n \cdot (n+1)/2$ many times. The body of the loop has the same number of instructions as for the other program.

$$2 + n \cdot (n+1) \cdot 3/2 + 1 = \frac{3}{2}n^2 + \frac{3}{2}n + 3$$

many steps.

Readers who know something about complexity will realize that having a linear, and a quadratic, function tells us something important.

Solution 28.

We reproduce the program for convenience. Assume the input is n.

```
y := n−1;
z := 1;
while y>1∧z=1 do
  r := n;
  while y≤r do
     r := r−y;
  if r=0 then
     z := 0;
  else
     skip;
   y := y−1;
)
```

There are two nested loops. The outer loop is controlled by two variables:

- y: This variable starts out as $n-1$, and reduces by 1 each time the body of the outer loop is executed until it reaches 1, which means the body of the outer loop is potentially carried out $n-2$ many times.
- z: This variable starts out as 1 and is set to 0 when a divisor of n has been found. In the worst case, where n is prime, z remains 1 for the entire computation.

The number of steps each time the body of the loop is carried out is given by one comparison, two assignments, the if statement plus the number of steps needed for the inner loop.

The number of times the inner loop is carried out is $n \operatorname{div} y$, and so it varies each time the body of the outer loop is carried out. The number of steps for the inner loop is two each time the body of the loop is executed, plus an extra check when the loop terminates.

If n is prime the number of steps carried out is

$$2 + \left(\sum_{y=2}^{n-1} 4 + (n \operatorname{div} y) \cdot 2 + 1 \right) + 1.$$

If n is not prime then the outer loop will terminate when a divisor for n has been found, and that will be its greatest proper divisor. There is no way of giving a general expression for this number—in the worst case the greatest proper divisor of n is $\sqrt{n}$, in which case the number of steps is

$$2 + \left(\sum_{y=2}^{n-1} 4 + n \operatorname{div}(n - y) \cdot 2 + 1 \right) + 1.$$

Solution 29.
In Section 3.2.3 it is shown that the function h which squares its inputs is not $O(f)$, where f is the function that maps each input $x \in \mathbb{R}^+$ to itself, but we can show that f is $O(h)$: For all $x \geq 1$ we have that $|x| = x \leq x^2$, so we have established the claim by setting both k and c to 1. Hence the relation is not symmetric.

To see that it is not anti-symmetric we note that also in Section 3.2.3, it is established that for the linear functions f and g given there we have both f is $O(g)$ and g is $O(f)$, and since f and g are not equal, the relation is not anti-symmetric.

Solution 30.
We begin with reflexivity. We may choose $c = 1$ and $k = 0$ to have that for all $x \geq 0$ we have $|fx| = fx \leq 1 \cdot fx$, and so f is $O(f)$. Note that we use the fact that for this exercise we have restricted the target set of our functions to the non-negative real numbers.

For transitivity, assume that f is $O(g)$ and g is $O(f)$. Then we have constants c, c', k and k' in $\mathbb{R}^+$ such that for all $x \geq k$ we have

$$|fx| \leq c \cdot gx,$$

and for all $y \geq k'$,

$$|gy| \leq c' \cdot hy.$$

We let $k'' = \max k, k'$ and $c'' = c \cdot c'$. Now for all $x \geq k''$ we get

$$|fx| \leq c \cdot gx \leq c \cdot (c' \cdot hx) = (c \cdot c') \cdot hx,$$

and so we have found the required constants to establish that f is $O(h)$.

Solution 31.
We give the complexity classes for each function.

(a) $O(n)$.

(b) $O(\log n)$

(c) $O(n^2)$

(d) exponential

(e) $O(1)$

(f) $O(n^2)$

(g) $O(n)$

This means that in rising order we have the functions from parts (e), (b), (a) and (g), (c) and (f), and finally (d).

Solution 32.
We solve all parts.

(a) Here is a suitable program.

```
x := 0;  i := 1;
while i≤n do
    j := 1;
    while j≤i do
        x := x+j;  j := j+1;
    i := i+1
```

(b) There are two steps as the program starts. The body of the outer loop is carried out n times, and the number of steps taken is 2 added to the number of steps taken by the inner loop.

The body of the inner loop is carried out i many times, where i changes each time the body of the outer loop is executed and there are two steps each time. Hence the function we are looking for is

$$2 + \sum_{i=1}^{n}(2 + 2 \cdot i).$$

We may now simplify this to

$$2 + 2n + 2 \cdot \sum_{i=1}^{n} i = 2 + 2n + 2 \cdot \frac{n \cdot (n+1)}{2} = 2 + 2n + n(n+1) = n^2 + 3n + 2.$$

(c) The function from the previous part is a polynomial of degree 2 and so our algorithm has quadratic complexity.

Solution 33.
Solutions to both parts are as follows:

(a) The time complexity of the given programs may be described as follows:

(i) The outer loop runs approximately `r` mod `y` many times, and the inner loop approximately `y` many times, so overall the number of instructions is proportional to `r` and this is a program with linear complexity.

(ii) $O(n)$, the loop runs n times

(iii) $O(1)$, the loop runs a constant number of times

(iv) $O(1)$, the loop runs a constant number of times

(b) This means that the programs should be ordered as (iii) and (iv), followed by (ii) and (i) in rising order.

Solution 34.
Assume that f is $\Omega(g)$. Then $g \in O(f)$ and so we may find $c', k' \in \mathbb{R}^+$ so that for all $x \geq k'$ we have $|gx| \leq c' \cdot fx$. If $c = 0$ then $gx = 0$ for all $x \geq k$ and the claim is true for $k = k'$ and any c. Otherwise for all $x \geq k'$ we have $1/c' \cdot |gx| \leq fx$, and we obtain the claim for $k = k'$ and $c = 1/c'$.

Now assume that we have c and k as given. Since $c \neq 0$ we may invert it to obtain $1/c \cdot fx \geq |gx|$, which implies that $g \in O(f)$ as required.

Solution 35.
All the required properties follow directly from the fact that the equality relation has them:

We begin with reflexivity. Clearly $O(f) = O(f)$.

Symmetry follows from directly from the definition. If $f \sim g$ for two suitable functions then $O(f) = O(g)$ which implies $O(g) = O(f)$ and so $g \sim f$.

Transitivity comes from $f \sim g$ and $g \sim h$ implying $O(f) = O(g)$ and $O(g) = O(h)$, which imply $f \sim h$.

Solution 36.
We begin with the only if direction. We know that $g \in O(f)$ and so we may find $d, l \in \mathbb{R}^+$ such that for all $x \geq l$ we have $|gx| \leq d \cdot fx$. If $d = 0$ then we must have that g is the function that returns the value 0 for all inputs, and then f must be the same function for $f \in O(g)$ to hold and the claim is trivial. Otherwise we set $c = 1/d$ which gives $c \cdot gx \leq c \cdot |gx| \leq fx$ for all $x \geq l$.

We also know that $f \in O(g)$ so we may find $d', l' \in \mathbb{R}^+$ such that for all $x \geq l'$ we have $|fx| \leq d' \cdot gx$. If we set $c' = d'$ and $k = \max l, l'$ we have for all $x \geq k$ that

$$c \cdot gx \leq fx \leq d' \cdot gx$$

as required.

For the if direction, assume we have constant as given. The first given inequality gives $|gx| \leq 1/c \cdot fx$ for $x \geq k$ and so gives $g \in O(f)$. It also tells us that $fx \geq 0$ for all $x \geq k$, and so $|fx| = fx \leq c' \cdot gx$ for these x, which implies $f \in O(g)$.

Solution 37.
A naive algorithm for solving the given problem is to have an outer loop from 2 to $6n$ (since whenever we go from i to $i + 6$ we should find at least two numbers that are divisible by 2 and 3 only) which checks each number's divisibility by 2 and by 3, and increases a counter correspondingly, such as the following, where we assume that the input is given via the variable n, and the output is produced in the variable x.

```
m:=n;  y:=2,  c:=0;
 while y≤6×n ∧ c<n do
    z:=y
    while 2≤y do
       y:=y−2;
```

```
if y=0 then
    b := 1;
else
    b := 0;
y := z;
while 3≤y do
    y := y−3;
if b=1∧¬y=0 then
    c := c+1;
    else
        if y=0 then
            c := c+1;
        else
            skip;
x := y;  y := y+1;
```

We give a more detailed analysis here than we would expect students to provide in the interest of allowing students to see how the answer arises.

This program has an outer loop and two inner not nested loops. There are say k_1 many instructions outside of the outer loop, and say k_2 many instructions in the outer loop apart from the two inner loops. The outer loop is performed at between $2n$ and $6n$ many times, let's say it's approximately $c_1 \cdot n$ many times.

The first of the inner loops is carried out between 1 and $c_1 n/2$ many times and has k_3 many instructions, so we have an expression of the form

$$k_1 + c_1 n \cdot k_2 + \left(\sum_{y=2}^{c_1 n} \frac{1}{2} y \cdot k_3 \right).$$

We may see that the various constants do not have an impact on the complexity class of the function we describe here.

We know that

$$\sum_{y=2}^{c_1 n} \frac{1}{2} y \cdot k_3 = \frac{k_3}{2} \cdot \sum_{y=2}^{c_1 n} y = \frac{k_3}{2} \cdot \left(\frac{c_1 n \cdot (c_1 n + 1)}{2} - 1 \right) = \frac{k_3}{4} \cdot \left(c_1^2 \cdot n^2 + c_1 n - 2 \right),$$

so this is the part of the term that gives the highest degree polynomial: It is quadratic in n. We also emphasize again that none of the constants have an impact on the degree of the polynomial as long as $c_1 > 0$.

Hence we know that the complexity of the problem is at most quadratic.

We may also argue that it is not possible to find the nth value of our list without going through potential entries, and so the problem is in $\Omega(n)$.

We may then wonder whether it is possible to provide a linear algorithm that provides the answer. If we look at the list in question, we find that it is

$$2,\ 3,\ 4,\ 8,\ 9,\ 10,\ 14,\ 15,\ 16,\ 20,\ 21,\ 22,\ \ldots$$

This shows a distinct pattern: if we have a number of the form $6k + 2$, where $k \in \mathbb{N}$, then this number and the next two, $6k+3$ and $6k+4$, are elements of the set, but $6k+5$ and $6k+1$ are not because they are not divisible by either 2 or 3, and $6k$ is divisible by both.

This analysis allows us to devise a linear algorithm:

```
z := 0;  x := 2;
while z<n do
```

```
z := z+1;
if z<n then
   z := z+1; x := x+1;
   if z<n then
      z := z+1; x := x+1;
      if z<n then
         x := x+4;
      else
         skip;
   else
      skip;
else
   skip;
```

Hence we have shown that this is a linear problem.

Chapter 4

Proving While Programs Correct

In Chapter 2 we provide a way of reasoning about the computation carried out when executing a While program by providing an account of how each step of the program affects the values held in the various variables mentioned by the program.

However, checking what happens when the program is executed separately for each state is not really feasible when we want to assure ourselves that a program behaves as intended.

In this chapter we address the question how one might go about formally establishing that a particular program does a particular job.

For While programs this can be formalized by establishing statements of the form:

If the start state satisfies property P
then after execution of the program the resulting state satisfies property Q.

We want to be able to formally prove that such statements hold, and that requires a system of reasoning rules that allow us to do so.

The aim is to prove, for example, that for the program from Example 3, it is the case that after execution has terminated, the value held in the variables `x` (and `y`) is indeed the greatest common divisor of the inputs.

Proving that programs behave in a particular way is not easy and costs a lot of time and effort—most of the software we use in every day life has not gone through a rigorous process like that. This also explains why we sometimes see unexpected or undesirable behaviour from programs we use.

The typical use-cases justifying the use of program correctness are safety critical systems and security systems.The following websites

https://en.wikipedia.org/wiki/List_of_software_bugs
http://www.cse.psu.edu/~gxt29/bug/softwarebug.html
https://www5.in.tum.de/~huckle/bugse.html

give numerous examples of software bugs that led to the loss of millions of dollars and even the loss of life. Many of these bugs were due to seemingly simple mistakes such as out by one errors[1]. Arguably these mistakes could not have been made if the techniques introduced in this chapter had been used.

4.1 Criteria for programs

There are a number of properties one might ask for to consider a program to be suitable.

At a minimum we expect programs to be **syntactically well-formed**. Every programmer has written programs that didn't satisfy this property at one point or another, for example by

[1]Some are clearly more complex, for example *race conditions* in concurrent software cannot be dealt with using techniques in this chapter, but there are formal methods that can be applied to detect such bugs.

misspelling keywords, missing semicolons, or otherwise not matching the grammar defining the language. Usually there's software that lets us know about such issues, for example a compiler or an IDE.

If we are working in a language with types we expect our programs to be **well typed**. Java is an example of a typed language, and again the programmer may get a complaint from the compiler, and some issues may only come to light at run time. While is not a typed language as such, but one could consider using an arithmetic expression where a boolean one is needed, or vice versa, a similar issue.

We also would like programs to be **well-behaved at run time**, in the sense that a Java program does not throw any exceptions, for example by trying to access an entry in an array outside of the indices for which the array is defined, or by trying to access an object in memory when using a reference that is a null pointer, or by dividing by 0. Different languages have different issues that may occur. This kind of potential issue does not arise with While programs.

We write programs for a reason: We want the program to do a particular job. We may think of a program that does the expected job as a **semantically correct** program—its behaviour is as intended. This chapter provides a framework both for making precise statements about the behaviour of a While program, and for proving that such statements hold.

A particular issue is that of **termination**. Although programs exist that we do not expect to terminate, such as the code controlling a server, which is supposed to provide access to date continuously, for While programs we are looking for answers to specific questions, and ideally we would like the program to terminate and provide us with the answer. However, as in Exercise 7 we may work with programs that are supposed to find a solution if one exists, and if we run our program in a situation where such a solution does not exist we expect our program to keep looking.

For programs that are not expected to terminate, other criteria exist that we would like to apply to them—students interested in this idea may want to research the concepts of *progress* or *fairness* in this context.

4.2 Example: Using predicates

We return to our example program

```
r :=m;
d:= 0;
while n≤r do
    d:=d+1;  r:=r−n
```

from Example 1 that implements integer division for natural numbers. Readers who prefer to see the formal rules before looking at examples are directed to Section 4.4.5 for the rules with explanations, or at Section 4.6 for the entire system in a compact format.

We think of this program as expecting inputs in the variables m and n and producing outputs in the values held in the variables d and r once the program terminates. This program is intended to be used for inputs from the *natural numbers*, and we don't want to divide by 0, so we are only interested in the question of what happens if we start the program in a state σ that satisfies

$$\sigma\mathrm{m} \geq 0 \wedge \sigma\mathrm{n} > 0.$$

However, we want our system not to refer to *individual states*, but instead to become *conditions for states*. We use the following notation

$$\mathrm{m} \geq 0 \wedge \mathrm{n} > 0$$

to create a condition which is satisfied for those states for which the values held in the named variables satisfy the given condition. This is known as a *predicates for states*.

We further expect that when the program terminates we have condition Q

$$\mathrm{m} = \mathrm{d} \cdot \mathrm{n} + \mathrm{r},$$

that is, our program computes the divisor and remainder when dividing m by n.

So if we know that when we start our program S in a state satisfying predicate P, and it ends in a state satisfying predicate Q, does that mean our program carries out the required computation? What about the following program?

```
m:= 0;  n:= 0;  d:= 0;  r:= 0
```

This program has the specified property but it does not carry out the computation we require. What has gone wrong is that we have not remembered the original input values! We need to be able to refer to specific natural numbers such as m and n being held in the corresponding variables when the program starts. We adjust our predicates as follows: The precondition should be

$$P : \ \mathtt{m} = m \wedge m \geq 0 \wedge \mathtt{n} = n \wedge n > 0,$$

and the postcondition

$$m = \mathtt{d} \cdot n + \mathtt{r}.$$

The difference between the new and the old postcondition is easy to miss, which is why we have circled the critical difference: In the new post condition we refer to the natural numbers m and n that provide the original values for the variables m and n respectively.

This allows us to ensure that for given input values the output values in d and r satisfy the required equality for the *input values*. But we still haven't quite captured what we are looking for: In order for

$$m = \mathtt{d} \cdot n + \mathtt{r}$$

to imply that

$$\mathtt{d} = m \operatorname{div} n \qquad \text{and} \qquad \mathtt{r} = m \bmod n$$

we also need that

$$0 \leq \mathtt{r} < n.$$

So we adjust the postcondition to

$$Q : \ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n.$$

We expect the system we produce to allow us to prove assertions written as

$$\{ \ P \ \} \ \mathtt{S} \ \{ \ Q \ \}$$

with the meaning just indicated. Here

- P is the **precondition**, that is, we are only interested in how the statement S behaves for start states that satisfy P;
- S is the While program we are considering and
- Q is the **postcondition**, that is the condition that is expected to hold after the termination of the statement S when started in a state satisfying the precondition.

The whole assertion is known as a **Hoare triple**, after the computer scientist Tony Hoare who came up with this system. The corresponding system is known as **Hoare logic**, or **Floyd Hoare logic** since Hoare was inspired by earlier work by Robert Floyd.

We revisit this example in Section 4.5 to prove that the above assertion holds for P and Q as given above.

4.3 Partial and total correctness

There are two different systems that are used to talk about the behaviour of While programs. Both of them make assertions consisting of a precondition, a statement and a postcondition, but they have different interpretations.

We use

$$\{\ P\ \}\ \mathtt{S}\ \{\ Q\ \}$$

to mean if S is executed in a state that satisfies the precondition P then if the program terminates, it does so in a state that satisfies the postcondition Q. Note that this does *not* make any claims about S terminating, so if S does not terminate, it satisfies this assertion by default. This is known as **partial correctness**: It does not make any guarantees that the program produces our desired result, it merely guarantees that if the program terminates then it does so.

We use

$$\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow Q\ \}$$

to mean that if S is executed in a state that satisfies precondition P then it terminates, and the postcondition Q holds. This makes a guarantee that our program will always solve our problem, and this is known as **total correctness**.

We require two systems to cope with both kinds of assertions.

Exercise 38.
Knowing that $\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow Q\ \}$ holds for some statement S is stronger than its counterpart. Justify that $\{\ P\ \}\ \mathtt{S}\ \{\ Q\ \}$ allows us to assert properties that are useful when it comes to describing the behaviour of While programs.

4.4 A system for partial correctness

We need to define a formal system that allows us to reason with Hoare triples. We have indicated that we expect the two conditions to be *predicates for states*, that is, given such a predicate and a state, the predicate either holds in that state or it does not (formally this makes it a function $\mathsf{St} \longrightarrow \{\mathbf{tt}, \mathbf{ff}\}$).

4.4.1 Generalized arithmetic and boolean expressions

We need to be able to make statements about the values of variables, which have integer values, and we need to be able to use integer operations, as well as integer numbers, in these statements.

One may think of this as providing a *generalized notion of boolean expressions*, now outside of the While language. This uses a *generalized notion of arithmetic expression*, using variables, integers, and properties of integers where we now do not restrict ourselves to the few operations in the While language. In other words, we allow ourselves to use expressions such as $m \operatorname{div} n$ or even $m \operatorname{div} \mathtt{n}$, and facts about integers such as prime factorization, see Fact 2, or significantly more sophisticated ones such as theorems from number theory.

Examples of such expressions appear above, in the form of $\mathtt{m} = m$ and $\mathtt{m} \geq 0$, or $m = \mathtt{d} \cdot n + \mathtt{r}$.

We then allow ourselves to combine such statements with the usual logical connectives, $\wedge$, $\vee$, $\neg$ and $\rightarrow$. We also need to be able to talk about updating a variable and we use the notation

$$P[\mathtt{x} \mapsto a],$$

where

- P is a predicate
- $\mathtt{x}$ is a variable and

- a is the interpretation in $\mathbb{Z}$ of a generalized arithmetic expression.

The predicate

$$P[\mathtt{x} \mapsto a] \qquad \text{holds for a state } \sigma$$

if and only if the predicate

$$P \qquad \text{holds for the state } \sigma[\mathtt{x} \mapsto a].$$

As before we use $\mathcal{A}$ and $\mathcal{B}$ to provide values for arithmetic and boolean expressions, but this time the state is left implicit (just as it is for the predicates). So we might have an assignment $\mathtt{x} := \mathtt{a}$ in the language, and we use $\mathcal{A}\, a$ to refer to the integer that we may think of as the interpretation of the expression a.

We use **tt** and **ff**, the potential results of evaluating a boolean expressions, as the constants for truth and falsehood in our logic for describing predicates.

4.4.2 Statements

We provide the rules that allow us to derive that a statement S upon termination produces a state that satisfies condition Q, provided it starts in a state satisfying condition P. The structure is similar to the rules that define the small and big step semantics.

bc := This rule describes the behaviour of the assignment instruction: If a state σ satisfies the predicate $P[\mathtt{x} \mapsto \mathcal{A}\, a]$, that is, if $\sigma[\mathtt{x} \mapsto \mathcal{A}\ \mathtt{a}\, \sigma]$, then after carrying out the assignment $\mathtt{x} := \mathtt{a}$ the resulting state satisfies the predicate P.

$$\frac{}{\{\ P[\mathtt{x} \mapsto \mathcal{A}\, a]\ \}\ \mathtt{x} := \mathtt{a}\ \{\ P\ \}}$$

bc skip The skip rule does not affect the state, and so any condition that holds before it is executed also holds after.

$$\frac{}{\{\ P\ \}\ \textbf{skip}\ \{\ P\ \}}$$

sc ; If when we execute S from a state satisfying P we go to a state satisfying Q, and if when we execute S' from a state satisfying Q we go to a state satisfying R then starting in a state satisfying P and executing S;S' gets us to a state satisfying Q.

$$\frac{\{\ P\ \}\ \mathtt{S}\ \{\ Q\ \}\ \ \{\ Q\ \}\ \mathtt{S'}\ \{\ R\ \}}{\{\ P\ \}\ \mathtt{S;S'}\ \{\ R\ \}}$$

sc if This is a bit trickier than one might expect: In order to say something about the properties of a state we are in after executing an if statement we need to be able to find a property that works no matter which branch we execute.

$$\frac{\{\ P \wedge \mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S}\ \{\ Q\ \}\ \ \{\ P \wedge \neg\mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S'}\ \{\ Q\ \}}{\{\ P\ \}\ \textbf{if}\ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \textbf{else}\ \mathtt{S'}\ \{\ Q\ \}}$$

sc while For a while loop there is not so much we are able to assert in general. We need a property that is unaffected by executing the body of a loop to obtain a valid Hoare triple.

$$\frac{\{\ P \wedge \mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S}\ \{\ P\ \}}{\{\ P\ \}\ \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}\ \{\ P \wedge \neg\mathcal{B}\,\mathtt{b}\ \}}$$

cons There is a final logical rule, the *rule of consequence*. This allows us to manipulate our pre- and postconditions using logic. It says that we may always use a stronger precondition and a weaker postcondition.

$$\text{if } P \to P' \text{ and } Q' \to Q \qquad \frac{\{\ P'\ \}\ \mathtt{S}\ \{\ Q'\ \}}{\{\ P\ \}\ \mathtt{S}\ \{\ Q\ \}}$$

The rule of consequence is very helpful in practice: When we compose statements to get longer programs we want to be able to reason about these statements to obtain a property of the longer program, but it isn't always the case that the postcondition Q we are able to establish for the statement S equals the precondition P' we use for the statement S', but as long as Q implies P, that is no problem.

More generally if we know that $\{\ P'\ \}\ \mathtt{S}\ \{\ Q'\ \}$, as well as $P \to P'$ and $Q' \to Q$ then if we run the statement S in a state σ that satisfies the predicate P, then it must also satisfy the predicate P' because P implies P', so if we run S in the state σ, we may invoke $\{\ P'\ \}\ \mathtt{S}\ \{\ Q'\ \}$ to obtain that if S terminates then the state we are in, say ρ, satisfies property Q'. But since that implies that ρ satisfies Q, we have indeed that we end in a state that satisfies the desired predicate Q.

For example, if we have

$$\{\ \mathtt{x} \geq 0\ \}\ \mathtt{S}\ \{\ \mathtt{x} = 3\ \}$$

we also have

$$\{\ \mathtt{x} = 5\ \}\ \mathtt{S}\ \{\ \mathtt{x} \geq 0\ \}.$$

If we start the execution of S in a state where x has the value 5, then it is the case that x has a value greater than or equal to 0, so we know that if S terminates then it will do so in a state where the value of x is 3, and that is a value greater than or equal to 0.

4.4.3 Loop invariants

It is treating loops correctly that is the trickiest part of establishing the behaviour of While programs. The only property we have to establish something about loops has the *same predicate P* appearing in the pre- and postcondition.

To use this rule we have to find a condition that *does not change* as the body of the loop is carried out. This is known as a **loop invariant**, and finding appropriate loop invariants for programs is a hard problem. Of course, it is possible to state such properties (for example, properties that cover only states not mentioned in the program at issue), but we need to pick our loop invariants in such a way that we may prove that our program does a particular job, which requires us to establish a particular postcondition, and that is something of an art.

4.4.4 Chaining assignment rules

The assignment rule looks odd at first sight and the notation may be a bit confusing at first. Effectively it says that any condition that is true before the assignment is carried out remains true if it is not affected by the variables whose value changes, we may adjust conditions for those variables, and we may *add* conditions for those variables.

Here are a few examples:

- If we carry out an assignment $\mathtt{x} := 1$ we may use $\mathtt{x} = 1$ in the postcondition.
- If we have $\mathtt{x} \in \mathbb{N}$ in the precondition and carry out the assignment $\mathtt{y} := \mathtt{y} + 1$ we have $\mathtt{x} \in \mathbb{N}$ in the postcondition since the assignment carried out does not affect the value held in the variable x.
- If we have $\mathtt{x} > 5$ in the precondition and we carry out the assignment $\mathtt{x} := \mathtt{x} + 1$ we have $\mathtt{x} > 6$ in the postcondition.

- If we have $\mathtt{x} + \mathtt{y} \leq 3$ in the precondition and we carry out the assignment $\mathtt{x} := \mathtt{x} - 1$ we have $\mathtt{x} + \mathtt{y} \leq 2$ in the postcondition.

To put it differently, the := rule behaves as one might expect it to behave. This makes it easy to describe successive assignments such as $\mathtt{x} := \mathtt{x} + 1; \mathtt{y} := \mathtt{y} \times 2$: Given a precondition, we adjust it based on the change to the value of the variable x to obtain an intermediate postcondition which becomes the precondition to the second assignment, and then we adjust it again based on the change to the variable y to obtain a final postcondition after both assignments have been carried out, but this happens in a very straightforward way.

In practice, we often use the rule of consequences when invoking the assignment rule to collect additional conditions that are known to hold given the precondition and the assignment we carry out

For the derivations we give below we treat several successive assignments in one step for these reasons.

4.4.5 A system for partial correctness

For convenience we provide the rules in a condensed format.

$$\frac{}{\{\ P[\mathtt{x} \mapsto \mathcal{A}\,a]\ \}\ \mathtt{x} := \mathtt{a}\ \{\ P\ \}} \qquad \frac{}{\{\ P\ \}\ \textbf{skip}\ \{\ P\ \}}$$

$$\frac{\{\ P\ \}\ \mathtt{S}\ \{\ Q\ \}\ \ \{\ Q\ \}\ \mathtt{S'}\ \{\ R\ \}}{\{\ P\ \}\ \mathtt{S;S'}\ \{\ R\ \}}$$

$$\frac{\{\ P \wedge \mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S}\ \{\ Q\ \}\ \ \{\ P \wedge \neg\mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S'}\ \{\ Q\ \}}{\{\ P\ \}\ \textbf{if}\ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \textbf{else}\ \mathtt{S'}\ \{\ Q\ \}}$$

$$\frac{\{\ P \wedge \mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S}\ \{\ P\ \}}{\{\ P\ \}\ \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}\ \{\ P \wedge \neg\mathcal{B}\,\mathtt{b}\ \}}$$

$$\text{if } P \to P' \text{ and } Q' \to Q \qquad \frac{\{\ P'\ \}\ \mathtt{S}\ \{\ Q'\ \}}{\{\ P\ \}\ \mathtt{S}\ \{\ Q\ \}}$$

We say that a Hoare triple is **valid** if it is indeed the case that if we start the program in question in a state that satisfies the give precondition then after the program terminates the resulting state satisfies the given postcondition.

4.5 Extended example: establishing partial correctness

As indicated above we would like to establish that if precondition P

$$\mathtt{m} = m \wedge m \geq 0 \wedge \mathtt{n} = n \wedge n > 0$$

holds when we run the program

```
r :=m;  d := 0;
while n≤r do
    d := d+1;  r := r−n
```

once it terminates the condition Q

$$m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n$$

holds.

We note that the precondition P is only concerned with the values of the variables m and n and their properties, and these do not change as our program is executed. We find it convenient to keep using the shortcut P for this condition.

We have, by the := rule and the ; rule, that

$$\{\ P\ \}\ \mathtt{r} := \mathtt{m};\ \mathtt{d} := 0\ \{\ P \wedge \mathtt{r} = m \wedge \mathtt{d} = 0\ \}.$$

This leaves us with understanding how the loop works. For that we need a loop invariant, and as indicated above, finding good loop invariants is difficult.

We may see that the property a loop invariant should satisfy is that it holds after the body of the loop has been executed, provided that the boolean expression b evaluates to true. We should think of a loop invariant as something that holds before the body of the loop is executed, and it holds again after the body of the loop has been executed (but it typically does not hold at every stage of executing the body).

There are some heuristics that help with finding loop invariants: They should involve the variables that change when we execute the loop, and they should have a connection with the the postcondition Q we are aiming for.

The body of our loop subtracts n from r (which starts out being m) and increases the count in d of how many times this has been carried out so far. It can help to look at specific instances of the values taken by the variables to gain an idea of what is happening. We repeat here the table from Example 8 for inputs $m = 10$ and $n = 3$.

	d	r
start	0	10
loop	1	7
loop	2	4
loop	3	1

We would like to show that $m = \mathtt{d} \cdot n + \mathtt{r}$, and we can see from the table that this is, in fact, a loop invariant: Before we carry out the loop we have $10 = 0 \cdot n + 10$, after the first time the body of the loop has been executed we have $10 = 1 \cdot 3 + 7$, then $10 = 2 \cdot 3 + 4$ and finally $10 = 3 \cdot 4 + 1$.

To make use of this we need to establish that

$$\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge n \leq \mathtt{r}\ \}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ m = \mathtt{d} \cdot n + \mathtt{r}\ \}.$$

After carrying out these assignment, the value of d has increased by 1 and the value of r has decreased by n, so for our loop invariant to still hold we need

$$m = (\mathtt{d} + 1) \cdot n + (\mathtt{r} - n) = \mathtt{d} \cdot n + n + \mathtt{r} - n = \mathtt{d} \cdot n + \mathtt{r},$$

which is the condition we have before we execute the two assignments that make up the body of our loop.

Using the while rule we therefore get

$$\{\ m = \mathtt{d} \cdot n + \mathtt{r}\ \}\ \textbf{while}\ \mathtt{n} \leq \mathtt{r}\ \textbf{do}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge \mathtt{r} < n\ \}.$$

This is almost what we want, in Section 4.2 we identified our desired postcondition as

$$m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n.$$

We are missing the part that says $0 \leq \mathtt{r}$, but that is also a loop invariant.

We may check that we have

$$\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge n \leq \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}.$$

The part that we haven't checked is the second condition, and by the := rule this needs to be $0 \leq \mathtt{r} - n$, which is ensured by $n \leq \mathtt{r}$.

Hence using the while rule we also have

$$\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}\ \textbf{while}\ \mathtt{n} \leq \mathtt{r}\ \textbf{do}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n\ \}.$$

Putting it all together: We have

$$\{\ P\ \}\ \mathtt{r} := \mathtt{m};\ \mathtt{d} := 0\ \{\ P \wedge r = m \wedge d = 0 \wedge m = \mathtt{d} \cdot n + \mathtt{r}\ \},$$

where we have strengthened our postcondition from above to get us to where we may make use of the loop invariant we have identified; in the presence of the other conditions we may verify $m = 0 \cdot n + m = \mathtt{d} \cdot n + \mathtt{r}$.

Now the postcondition for this statement almost matches the precondition for the loop: It is a bit stronger since it has $\mathtt{r} = m$, whereas the loop precondition only needs $\mathtt{r} \geq 0$. But the rule of consequence allows us to invoke the ; rule provided that the postcondition of the first Hoare triple implies the precondition of the second. Hence may use the ; rule to combine this assertion with the one above concerning the loop to obtain for the whole program S that

$$\{\ P\ \}\ \mathtt{S}\ \{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n\ \}.$$

Presenting this as a formal derivation is very space-consuming. Instead we use the same idea we employed when providing derivations for the big step semantics in Chapter 2, see Section 2.7.1 for an example. We give a summary of our reasoning from above in this style.

1. $\{\ P\ \}\ \mathtt{r} := \mathtt{m};\ \mathtt{d} := 0\ \{\ P \wedge \mathtt{r} = m \wedge \mathtt{d} = 0 \wedge m = \mathtt{d} \cdot n + \mathtt{r}\ \}$ by the := rule.

2. $\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge n \leq \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}$ by the := rule.

3. $\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}\ \textbf{while}\ \mathtt{n} \leq \mathtt{d}\ \textbf{do}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ m = \mathtt{d} \cdot n + r \wedge 0 \leq \mathtt{r} < n\ \}$ from 2 by the while rule.

4. $\{\ P\ \}\ \mathtt{r} := \mathtt{m};\ \mathtt{d} := 0;\ \textbf{while}\ \mathtt{r} \leq \mathtt{n}\ \textbf{do}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq r < n\ \}$ by the ; rule applied to 1 and 3 using the rule of consequence.

4.6 A system for total correctness

In Section 4.3 we discuss that there are two kinds of statements one might be aiming for, where the second guarantees that the given program terminates. Recall that we use the notation

$$\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow Q\ \}$$

to mean that if the statement S is executed in a state that satisfies predicate P then it terminates and is in a state that satisfies property Q. The only kind of statement that can cause non-termination is a loop, so it should come as no surprise that we may re-use most of the rules from the system for partial correctness, we just have to change the one for loops.

We look at the full system before having a closer look at the new rule.

4.6.1 A system for total correctness

$$\frac{}{\{\ P[\mathtt{x} \mapsto \mathcal{A}\,a]\ \}\ \mathtt{x} := \mathtt{a}\ \{\ \Downarrow P\ \}} \qquad \frac{}{\{\ P\ \}\ \textbf{skip}\ \{\ \Downarrow P\ \}}$$

$$\frac{\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow Q\ \}\ \ \{\ Q\ \}\ \mathtt{S'}\ \{\ \Downarrow R\ \}}{\{\ P\ \}\ \mathtt{S;S'}\ \{\ \Downarrow R\ \}}$$

$$\frac{\{\ P \wedge \mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S}\ \{\ \Downarrow Q\ \}\ \ \{\ P \wedge \neg\mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S'}\ \{\ \Downarrow Q\ \}}{\{\ P\ \}\ \textbf{if}\ \mathtt{b}\ \textbf{then}\ \mathtt{S}\ \textbf{else}\ \mathtt{S'}\ \{\ \Downarrow Q\ \}}$$

if $P \to a \geq 0$
$$\frac{\{\ P \wedge \mathcal{B}\,\mathtt{b} \wedge \mathcal{A}\,a = k\ \}\ \mathtt{S}\ \{\ \Downarrow P \wedge \mathcal{A}\,a < k\ \}}{\{\ P\ \}\ \textbf{while}\ \mathtt{b}\ \textbf{do}\ \mathtt{S}\ \{\ \Downarrow P \wedge \neg\mathcal{B}\,\mathtt{b}\ \}}$$

if $P \to P'$ and $Q' \to Q$
$$\frac{\{\ P'\ \}\ \mathtt{S}\ \{\ \Downarrow Q'\ \}}{\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow Q\ \}}$$

4.6.2 Loop variants

In the rule for loops we have a (generalized) arithmetic expression a. We may see that the expectation is that when the body of the loop is executed, a becomes smaller, and that we know that a is greater than or equal to 0. So this is an expression that gets smaller every time the body loop is executed, but it cannot become negative—so this is our guarantee that the loop cannot keep going forever in the presence of the condition P. We call such expressions **loop variants**. Finding loop variants usually requires less imagination than finding loop invariants–looking at the loop condition, and how the variables involved change as the loop is executed, is what we need to do. This only becomes complicated in situations where we need to refer to an external condition that ensures the loop terminates.

4.6.3 Extended example: establishing total correctness

We return to our program S that carries out integer division

```
r :=m;
d:= 0;
while n≤r do
    d:=d+1;  r:=r−n
```

In Section 4.5 we establish that for the precondition P

$$\mathtt{m} = m \wedge m \geq 0 \wedge \mathtt{n} = n \wedge n > 0$$

and the postcondition Q

$$m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n$$

we have

$$\{\ P\ \}\ \mathtt{S}\ \{\ Q\ \}.$$

But we expect S to always terminate, we should be able to establish that we also have the stronger assertion

$$\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow Q\ \}.$$

Since the two systems have much the same rules we are able to use our previous arguments to obtain

$$\{\ P\ \}\ \mathtt{r} := \mathtt{m};\ \mathtt{d} := 0\ \{\ \Downarrow P \wedge \mathtt{r} = m \wedge \mathtt{d} = 0 \wedge m = \mathtt{d} \cdot n + \mathtt{r}\ \}.$$

We look at the loop. In this case it is quite easy to see that a loop variant is readily available: The variable r decreases (by n) each time the body of the loop is carried out, and the loop ensures that we stop when r has reached 0 (since we know that n is at least 1).

From Section 4.5 we know that

$$\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge n \leq r \wedge 0 \leq r\ \}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}.$$

Since this program consists of two assignments and therefore terminates, we also have

$$\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge n \leq \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ \Downarrow m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}.$$

We note that one of the two conditions we require, namely that our proposed loop variant r is at least 0, is already part of our condition. So we just have to ensure that we add an additional condition that establishes that r decreases each time. Indeed, the := rule gives us that

$$\{m = \mathtt{d} \cdot n + \mathtt{r} \wedge n \leq \mathtt{r} \wedge 0 \leq \mathtt{r} \wedge \mathtt{r} = k\}\mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\{\Downarrow m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} \wedge r = k - n\},$$

and since $n \geq 1$, that implies that $r < k$ in the postcondition. Using the while rule we get

$$\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}\ \textbf{while}\ \mathtt{n} \leq \mathtt{r}\ \textbf{do}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ \Downarrow m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n\ \}.$$

We may now use the ; rule to put this together with our assertion regarding the first two assignments, after again weakening the postcondition for the two initial statements to obtain the precondition for the loop, to get the desired

$$\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n\ \}.$$

So now we know that when started with values in variables m and n satisfying the criteria given in P our program always terminates and the values in r and d give the desired result of the remainder and quotient for integer division for m and n.

We once again provide a write-up using our established format for derivations of this kind.

1. $\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge n \leq \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ \Downarrow m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}$ by the := rule.

2. $\{m = \mathtt{d} \cdot n + \mathtt{r} \wedge n \leq \mathtt{r} \wedge 0 \leq \mathtt{r} \wedge \mathtt{r} = k\}\mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\{\Downarrow m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} \wedge r < k\}$ again by the := rule.

3. $\{\ m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}\ \}\ \textbf{while}\ \mathtt{n} \leq \mathtt{r}\ \textbf{do}\ \mathtt{d} := \mathtt{d} + 1;\ \mathtt{r} := \mathtt{r} - \mathtt{n}\ \{\ \Downarrow m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n\ \}$ from 2 using the while rule.

4. $\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r} < n\ \}$ from 1, 3 and noting that the postcondition from 1 implies the precondition from 3.

4.7 Further examples

We produce further examples to show readers a wider range of arguments one might construct. In some cases we do not carry out a full proof that the program does implement what we intended, but instead we look at properties that provide us with some of what we meant the program to do—this is to keep the arguments easier to follow.

In Section 4.5 we note that coming up with suitable conditions to fully describe the behaviour of a program is not easy—our original conditions were satisfied by a program that does certainly not provide the desired solution. For the examples that follow we encourage readers to think about what one should be establishing, and what that looks like in our language of post- and preconditions.

4.7.1 If statements

We start with a simple example to look in more detail at conditionals without the presence of loops. The following program implements the function $\beta : \mathbb{Z} \longrightarrow \mathbb{N}$ from Section 6.2.1, assuming the input is given by n and the output by x.

```
if 0≤n then
   x := 2×n
else
   x := −2×n−1
```

One would typically want to show that this is indeed an implementation of the function β. The most interesting part of that is to show that the output is always a natural number, and that is what we establish here. Readers are invited to consider how to add to our postcondition and to establish the intended behaviour in its entirety.

Assume we want to show that for this program S we have that

$$\{\ \ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} \in \mathbb{N}\ \}.$$

We are able to make use of the condition that governs the conditional statement when establishing properties of the two assignments.

1. $\{\ 0 \leq \mathtt{n}\ \}\ \mathtt{x} := 2 \times \mathtt{n}\ \{\ \Downarrow\ \mathtt{x} \in \mathbb{N}\ \}$ by the := rule.

2. $\{\ \mathtt{n} < 0\ \}\ \mathtt{x} := -2 \times \mathtt{n} - 1\ \{\ \Downarrow\ \mathtt{x} \in \mathbb{N}\ \}$ also by the := rule observing that $-2\mathtt{n} - 1 \geq 0$ for $\mathtt{n} < 0$ since multiplying both sides by -2 gives $-2\mathtt{n} > 0$, and subtracting one on both sides the desired statement.

3. $\{\ \ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} \in \mathbb{N}\ \}$ from 1 and 2 by the if rule.

Exercise 39.
For the program S

```
if 0≤n then
   x := 2×n
else
   x := −2×n −1
```

which implements the function β from Section 6.2.1 carry out the following tasks.

(a) Give a Hoare triple that verifies that this program implements the given function.

(b) Adjust the derivation given above to establish that the Hoare triple from part (a) is valid.

4.7.2 Greatest common divisor

We next look at the program computing the greatest common divisor of two numbers given in m and n in $\mathbb{N}$, and produces the output in $\mathtt{x} = \mathtt{y}$, see Example 3.

```
x:=m; y:=n;
while ¬x=y do
   if y<x then
      x := x−y
   else
      y := y−x;
```

Fully establishing that this computes the greatest common divisor seems to require quite complicated conditions, and we aim for something weaker. Again we encourage readers to first of all think about how one would have to strengthen the postcondition to fully describe what the intended behaviour of this program.

We use S for this program, L for its loop and I for the if statement. We note that our program does not work if one of m or n is 0 (unless they are both 0), and we ensure that we only look at non-zero inputs.

We would (eventually) like to show that the output is a divisor of the two input numbers.

$$\{\ \mathtt{m} = m \in \mathbb{N} \setminus \{0\} \wedge \mathtt{n} = n \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} \text{ divides } m \text{ and } n\ \}.$$

We begin by looking at a partial correctness proof of this statement—see Section 4.6.3 for an example that establishes that we are able to reuse most of that derivation to obtain total correctness if we can be sure that our program terminates.

We observe first of all that we have

$$\{\ \mathtt{m} = m \in \mathbb{N} \setminus \{0\} \wedge \mathtt{n} = n \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{x} := \mathtt{m};\ \mathtt{y} := \mathtt{n}\ \{\ \mathtt{m} = m = \mathtt{x} \in \mathbb{N} \setminus \{0\} \wedge \mathtt{n} = n = \mathtt{y} \in \mathbb{N} \setminus \{0\}\ \}$$

by the assignment rule, but we suspect that we will need to strengthen our conditions to cope with the loop. In what follows we use P for the predicate $\mathtt{x} \in \mathbb{N} \setminus \{0\} \wedge \mathtt{y} \in \mathbb{N} \setminus \{0\}$.

What loop invariant may we be aiming for? We may see that precisely one of x and y changes each time the body of the loop is executed, and unlike our original extended example, there is no obvious equality we may use here. But we do expect to be able to state something about divisibility here.

We expect that for $i \in \mathbb{N}$ we have i divides m and n if and only if i divides both x and y after each time the body of the loop has been executed. Since the output is x this implies that x divides both m and n, so it does imply the conclusion we are interested in.

Let R be the condition $\forall i \in \mathbb{N}.\,(i \text{ divides } m,\ n) \leftrightarrow (i \text{ divides } x \text{ and } y)$.

So we are aiming to show

$$\{\ P \wedge R\ \}\ \textbf{while}\ \neg\mathtt{x} = \mathtt{y}\ \textbf{do}\ \mathtt{I}\ \{\ P \wedge R \wedge \mathtt{x} = \mathtt{y}\ \}.$$

For that in turn we need

$$\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y})\ \}\ \mathtt{I}\ \{\ P \wedge R\ \}$$

to be able to apply the while rule.

That in turn means we need

$$\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge \mathtt{y} < \mathtt{x}\ \}\ \mathtt{x} := \mathtt{x} - \mathtt{y}\ \{\ P \wedge R\ \}$$

as well as

$$\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge \mathtt{y} \geq \mathtt{x}\ \}\ \mathtt{y} := \mathtt{y} - \mathtt{x}\ \{\ P \wedge R\ \}$$

in order to be able to apply the if rule. Having a double implication in R makes it a bit harder to see whether this is the case.

We look at the case where $\mathtt{y} < \mathtt{x}$. Assume that we have R and $\mathtt{y} < \mathtt{x}$. We now update x to $\mathtt{x} - \mathtt{y}$. If we have that $i \in \mathbb{N}$ divides both m and n then it divides x and y before the update, and so it also divides $\mathtt{x} - \mathtt{y}$. On the other hand, if $i \in \mathbb{N}$ divides both y and $\mathtt{x} - \mathtt{y}$ then it also divides x, and by R that means it divides both m and n. The else case is very similar.

What about the condition P? We subtract only if the two values of x and y are different, so we do not get the value 0 when we carry out either of the two assignments.

We may summarize our argument, making some adjustments to our conditions.

1. $\{\ \mathtt{m} = m \in \mathbb{N} \setminus \{0\} \wedge \mathtt{n} = n \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{x} := \mathtt{m};\ \mathtt{y} := \mathtt{n}\ \{\ \mathtt{x} = m \wedge \mathtt{y} = n \wedge P \wedge R\ \}$ by the := rule, observing that adding the condition R to the postcondition is safe.

2. $\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge \mathtt{y} < \mathtt{x}\ \}\ \mathtt{x} := \mathtt{x} - \mathtt{y}\ \{\ P \wedge R\ \}$ by the := rule, as argued in the text above.

3. $\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge \mathtt{y} \geq \mathtt{x}\ \}\ \mathtt{y} := \mathtt{y} - \mathtt{x}\ \{\ P \wedge R\ \}$ by the := rule, again as argued in the text above.

4. $\{\ P \wedge R \wedge \neg(x = y)\ \}\ \mathtt{I}\ \{\ R\ \}$ from 2 and 3 using the if rule.

5. $\{\ P \wedge R\ \}$ **while** $\neg\mathtt{x} = \mathtt{y}$ **do** $\mathtt{I}\ \{\ P \wedge R \wedge \mathtt{x} = \mathtt{y}\ \}$ from 4 by the while rule.

6. $\{\ \mathtt{m} = m \in \mathbb{N} \setminus \{0\} \wedge \mathtt{n} = n \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{S}\ \{\ P \wedge R \wedge \mathtt{x} = \mathtt{y}\ \}$ from 1 and 5 by the ; rule observing that the postcondition of 1 implies the precondition of 5.

It is not completely clear at this stage that we have what we were aiming for, namely that once the program terminates we know that $\mathtt{x} = \mathtt{y}$ divides both m and n.

But our condition R is strong enough to give us even more: We claim that this tells us that once the program terminates $\mathtt{x} = \mathtt{y}$ is indeed the greatest common divisor of m and n. This is an argument completely independent from our system, and is entirely concerned with mathematics.

Firstly it is the case that $\mathtt{x}$ divides itself (and so also $\mathtt{y}$), and so our condition R tells us that $\mathtt{x}$ divides both m and n, which was our original goal.

But we also know that any number k which divides both m and n must divide $\mathtt{x} = \mathtt{y}$, and that ensures that we have indeed found the greatest common divisor of m and n since their greatest common divisor must divide $\mathtt{x}$ and $\mathtt{x}$ itself is a divisor they have in common.

So we set out to prove something stronger than we were able to establish!

If we now want to extend the derivation from above to obtain

$$\{\ \mathtt{m} = m \in \mathbb{N} \setminus \{0\} \wedge \mathtt{n} = n \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{S}\ \{\ \Downarrow R\ \}$$

we need to concentrate on finding a loop variant in the first instance, and then we would expect to reuse most of the partial correctness derivation from above.

To get a loop variant we have to identify an expression that is guaranteed to be greater than or equal to 0 and which is also guaranteed to become smaller each time the body of the loop is executed. That rules out both $\mathtt{x}$ and $\mathtt{y}$ since neither of those is guaranteed to become smaller when the body of the loop is executed, but we may address that issue by using $\mathtt{x} + \mathtt{y}$: One of them is guaranteed to become smaller each time the body of the loop is executed, and so this idea works.

Hence we want to establish

$$\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge 0 \leq \mathtt{x} + \mathtt{y} = k\ \}\ \mathtt{I}\ \{\ \Downarrow P \wedge R \wedge 0 \leq \mathtt{x} + \mathtt{y} < k\ \}.$$

For this we need

$$\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge 0 \leq \mathtt{x} + \mathtt{y} = k \wedge y < x\ \}\ \mathtt{x} := \mathtt{x} - \mathtt{y}\ \{\ \Downarrow P \wedge R \wedge 0 \leq \mathtt{x} + \mathtt{y} < k\ \},$$

and this is valid based on the assignment rule since we know that $\mathtt{x} > \mathtt{y}$ and that $\mathtt{y} > 0$. We similarly obtain the required property for other branch of the if statement. Hence overall we obtain the following derivation:

1. $\{\ \mathtt{m} = m \in \mathbb{N} \setminus \{0\} \wedge \mathtt{n} = n \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{x} := \mathtt{m};\ \mathtt{y} := \mathtt{n}\ \{\ \Downarrow \mathtt{x} = m \wedge \mathtt{y} = n \wedge P \wedge R\ \}$ from 1 in the previous derivation since the rules are identical for assignments.

2. $\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge \mathtt{x} + \mathtt{y} = k \wedge \mathtt{y} < \mathtt{x}\ \}\ \mathtt{x} := \mathtt{x} - \mathtt{y}\ \{\ \Downarrow P \wedge R \wedge \mathtt{x} + \mathtt{y} < k\ \}$ as in 2 from the previous derivation, observing that adding the condition about $\mathtt{x}$ and $\mathtt{y}$ is safe.

3. $\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge \mathtt{x} + \mathtt{y} = k \wedge \mathtt{y} \geq \mathtt{x}\ \}\ \mathtt{y} := \mathtt{y} - \mathtt{x}\ \{\ \Downarrow P \wedge R \wedge \mathtt{x} + \mathtt{y} < k\ \}$ as in 3 from the previous derivation, and again observing that the additional condition is satisfied.

4. $\{\ P \wedge R \wedge \neg(\mathtt{x} = \mathtt{y}) \wedge \mathtt{x} + \mathtt{y} = k\ \}$ `I` $\{\ \Downarrow\ P \wedge R \wedge \mathtt{x} + \mathtt{y} < k\ \}$ from 2 and 3 using the if rule.

5. $\{\ P \wedge R\ \}$ **while** $\neg\mathtt{x} = \mathtt{y}$ **do** `I` $\{\ \Downarrow\ P \wedge R \wedge \mathtt{x} = \mathtt{y}\ \}$ by the while rule.

6. $\{\mathtt{m} = m \in \mathbb{N} \setminus \{0\} \wedge \mathtt{n} = n \in \mathbb{N} \setminus \{0\}\}$ `S` $\{\ \Downarrow\ P \wedge R \wedge \mathtt{x} = \mathtt{y}\}$ from 1 and 5 by the ; rule observing that the postcondition of 1 implies the precondition of 5.

This concludes a proof that provided the inputs are non-zero natural numbers, our programme terminates and the output is indeed the greatest common divisor of the two inputs.

4.7.3 Solving a quadratic equation

In Example 2 we give a program that aims to find an integer solution to a quadratic equation, and we repeat that program here.

```
x := 0;  y := 0;
while ¬y=1 do
    if a×x×x+b×x+c=0 then
        y := 1
    else
        if a×x×x−b×x+c=0 then
            y := 1
        else
            skip
    x := x+1
```

Which Hoare triple might one want to prove valid for this program? Unless one has a precondition that ensures the equation has an integer solution, one can only hope to prove partial correctness for this program.

For example, we expect to be able to show for this program `S` that

$$\{\ \mathtt{a} = a \wedge \mathtt{b} = b \wedge \mathtt{c} = c\ \}\ \mathtt{S}\ \{\ a\mathtt{x}^2 + b\mathtt{x} + c = 0 \vee a(-\mathtt{x})^2 + b(-\mathtt{x}) + c = 0\ \}.$$

A more challenging Hoare triple to derive would be

$$\{\ P \wedge \exists x \in \mathbb{Z}.\, ax^2 + bx + c = 0\ \}\ \mathtt{S}\ \{\ \Downarrow\ R\ \}.$$

We look at the first Hoare triple.

We are going to use the precondition $\mathtt{a} = a \wedge \mathtt{b} = b \wedge \mathtt{c} = c$ quite a bit so we are going to use P for this condition, and we are going to use $Q(\mathtt{x})$ for the condition

$$a\mathtt{x}^2 + b\mathtt{x} + c = 0 \vee a(-\mathtt{x})^2 + b(-\mathtt{x}) + c = 0.$$

We use `S` for the given program, `B` for the body of the loop, `I` for the first, and `I'` for the second if statement.

To show the claim we need to be able to show

$$\{\ P \wedge \mathtt{x} = 0 \wedge \mathtt{y} = 0\ \}\ \textbf{while}\ \neg\mathtt{y} = 1\ \textbf{do}\ \mathtt{B}\ \{\ \mathtt{y} = 1 \wedge Q(\mathtt{x})\ \},$$

and for that we need a loop invariant. This is another example where we are able to use what we would like to show as the invariant, namely

$$\mathtt{y} = 1 \leftrightarrow Q(\mathtt{x}).$$

But this doesn't quite work for the program as given because it may fail to be true before we enter the loop if 0 is a solution for our quadratic equation. We make some adjustments to our program.

```
x := 0;  y := 0;
if c=0 then
   y := 1;
else
   skip
while ¬y=1 do
   x := x+1
   if a×x×x+b×x+c=0 then
       y := 1
   else
      if a×x×x−b×x+c=0 then
          y := 1
      else
         skip
```

Now we are able to provide the desired derivation. The predicates in use are quite large. We use $Q'(\mathrm{x})$ if $a\mathrm{x}^2 + b\mathrm{x} + c = 0$, that is, if x is a solution to the given quadratic equation, and $Q''(\mathrm{x})$ if $-\mathrm{x}$ is a solution.

1. $\{\ P\ \}$ $\mathrm{x} := 0;\ \mathrm{y} := 0$ $\{\ P \wedge \mathrm{x} = 0 \wedge \mathrm{y} = 0\ \}$ by the := rule.

2. $\{\ P \wedge \mathrm{x} = 0 \wedge \mathrm{y} = 0 \wedge \mathrm{c} = 0\ \}$ $\mathrm{y} := 1$ $\{\ P \wedge \mathrm{x} = 0 \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ by the := rule since we set y to 1 if and only of 0 solves the equation.

3. $\{\ P \wedge \mathrm{x} = 0 \wedge \mathrm{y} = 0 \wedge \neg(\mathrm{c} = 0)\ \}$ **skip** $\{\ P \wedge \mathrm{x} = 0 \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ by the skip rule since we set y to 1 if and only of 0 solves the equation.

4. $\{\ P \wedge \mathrm{x} = 0 \wedge \mathrm{y} = 0\ \}$ **if** $\mathrm{c} = 0$ **then** $\mathrm{y} := 1;$ **else skip** $\{\ P \wedge \mathrm{x} = 0 \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ by the if rule.

5. $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x})) \wedge \neg(\mathrm{y} = 1)\ \}$ $\mathrm{x} := \mathrm{x} + 1$ $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x} - 1)) \wedge \neg(\mathrm{y} = 1)\ \}$ by the := rule.

6. $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x} - 1)) \wedge \neg(\mathrm{y} = 1) \wedge Q'(\mathrm{x})\ \}$ $\mathrm{y} := 1$ $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ by the := rule since x is a solution.

7. $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x} - 1)) \wedge \neg(\mathrm{y} = 1) \wedge \neg Q'(\mathrm{x}) \wedge Q''(\mathrm{x})\ \}$ $\mathrm{y} := 1$ $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ by the := rule since $-\mathrm{x}$ is a solution.

8. $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x} - 1)) \wedge \neg(\mathrm{y} = 1) \wedge \neg Q'(\mathrm{x}) \wedge \neg Q''(\mathrm{x})\ \}$ **skip** $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ by the skip rule and the rule of consequence since the precondition ensures that neither x nor $-\mathrm{x}$ is a solution and so the precondition implies the postcondition.

9. $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x} - 1)) \wedge \neg(\mathrm{y} = 1) \wedge \neg Q'(\mathrm{x})\ \}$ I' $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x})\ \}$ from 7 and 8 by the if rule.

10. $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x} - 1)) \wedge \neg(\mathrm{y} = 1)\ \}$ I $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ from 6 and 9 by the if rule.

11. $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x})) \wedge \neg(\mathrm{y} = 1)\ \}$ B $\{\ P \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ from 5 and 10 by the ; rule.

12. $\{\ P \wedge \mathrm{x} = 0 \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ **while** $\neg\mathrm{y} = 1$ **do** B $\{\ P \wedge \mathrm{y} = 1 \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x})\ \}$ from 11 by the while rule.

13. $\{\ P\ \}$ S $\{\ P \wedge \mathrm{y} = 1 \wedge (\mathrm{y} = 1 \leftrightarrow Q(\mathrm{x}))\ \}$ from 1, 4 and 12 by the ; rule used twice.

This ensures that if our program claims to have found a solution such a solution does indeed exist, and it is either x or its negative. We are not able to give a total correctness derivation for this Hoare triple since there may not be an integer solution to the given quadratic equation, in which case this program fails to terminate.

If we know that there exists $k \in \mathbb{Z}$ that solves the equation then we could use $|k| - \mathtt{x}$ as a loop variant, and it would indeed be possible to show the total correctness Hoare triple from above. We do not give such a derivation.

4.7.4 The Collatz conjecture

For the following While program S it is not known whether it terminates for all inputs $n \in \mathbb{N}$ provided via the variable n:

```
while ¬n=1 do
   x:=n; m:=0;
   while 2≤x do
      x:=x−1; m:=m+1;
   if x=0 then
      n:=m
   else
      n:=3×n+1
```

The Collatz conjecture says that this program terminates for all natural number inputs (in which case the value of n upon termination is 1). If this conjecture is true then

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ n = 1\ \}.$$

What is currently known is that

$$\{\ \mathtt{n} = n \in \mathbb{N} \wedge n \leq 2.95 \cdot 10^{2}0\ \}\ \mathtt{S}\ \{\ \Downarrow\ n = 1\ \}.$$

We are also able to assert the following partial correctness claim

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ n = 1\ \}$$

since for the program to terminate the value in n must be 1.

4.8 Establishing properties using Hoare triples

We provide more general guidance on using our two systems.

4.8.1 Which Hoare triple to aim for?

Coming up with a Hoare triple that appropriately describes the behaviour we expect from our program is non-trivial. In the examples detailed above there is some discussion about what one might aim for. A complete description of the expected behaviour may be complex, and that may make it harder to establish. In these situations one might first aim to find derivations for a partial description of the desired behaviour. For example, in Section 4.7.2 our original aim was to show that the output divides both the inputs.

Pitfalls of not correctly capturing the desired behaviour appear in Section 4.2, where a trivial program is given that fulfils the original specification, but does not provide the intended solution.

When a Hoare triple has been chosen, a next step should be to consider whether there are ways of satisfying that with a program that does not behave as intended.

4.8.2 Loop invariants

Finding suitable loop invariants as a general problem is hard. A typical loop has infinitely many invariants (we may always add properties irrelevant to the computation at hand), and there is not just one that is suitable to establish a particular property.

We provide some guidance on what to look for:

- The loop invariant has to relate to the property we are aiming to establish for the loop. For our initial example on modular arithmetic, part of the intended postcondition for the program $m = \mathtt{d} \cdot n + \mathtt{r} \wedge 0 \leq \mathtt{r}$ turned out to be a loop invariant, but that is unusual. For the example concerned with the greatest common divisor (see Section 4.7.2) we required a logically strong statement, while for the program that solves a quadratic equation we ensure that our marker for having found a solution is only set when a solution has been found.

- Typically a loop invariant involves many of the variables that are named in the body of the loop. In the initial example, the variables `d` and `r` are the only variables mentioned in the body of the loop, and the same is true for the greatest common divisor and quadratic equation examples.

- Consider how the variables change, and whether these changes can be used to establish an equality, or another joint property (as in the greatest common divisor example).

4.8.3 Loop variants

For the derivation of total correctness triples we also need to identify loop variants. This is typically easier to do: To establish total correctness the program has to terminate for the given precondition. That means that the boolean expression governing the loop must become false after executing the body of the loop a finite number of times. The loop condition therefore provides us with variables to consider.

However, it is not necessarily the case that we may easily define a variable (or an arithmetic expression) that decreases every time the loop is executed.

We may have a while loop that is similar to a for loop in behaviour, of the form

```
x := 1;
while x≤n do
    B;
    x := x+1
```

where `B` is a statement that does not change the values of `x` and `n` and the value of $\mathtt{n} = n$ is a positive integer. Clearly this loop terminates after the body has been executed n many times, but `x` does not decrease each time.

In this situation the expression $\mathtt{n} - \mathtt{x}$ is an expression that reduces each time the body of the loop is executed, and it is also guaranteed to remain greater than or equal to 0, so it is a suitable loop variant.

In Section 4.7.2 we needed to use $\mathtt{x} + \mathtt{y}$ to obtain an expression satisfying the condition of decreasing on each iteration and being guaranteed to remain greater than or equal to 0.

Conditions that are harder to deal with are those where we use a variable as a switch, the boolean expression governing the loop is something like $\neg(x = 1)$, and the loop eventually sets the switch `x` to 1. In that situation analysing why the loop terminates may be significant challenge.

Exercise 40.

For the following problems and programs, provide Hoare triples that describe the expected behaviour of the program as fully as possible.

First state a condition that confirms that the program carries out the task we intended it to do, then aim for a condition that describes any additional behaviour, for example for inputs

that don't match the intended interpretation.

(a) The program determining whether a given input is a prime number, Example 4.

(b) The program from Exercise 5 that computes $n \bmod m$ for integers n and m.

(c) For the program that implements the function ϕ from Section 6.2.2 that appears in Exercise 9.

(d) For the program that implements the function ϕ' from Section 6.2.2 that appears in Exercise 9 if we are only allowed to refer to the function ϕ in the pre- and postcondition.

(e) For the two programs from Exercise 19.

Exercise 41.
The first While program given in Chapter 1 is one that computes the power of 2 for an input. More generally, here is again a program that computes m^n for values given via the input variables m and n.

```
x := 1;  y := n;
while y>1 do
    x := x×m;  y := y−1
```

Carry out the following tasks.

(a) Scrutinize the program to ensure it works for all suitable inputs and adjust it if necessary. Call the resulting program S.

(b) Write down a precondition P and a postcondition Q that give a Hoare triple that describes the expected behaviour of this program.

(c) Identify a loop invariant and provide an argument that it is one.

(d) Give a derivation of $\{\ P\ \}$ S $\{\ Q\ \}$.

(e) Identify a loop variant.

(f) Adjust your derivation from part (d) to derive $\{\ P\ \}$ S $\{\ \Downarrow Q\ \}$.

Exercise 42.
Consider the following program S.

```
x := 0;  i := 1;
while i≤n do
    x := x+i;
    i := i+1
```

We want to show that this program computes the sum of the first n positive integers.

(a) Write down a precondition P and a postcondition Q that give a Hoare triple that describes the expected behaviour of this program.

(b) Identify a loop invariant and provide an argument that it is one.

(c) Give a derivation of $\{\ P\ \}$ S $\{\ Q\ \}$.

(d) Identify a loop variant.

(e) Adjust your derivation from part (c) to derive $\{ P \}$ S $\{ \Downarrow Q \}$.

Exercise 43.
We may compute an integer approximation to the logarithm (for base 2) as follows, where n is the input and z the output variable.

```
x := 1;  z := 0;
while x×2≤n do
    z := z+1;  x := x×2
```

Carry out the following tasks.

(a) What do we mean by 'integer logarithm'? Write down your understanding of this idea.

(b) Write down a precondition P and a postcondition Q that give a Hoare triple that describes the expected behaviour of this program.

(c) Identify a loop invariant and provide an argument that it is one.

(d) Give a derivation of $\{ P \}$ S $\{ Q \}$.

(e) Identify a loop variant.

(f) Adjust your derivation from part (c) to derive $\{ P \}$ S $\{ \Downarrow Q \}$.

Exercise 44.
Carry out the following tasks.

(a) Give a While program that tells us whether the final digit of the input $n \in \mathbb{N}$ is divisible by 3. *Hint: You'll make the following tasks easier if you make your program simple rather than efficient.*

(b) Write down a precondition P and a postcondition Q that give a Hoare triple that describes the expected behaviour of this program.

(c) Identify loop invariants and provide an argument that they are.

(d) Give a derivation of $\{ P \}$ S $\{ Q \}$.

(e) Identify a loop variant.

(f) Adjust your derivation from part (c) to derive $\{ P \}$ S $\{ \Downarrow Q \}$.

Exercise 45.
Consider the alternative language to While where while loops have been replaced by 'repeat until' loops, compare Exercise 22.

(a) For this language provide a 'repeat until' rule for partial correctness that replaces the while rule.

(b) Show that if $\{ P \}_r$ **repeat** S **until** b $\{ Q \}_r$ holds in your system then for your solution S' to Exercise 3 we have $\{ P \}$ S' $\{ Q \}$ in the system given in the notes.

4.9 Properties of our systems

Above we appeal to ideas from the operational semantics to justify the rules we give for the two systems. There is, indeed, a connection between the two.

Proposition 11. *Let P and Q be predicates for states and let* S *be a statement.*

(i) We have

$$\{\ P\ \}\ \mathtt{S}\ \{\ Q\ \}$$

if and only if for every state σ satisfying the predicate P, if there exists a (necessarily unique) state σ' with

$$\langle \mathtt{S}, \sigma \rangle \Rightarrow^* \langle \mathbf{skip}, \sigma' \rangle$$

then σ' satisfies the predicate Q.

(ii) We have

$$\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow Q\ \}$$

if and only if for every state σ that satisfies the predicate P there exists a (necessarily unique) state σ' satisfying the predicate Q with

$$\langle \mathtt{S}, \sigma \rangle \rightarrow \Downarrow \sigma'.$$

We do not provide a proof of this result but we encourage readers to think a little about what such a proof might look like.

4.10 Summary

In this section we establish two formal systems that allow us to prove that a given program shows a particular behaviour across a variety of states. This is done by stating a precondition—we only make claims for those states which satisfy this precondition, and a postcondition which establishes properties that we know hold when the program terminates. One of our systems only makes claims *in case* the program terminates, the other system also guarantees termination. There is a formal connection between these two systems and the small step, respectively the big step, semantics given in Proposition 11.

4.11 Solutions to Exercises

Solution 38.
A relevant example is given by the programs that appear in Exercise 7 or Example 2. If we were only able to make assertions about the total correctness of programs we would not be able to state that *if* the given equality has a solution then if our programs terminates they have found a valid solution.

If we only had the system for total correctness we could only make claims that are satisfied by this program provided the precondition guarantees that a solution exists. It may not be possible to determine this at all for some problems.

Solution 39.
We solve each part.

(a) We have to ensure that our program works appropriately for all inputs as held in the variable n by producing the appropriate output in the variable x.

$$\{\ \mathtt{n} = n\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = \beta n\ \}.$$

(b) The derivation is almost identical to that given in the text:

1. $\{\mathtt{n} = n \wedge 0 \leq \mathtt{n}\}\ \mathtt{x} := 2 \times \mathtt{n}\ \{\Downarrow\ \mathtt{x} = \beta n\}$ by the := rule, observing that if n is a non-negative input then $\beta n = 2n$.
2. $\{\ \mathtt{n} = n \wedge \mathtt{n} < 0\ \}\ \mathtt{x} := -2 \times \mathtt{n} - 1\ \{\ \Downarrow\ \mathtt{x} = \beta n\ \}$ also by the := rule observing that if n is a negative input then $\beta n = -2n - 1$.
3. $\{\ \mathtt{n} = n\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = \beta n\ \}$ from 1 and 2 by the if rule.

Solution 40.
We provide suitable triples with a short explanation. Each time we use S to refer to the program in question from the solution to the relevant exercise, which also provides us with the output variable(s).

(a) For this program we expect termination for all natural number inputs, so a suitable Hoare triple is

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{z} = 1 \leftrightarrow n \text{ is prime}\ \}.$$

(b) The function ϕ is from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{N}$, and we only make demands for these inputs. Note that our post-condition implies that x is a natural number as required by the target set of our function.

$$\{\ \mathtt{m} = m \in \mathbb{N} \wedge \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = \phi(m, n)\ \}$$

(c) This program should terminate for all integer inputs, so in the precondition we just have to record the inputs.

$$\{\ \mathtt{m} = m \wedge \mathtt{n} = n\ \}\ \mathtt{S}\ \{\ \Downarrow\ n \bmod n = \mathtt{x}\ \}.$$

(d) The function ϕ is from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{N}$, and we only make demands for these inputs. Note that our post-condition implies that x is a natural number as required by the target set of our function.

$$\{\ \mathtt{m} = m \in \mathbb{N} \wedge \mathtt{n} = n\mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = \phi(m, n)\ \}.$$

Again we check what happens if one or both of the inputs are outside of the domain of the function ϕ. If m is negative the loop is not carried out, the program terminates, and the the output variable x is equal to $2n+1$. If m is positive, but n is not, the program still terminates and x is equal to $2^m(2n+1)$. We may capture this behaviour by the following Hoare triple.

$$\{\ \mathtt{m} = m \wedge \mathtt{n} = n\ \}\ \mathtt{S}\ \{\ \Downarrow\ (m < 0 \rightarrow \mathtt{x} = 2n+1 \wedge m \geq 0 \rightarrow \mathtt{x} = 2^m(2n+1))\ \}.$$

(e) The function ϕ' is from $\mathbb{N}$ to $\mathbb{N}\times\mathbb{N}$, so we expect a particular behaviour only for $n \in \mathbb{N}$. We may formulate the postcondition by remembering that ϕ' is the inverse function for ϕ, which means that

$$\phi' n = (x, y) \qquad \text{if and only if} \qquad n = \phi(x, y)$$

for all $n, x, y \in \mathbb{N}$, so a suitable Hoare triple is

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ n = \phi(\mathtt{x}, \mathtt{y})\ \}.$$

If n is negative the first loop is not executed, the value of e is 0. Hence the second loop is also not executed. This means that the value of x is whatever it was before the program started, and the same is true for m until we come to the stage where 1 is subtracted from that variable. This means that the number of times the final loop is carried out is dependent on this value, and so the value of y upon termination depends on the value of m when the program starts. While one could provide more information about the values in the output variables, there does not seem much point to it. We suggest the following Hoare triple

$$\{\ \ \}\ \mathtt{S}\ \{\ \Downarrow\ \},$$

which only asserts termination.

(f) Let the program on the left by S and that on the right be S'. Based on the analysis provided in the solution to Exercise 19 we expect

$$\{\ \mathtt{i} = i \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{a} = \sum_{j}^{i} j\ \},$$

and

$$\{\ \mathtt{i} = i \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{S'}\ \{\ \Downarrow\ \mathtt{y} = \sum_{j}^{i} j\ \}.$$

For the program S, if i is less than 1, the loop is not carried out, the program terminates, and the output variable y is one, so we may use

$$\{\ \mathtt{i} = i \leq 0\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{y} = 1\ \}.$$

For the program S', if i is less than 1, the behaviour is more interesting. It always returns $(\mathtt{i} \cdot \mathtt{i} + 1)/2$, so we get the sum of the first $|\mathtt{i}|$ numbers and we may describe this as

$$\{\ \mathtt{i} = i\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{y} = \sum_{j}^{|i|} j\ \}.$$

Solution 41.
We solve each part.

(a) It is important to check that the program works as expected for values such as $n = 0$ and

$n = 1$. If $n = 0$ then the body of the loop is never executed and $\mathtt{x} = 1$ is the correct value to return. If $n = 1$ the body of the loop is carried out once, and $\mathtt{x} = m$ holds, which is also as required. Hence no adjustments to the program are required.

(b) We expect to be able to show for this program S that

$$\{\ \mathtt{m} = m \in \mathbb{N} \wedge \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = m^n\ \}.$$

(c) The two variables that change each time the body of the loop is executed are x and y, with the latter being reduced by 1 each time and the former being multiplied by m. We may think of x as providing a value for the calculation so far, and y as determining a value of the calculation still to go. This observation may be summarized in the following table tracking the value of the variables, where we use $\mathtt{n} = 3$:

x	y	
initial	1	3
loop	m	3
loop	m^2	2
loop	m^3	1

And indeed, each time the body of the loop has been carried out we have that

$$\mathtt{x} = m^{n-\mathrm{y}}$$

for the following reason: When the loop starts we have $\mathtt{x} = 1$ and $\mathtt{y} = n$, and indeed $m^{n-n} = m^0 = 1 = \mathtt{x}$, and so the condition holds. If we execute the body of the loop from a position where the condition holds then after the update the value of x is m times what it was before the update and y has been reduced by 1, so the condition becomes

$$\mathtt{x} \cdot m = m^{n-\mathrm{y}+1} = m^{n-(\mathrm{y}-1)}.$$

(d) We give a derivation, where we use P for the condition $\mathtt{m} = m \in \mathbb{N} \wedge \mathtt{n} = n \in \mathbb{N}$.

1. $\{\ P\ \}\ \mathtt{x} := 1;\ \mathtt{y} := \mathtt{n}\ \{\ P \wedge \mathtt{x} = 1 \wedge \mathtt{y} = n\ \}$ by the := rule.
2. $\{\ P \wedge \mathtt{x} = m^{n-\mathrm{y}} \wedge \mathtt{y} > 0\ \}\ \mathtt{x} := \mathtt{x} \times \mathrm{m}; \mathtt{y} := \mathtt{y} - 1\ \{\ P \wedge \mathtt{x} = m^{n-\mathrm{y}}\ \}$ by the := rule as reasoned in (c).
3. $\{\ P \wedge \mathtt{x} = m^{n-\mathrm{y}}\ \}$ **while** y>0 **do** $\mathtt{x} := \mathtt{x} \times \mathrm{m};\ \mathtt{y} := \mathtt{y} - 1\ \{\ P \wedge \mathtt{x} = m^n \wedge \mathtt{y} = 0\ \}$ by the while rule, noting that $n - \mathrm{y} = n - 0 = n$ when the loop terminates.
4. $\{\ P\ \}\ \mathtt{S}\ \{\ P \wedge \mathtt{x} = m^n \wedge \mathtt{y} = 0\ \}$ by 1 and 3 using the ; rule and noting that the postcondition of 1 implies the precondition of 4.

(e) Finding a loop variant is fairly easy for our present program since y reduces each time the body of the loop is executed and it stays greater than 0. Hence applying a small modification of the derivation from above is sufficient to get what we want.

(f) We adjust the partial correctness derivation making use of the loop variant.

1. $\{\ P\ \}\ \mathtt{x} := 1;\ \mathtt{y} := \mathtt{n}\ \{\ \Downarrow\ P \wedge \mathtt{x} = 1 \wedge \mathtt{y} = n\ \}$ as 1 from above.
2. $\{\ P \wedge \mathtt{x} = m^{n-\mathrm{y}} \wedge 0 < \mathtt{y} = k \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{x} \times \mathrm{m}; \mathtt{y} := \mathtt{y} - 1\ \{\ \Downarrow\ P \wedge \mathtt{x} = m^{n-\mathrm{y}} \wedge \mathtt{y} < k\ \}$ as 2 from above, with additional conditions for y.

3. $\{ P \wedge \mathtt{x} = m^{n-\mathtt{y}} \}$ **while** y>0 **do** $\mathtt{x} := \mathtt{x} \times \mathtt{m}$; $\mathtt{y} := \mathtt{y} - 1 \{ \Downarrow P \wedge \mathtt{x} = m^n \wedge \mathtt{y} = 0 \}$ by the while rule, noting that $n - \mathtt{y} = n - 0 = n$ when the loop terminates.
4. $\{ P \} \mathtt{S} \{ \Downarrow P \wedge \mathtt{x} = m^n \wedge \mathtt{y} = 0 \}$ by 1 and 3 using the ; rule and noting that the postcondition of 1 implies the precondition of 4.

Solution 42.
We solve each part.

(a) We expect to show for this program that

$$\{ \mathtt{n} = n \in \mathbb{N} \setminus \{0\} \} \ \mathtt{S} \ \{ \Downarrow \mathtt{x} = \sum_{i=1}^{n} i \}.$$

We use P for this precondition.

(b) We look at the loop with the understanding that we will be able to use the precondition

$$\mathtt{x} = 0 \wedge \mathtt{i} = 1 \wedge P.$$

We need a loop invariant. As in Section 4.5 we look at the values of the variables mentioned in the program as the body of the loop is executed repeatedly.

	i	x
start	1	0
loop	2	1
loop	3	3
loop	4	6
loop	5	10
loop	6	15
loop	7	21

This does not look very helpful at first. But recalling how we overcame the same issue when looking at integer division, are we able to use *what we are trying to show* as the loop invariant? The answer is yes: Each time the body of the loop has been executed we have

$$\mathtt{x} = \sum_{j=1}^{\mathtt{i}-1} j,$$

and this is also true before the loop starts since

$$\sum_{j=1}^{0} j = 0.$$

This loop invariant is close to giving us the postcondition we are aiming for, but we need to do something to ensure that when the loop terminates, the sum that has been computed in the variable x goes up to the input n, and for that we also need a condition about i. We may see that we would like to have $\mathtt{i} = n + 1$ when the program terminates, and we note that as the body of the loop is carried out, i always stays below $n + 1$.

(c) We give a partial correctness derivation, where we use P' for the loop invariant

$$\mathtt{x} = \sum_{j=1}^{\mathtt{i}-1} j \wedge \mathtt{i} \leq n+1.$$

1. $\{\ P\ \}\ \mathtt{x} := 0;\ \mathtt{i} := 1\ \{\ P \wedge \mathtt{x} = 0 \wedge \mathtt{i} = 1\ \}$ by the := rule.
2. $\{\ P \wedge P' \wedge \mathtt{i} \leq \mathtt{n}\ \}\ \mathtt{x} := \mathtt{x} + \mathtt{i};\ \mathtt{i} := \mathtt{i} + 1\ \{\ P \wedge Q\ \}$ by the := rule since after the update we have (for values of $\mathtt{i}$ and $\mathtt{x}$ from before the update)

$$\sum_{j=1}^{\mathtt{i}+1-1} j = \left(\sum_{j=1}^{\mathtt{i}-1} j\right) + \mathtt{i} = \mathtt{x} + \mathtt{i}$$

   is the value of $\mathtt{x}$ after the update, while knowing that $\mathtt{i} \leq n$ from the precondition gives us $\mathtt{i} \leq n+1$ in the postcondition. You may find it confusing that we list both

$$\mathtt{i} \leq n+1 \qquad \text{and} \qquad \mathtt{i} \leq n$$

   in the precondition—the former is needed for the loop invariant (see step 4), and the latter is needed to apply the while rule in the next step.
3. $\{\ P \wedge P'\ \}$ **while** $\mathtt{i} \leq \mathtt{n}$ **do** $\mathtt{x} := \mathtt{x} + \mathtt{i};\ \mathtt{i} := \mathtt{i} + 1\ \{\ P \wedge P' \wedge \mathtt{i} > \mathtt{n}\ \}$ from 2 by the while rule.
4. $\{\ P\ \}\ \mathtt{S}\ \{\ P \wedge \mathtt{x} = \sum_{j=1}^{\mathtt{i}-1} j \wedge \mathtt{i} \leq n+1 \wedge \mathtt{i} > \mathtt{n}\ \}$ from 1 and 3 by the ;= rule since as observed in the text above, the postcondition from 1 implies the precondition from 3.
5. $\{\ P\ \}\ \mathtt{S}\ \{\ \mathtt{x} = \sum_{j=1}^{n} j\ \}$ from 4 by the rule of consequence since $\mathtt{i} \leq n+1 \wedge \mathtt{i} > n$ implies $\mathtt{i} = n+1$ and so $\mathtt{i} - 1 = n$ as required.

(d) The control of the loop is relatively straightforward: We have a variable $\mathtt{i}$ that starts at 1 and counts up to $\mathtt{n}$. That means a measure that stays above 0 and is guaranteed to go down each time the body of the loop is executed is $\mathtt{n} - \mathtt{i} + 1$.

(e) We adjust the previous derivation.

1. $\{\ P\ \}\ \mathtt{x} := 0;\ \mathtt{i} := 1\ \{\ \Downarrow P \wedge \mathtt{x} = 0 \wedge \mathtt{i} = 1\ \}$ is 1 from above.
2. $\{\ P \wedge P' \wedge 1 \leq \mathtt{i} \leq \mathtt{n} \wedge \mathtt{n} - \mathtt{i} + 1 = k \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{x} + \mathtt{i};\ \mathtt{i} := \mathtt{i} + 1\ \{\ \Downarrow P \wedge P' \wedge \mathtt{n} - \mathtt{i} + 1 < k\ \}$ as 2 from above noting that the additional condition is justified from the assignment rule.
3. $\{\ P \wedge P'\ \}$ **while** $\mathtt{i} \leq \mathtt{n}$ **do** $\mathtt{x} := \mathtt{x} + \mathtt{i};\ \mathtt{i} := \mathtt{i} + 1\ \{\ \Downarrow P \wedge P' \wedge \mathtt{i} > \mathtt{n}\ \}$ from 2 by the while rule, where we had to adjust the precondition to ensure that the loop variant $\mathtt{n} - \mathtt{i} + 1$ is greater than or equal to 0.
4. $\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = \sum_{j=1}^{n} j\ \}$ by the same reasoning as steps 4 and 5 from the previous derivation.

Solution 43.
We solve each part.

(a) We expect the integer logarithm to be the natural number m with the property that for the input n we have $2^m \leq n$ and $n < 2^{m+1}$.

(b) We expect to be able to establish the following Hoare triple to describe the behaviour of this program S:

$$\{\ \mathtt{n} = n \in \mathbb{N} \setminus \{0\}\ \}\ \mathtt{S}\ \{\ \Downarrow\ 2^{\mathtt{z}} \leq n < 2^{\mathtt{z}+1}\ \}.$$

(c) We look for a loop invariant. This time looking at the way the values held in the variables change does help:

	z	x
start	0	1
loop	1	2
loop	2	4
loop	3	8
loop	4	16

We may see that we have $2^{\mathtt{z}} = \mathtt{x}$.

When we reach the loop, we have $\mathtt{x} = 1$ and $\mathtt{z} = 0$, and so $2^{\mathtt{z}} = 2^0 = 1 = \mathtt{x}$ certainly holds. Now assume we have $2^{\mathtt{z}} = x$ and that we carry out the body of the loop again. Updating z increases it by one and updating x doubles it, and multiplying our assumed equality by 2 on both sides gives the required $2^{\mathtt{z}+1} = x \cdot 2$.

(d) We use P for the condition $\mathtt{n} = n \in \mathbb{N} \setminus \{0\}$

1. $\{\ P\ \}\ \mathtt{x} := 1;\ \mathtt{z} := 0\ \{\ P \wedge \mathtt{x} = 1 \wedge \mathtt{z} = 0\ \}$ by the := rule.
2. $\{P \wedge 2^{\mathtt{z}} = x \wedge x \cdot 2 \leq n\}\mathtt{z} := \mathtt{z} + 1;\ \mathtt{x} := \mathtt{x} \times 2\{P \wedge 2^{\mathtt{z}} = x\}$ by the := rule via the justification given in the text above.
3. $\{\ P \wedge 2^{\mathtt{z}} = x\ \}$ **while** $\mathtt{x} \times 2 \leq \mathtt{n}$ **do** $\mathtt{z} := \mathtt{z} + 1;\ \mathtt{x} := \mathtt{x} \times 2\ \{\ P \wedge 2^{\mathtt{z}} = x \wedge x \cdot 2 > n\ \}$ from 2 by the while rule.
4. $\{\ P\ \}\ \mathtt{S}\ \{\ P \wedge 2^{\mathtt{z}} = x \wedge x \cdot 2 > n\ \}$ from 1 and 3 observing that the postcondition from 1 implies the precondition from 3 since $2^0 = 1$.

It is not immediately obvious that the final postcondition implies the condition we are interested in. We have one of the inequalities we are looking for, namely $2^{\mathtt{z}} \leq n$. But we also know that $2^{\mathtt{z}} = x$, and that $x \cdot 2 > n$, and putting those two together gives $2^{\mathtt{z}+1} = 2^{\mathtt{z}} \cdot 2 = x \cdot 2 > n$, so our final postcondition does indeed imply the condition we were aiming for.

(e) Variables that are used in the body of the loop increase, but we may see that we can use the boolean expression governing the loop to obtain $n - \mathtt{x} \cdot 2$ is a natural number that decreases each time the body of the loop is carried out.

(f) We adjust the derivation from above.

1. $\{\ P\ \}\ \mathtt{x} := 1;\ \mathtt{z} := 0\ \{\ \Downarrow P \wedge \mathtt{x} = 1 \wedge \mathtt{z} = 0\ \}$ as 1 from above.
2. $\{P \wedge 2^{\mathtt{z}} = x \wedge 0 \leq n - 2\mathtt{x} = k \wedge x \cdot 2 \leq n\}\mathtt{z} := \mathtt{z} + 1;\ \mathtt{x} := \mathtt{x} \times 2\{\Downarrow P \wedge 2^{\mathtt{z}} = x \wedge n - 2\mathtt{x} < k\}$ as 2 from above, noting that the additional condition works for the := rule.
3. $\{\ P \wedge 2^{\mathtt{z}} = x\ \}$ **while** $\mathtt{x} \times 2 \leq \mathtt{n}$ **do** $\mathtt{z} := \mathtt{z} + 1;\ \mathtt{x} := \mathtt{x} \times 2\ \{\ \Downarrow P \wedge 2^{\mathtt{z}} = x \wedge x \cdot 2 > n\ \}$ from 2 by the while rule.
4. $\{\ P\ \}\ \mathtt{S}\ \{\ \Downarrow P \wedge 2^{\mathtt{z}} = x \wedge x \cdot 2 > n\ \}$ from 1 and 3 observing that the postcondition from 1 implies the precondition from 3 since $2^0 = 1$.

Solution 44.

We solve each part.

(a) Here is a suitable program. It assume the input is provided via the variable n, and the

output in the variable x, with the value 1 encoding true, and 0, false.

```
y := n;
while 10≤y do
    y := y−10;
while 3≤y do
    y := y−3;
if y=0 then
    x := 1;
else
    x := 0
```

(b) We expect to be able to establish the following Hoare triple to describe the behaviour of this program S:

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ k = n \bmod 10 \to (k \bmod 3 = 0 \leftrightarrow \mathtt{x} = 1)\ \}.$$

(c) We require two loop invariants, but they are both very similar. For the first loop, we note that $\mathtt{y} \bmod 10$ is constant, and equal to $n \bmod 10$, so we may use $\mathtt{y} \bmod 10 = n \bmod 10$. For the second loop we may use $\mathtt{y} \bmod 3 = (n \bmod 10) \bmod 3$. We use P for the former, and P' for the latter predicate.

(d) In what follows we use I for the if statement. We further introduce the condition Q

$$((\mathtt{y} = 0 \wedge \mathtt{x} = 1) \vee \wedge(\neg(\mathtt{y} = 0) \wedge \mathtt{x} = 0))\,.$$

1. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{y} := \mathtt{n}\ \{\ \mathtt{n} = n \in \mathbb{N} \wedge \mathtt{y} = n\ \}$ by the := rule.
2. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge 10 < \mathtt{y}\ \}\ \mathtt{y} := \mathtt{y} - 10\ \{\ \mathtt{n} = n \in \mathbb{N} \wedge P\ \}$ by the := rule.
3. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P\ \}$ **while** $10 \leq \mathtt{y}$ **do** $\mathtt{y} := \mathtt{y} - 10\ \{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge 10 > \mathtt{y}\ \}$ from 2 by the while rule.
4. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge 3 \leq \mathtt{y}\ \}\ \mathtt{y} := \mathtt{y} - 3\ \{\ \mathtt{n} \in \mathbb{N} \wedge P \wedge P'\ \}$ by the := rule.
5. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P'\ \}$ **while** $3 \leq \mathtt{y}$ **do** $\mathtt{y} := \mathtt{y} - 3\ \{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge 3 > \mathtt{y}\ \}$ from 4 by the while rule.
6. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge \mathtt{y} = 0\ \}\ \mathtt{x} := 1\ \{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge Q\ \}$ by the := rule.
7. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge \neg(\mathtt{y} = 0)\ \}\ \mathtt{x} := 0\ \{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge Q\ \}$ by the := rule.
8. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P'\ \}\ \mathtt{I}\ \{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge Q\ \}$ from 6 and 7 by the if rule.
9. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge Q\ \}$ from 1, 3, 5 and 8 using the ; rule repeatedly. We make an argument that the relevant postconditions imply the relevant preconditions:
   - If $\mathtt{y} = n$ then $\mathtt{y} \bmod 10 = n \bmod 10$ and so P holds, and so the postcondition from 1 implies the precondition from 3.
   - If P holds we know that $\mathtt{y} \bmod 10 = n \bmod 10$, which with $\mathtt{y} < 10$ implies that $\mathtt{y} = n \bmod 10$, and that implies P', and so the postcondition from 3 implies the precondition from 5.
   - The postcondition from 5 clearly implies the precondition from 8.

It remains to argue that the final postcondition implies the condition we are aiming for. We know that at the end, $\mathtt{y} = (n \bmod 10) \bmod 3$, and for the final postcondition to hold this must be 0 if and only if $\mathtt{x} = 1$ as required.

(e) For the first loop we may use the loop variant y, and the same again for the second.

(f) We adjust the derivation from above.

1. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{y} \coloneqq \mathtt{n}\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge \mathtt{y} = n\ \}$ by the := rule.
2. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge 10 < \mathtt{y} \wedge 0 \leq \mathtt{y} = k\ \}\ \mathtt{y} \coloneqq \mathtt{y} - 10\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge 0 \leq \mathtt{y} < k\ \}$ by the := rule.
3. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P\ \}$ **while** $10 \leq \mathtt{y}$ **do** $\mathtt{y} \coloneqq \mathtt{y} - 10\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge 10 > \mathtt{y}\ \}$ from 2 by the while rule.
4. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge 3 \leq \mathtt{y} \wedge 0 \leq \mathtt{y} = k\ \}\ \mathtt{y} \coloneqq \mathtt{y} - 3\ \{\ \Downarrow\ \mathtt{n} \in \mathbb{N} \wedge P \wedge P' \wedge 0 \leq \mathtt{y} < k\ \}$ by the := rule.
5. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P'\ \}$ **while** $3 \leq \mathtt{y}$ **do** $\mathtt{y} \coloneqq \mathtt{y} - 3\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge 3 > \mathtt{y}\ \}$ from 4 by the while rule.
6. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge \mathtt{y} = 0\ \}\ \mathtt{x} \coloneqq 1\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge Q\ \}$ by the := rule.
7. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge \neg(\mathtt{y} = 0)\ \}\ \mathtt{x} \coloneqq 0\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge Q\ \}$ by the := rule.
8. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P'\ \}\ \mathtt{I}\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge Q\ \}$ from 6 and 7 by the if rule.
9. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge P \wedge P' \wedge Q\ \}$ from 1, 3, 5 and 8 using the ; rule repeatedly.

Solution 45.
We solve each part. As in the solution to Exercise 22 we use the subscript r to mark the new system.

(a) A suitable rule is the following:

$$\frac{\{\ P\ \}_r\ \mathtt{S}\ \{\ P'\ \}_r \quad \{\ P' \wedge \neg\mathcal{B}\,\mathtt{b}\ \}_r\ \mathtt{S}\ \{\ P'\ \}_r}{\{\ P\ \}_r\ \textbf{repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}\ \{\ P' \wedge \mathcal{B}\,\mathtt{b}\ \}_r}$$

(b) Assume we know that

$$\{\ P\ \}_r\ \textbf{repeat}\ \mathtt{S}\ \textbf{until}\ \mathtt{b}\ \{\ Q\ \}_r.$$

For this to be derivable Q must be of the form $P' \wedge \mathcal{B}\,\mathtt{b}$ and we must have

$$\{\ P\ \}_r\ \mathtt{S}\ \{\ P'\ \}_r \qquad \text{and} \qquad \{\ P' \wedge \neg\mathcal{B}\,\mathtt{b}\ \}_r\ \mathtt{S}\ \{\ Q'\ \}_r.$$

We want to establish

$$\{\ P\ \}\ \mathtt{S};\ \textbf{while}\ \neg\mathtt{b}\ \textbf{do}\ \mathtt{S}\ \{\ Q\ \}.$$

This is a proof by induction over use of rule for repeat until loops in the derivation. We have

$$\{\ P\ \}\ \mathtt{S}\ \{\ P'\ \}$$

since the derivation of $\{\ P\ \}_r\ \mathtt{S}\ \{\ P'\ \}_r$ has fewer instances of that rule, and we similarly have

$$\{\ P' \wedge \neg\mathcal{B}\,\mathtt{b}\ \}\ \mathtt{S}\ \{\ P'\ \},$$

and observing that $\mathcal{B}(\neg \mathtt{b}) = \neg\, \mathcal{B}\, \mathtt{b}$ by the while rule this gives us

$$\{\ P'\ \}\ \textbf{while}\ \neg \mathtt{b}\ \textbf{do}\ \mathtt{S}\ \{\ P' \wedge \mathcal{B}\, \mathtt{b}\ \}.$$

We observe that Q is $P' \wedge \mathcal{B}\, \mathtt{b}$, and we may now use the ; rule to obtain

$$\{\ P\ \}\ \mathtt{S};\ \textbf{while}\ \neg \mathtt{b}\ \textbf{do}\ \mathtt{S}\ \{\ Q\ \}.$$

Chapter 5

Computability

In this chapter we address the question of what can be computed with While programs, and we extend this idea beyond that specific language to ask the question whether there are limitations to what we may hope to compute more generally.

What we don't address are questions that may be found in the science fiction genre, such as whether computers are able to feel emotions, or whether they can become an intelligent entity in their own right.

What we are looking at here are questions that may strike the reader as quite narrow on first reading: We are asking which functions

$$f : \mathbb{N} \longrightarrow \mathbb{N}$$

may be computed.

Of course actual computers have limitations (such as finite memory) and so are not able to compute all outputs for all inputs, and indeed, most implementations will not be able to compute an output for a sufficiently large input. But one might also imagine that if we are interested in the output of the function f for a particular input n it should be possible to keep extending our system until we can cope with inputs of that size. One of the advantages of a theoretical programming language such as While is that we can go beyond these kinds of limitations.

The history of this field goes back to the nineteen thirties. Earlier in that decade the Austrian logician Kurt Gödel had published his *Incompleteness Theorem*. This says that in a formal system sufficiently powerful to encode arithmetic of the natural numbers there exist propositions that can be neither proved nor disproved within the system. At the time this was a seismic shock: At the beginning of the 20th century the mathematician David Hilbert had formulated a number of problems, one of which did indeed presume that it was possible to give a set of axioms so that all such propositions may be either proved or disproved (his second problem), and in his tenth problem he asked for a method that would allow us to find solutions to Diophantine equations, or to show that for a particular system of equations, no solution exists. The idea that this might not be possible was something not many people had contemplated. Indeed, one of the most famous Hilbert quote, inscribed on his tomb stone, is 'Wir müssen wissen. Wir werden wissen.' This translates as 'We must know. We will know.' which expresses an optimism that is representative of mathematician's attitude towards what they would be able to do eventually. And indeed, when he introduced his problems to the International Congress of Mathematicians in 1900, he said 'However unapproachable these problems may seem to us and however helpless we stand before them, we have, nevertheless, the firm conviction that their solution must follow by a finite number of purely logical processes.'

The question of whether a function $\mathbb{N}$ to $\mathbb{N}$ is computable was asked before computers existed. Indeed, as touched upon in Chapter 1, theoretical computational devices were created before the first programmable computers. We look at the question of what we mean when we talk about computable functions in Section 5.1, and we see that not all functions are computable. We then ask the question what we may say about problems that only have yes/no answers in Section 5.3.

As we briefly touch on in the introduction, Alan Turing used his abstract model of computation (in the form of Turing machines) to study whether there are limits to what we may compute in a finite number of steps. There are some similarities Turing's argument that there are problems whose answer is not computable and that employed by Gödel in the proof of the *Incompleteness Theorem*. Both rely on the idea that their system is powerful enough to encode enough information about itself that we may ask it questions about itself, and it is this *self-reference* that we may employ to arrive at a contradiction in both cases. We see a variation of that argument in Section 5.4.

Having provided a problem whose answer cannot be computed, we ask whether it is possible to at least use a computation to find the answer if that answer is 'yes'. That requires a fair amount of machinery that is introduced in Section 5.5, which makes use of the material from Sections 6.3 and 6.4.

A natural question is whether it matters which notion of computation we work with, Turing machines, While programs, or actual physical computing devices. In Section 5.6 we address the question of whether it matters what computational setting we study.

5.1 Computable Functions

We begin our study of what can by computed by looking at functions from the set of natural numbers to itself. This provides us with a rigorous definition of the problems we may want to compute, and gives us a first idea of whether there are limitations to what we can compute.

5.1.1 Computable functions

We begin by defining a notion of computable function based on our abstract model of computation provided by the While language.

Definition 12. A function $f : \mathbb{N} \longrightarrow \mathbb{N}$ is **computable** if and only if there is a While program $\mathtt{S}$ with the property that

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = fn\ \}.$$

In this definition we think of $\mathtt{S}$ as a program that computes the function f: The variable $\mathtt{n}$ holds the input in the form of some natural number, the program terminates, and the value of f at that input is provided in the variable $\mathtt{x}$. The correctness assertion means that for all states σ it is the case that there is a (necessarily unique) state σ' with the property that if $\sigma\mathtt{n} \in \mathbb{N}$ then

$$\langle \mathtt{S}, \sigma \rangle \to \Downarrow \sigma' \qquad \text{and} \qquad \sigma'\mathtt{x} = f(\sigma\mathtt{n}).$$

Note in particular that we demand that the program $\mathtt{S}$ terminates as long as $\sigma\mathtt{n}$ is a natural number, and so a suitable input for our function f.

The reader may think that defining computability based only on While programs is limiting, but as we discuss in Section 5.6 this definition is more general than it may appear.

We provide some examples of computable functions.

Example 12. The identity function $\mathsf{id} : \mathbb{N} \longrightarrow \mathbb{N}$ given by $fn = n$ for all $n \in \mathbb{N}$ is computable.

The program

$$\mathtt{x} := \mathtt{n}$$

satisfies the given condition: We have

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{n}\ \{\ \Downarrow\ \mathtt{x} = n = \mathsf{id}_{\mathbb{N}} n\ \}$$

by the := rule.

Example 13. The function

$$\begin{aligned} f : \mathbb{N} &\longrightarrow \mathbb{N} \\ x &\longmapsto x + m \end{aligned}$$

that adds a fixed number m to its input is computable. The following device

```
x := n+m
```

satisfies the condition from the definition since

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{n} + m\ \{\ \Downarrow\ \mathtt{x} = n + m = f n\ \}$$

by the := rule.

Example 14. The function that

$$\begin{aligned} g : \mathbb{N} &\longrightarrow \mathbb{N} \\ x &\longmapsto x \bmod 7 \end{aligned}$$

that returns the remainder when dividing by 7 is computable since the program

```
x := n ;
while y≤x do
    x := x−7
```

satisfies the condition of the definition based on the following derivation, where we use L for the loop.

1. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{n}\ \{\ \Downarrow\ \mathtt{x} = \mathtt{n} = n \in \mathbb{N}\ \}$ by the := rule.

2. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{n}\ \{\ \Downarrow\ \mathtt{x} = \mathtt{n} = n \in \mathbb{N} \wedge \mathtt{x} \bmod 7 = n \bmod 7 \wedge 0 \leq \mathtt{x}\ \}$ from 1 since the new postcondition is equivalent to the one given there. We use P for the condition $\mathtt{x} = \mathtt{n} = n \in \mathbb{N} \wedge \mathtt{x} \bmod 7 = n \bmod 7$.

3. $\{\ R \wedge 0 \leq \mathtt{x} = k \wedge 7 \leq \mathtt{x}\ \}\ \mathtt{x} := \mathtt{x} - 7\ \{\ \Downarrow\ R \wedge 0 \leq \mathtt{x} < k\ \}$ by the := rule.

4. $\{\ R\ \}\ \mathtt{L}\ \{\ \Downarrow\ R \wedge 0 \leq \mathtt{x} < 7\ \}$ by the while rule.

5. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{n};\ \mathtt{L}\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge \mathtt{x} \bmod 7 = n \bmod 7 \wedge 0 \leq \mathtt{x} < 7\ \}$ from 2 and 4 by the ; rule.

6. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{n};\ \mathtt{L}\ \{\ \Downarrow\ \mathtt{x} = n \bmod 7\ \}$ from 5 by the rule of consequence since the postcondition from 5 implies the postcondition given here.

Exercise 46.
Show that the following functions are computable. It is sufficient to give a program that satisfies the condition from the definition unless the task specifically asks you to do more.

(a) $\mathbb{N} \mapsto \mathbb{N} : n \longrightarrow n^2$.

(b) The function $f : \mathbb{N} \longrightarrow \mathbb{N}$ given by

$$f n = \begin{cases} 1 & n \text{ is prime} \\ 0 & \text{else.} \end{cases}$$

(c) The composite $g \circ f$ of two computable function $f : \mathbb{N} \longrightarrow \mathbb{N}$ and $g : \mathbb{N} \longrightarrow \mathbb{N}$. For this part write down a Hoare triple that states the computability of this function and give a formal derivation of that triple.

(d) The function

$$gn = \begin{cases} 1 & fn \text{ is even} \\ 0 & \text{else} \end{cases}$$

where $f : \mathbb{N} \longrightarrow \mathbb{N}$ is computable. For this part write down a Hoare triple that states the computability of this function and give a formal derivation of that triple. You may assume you have a program S that returns n mod 2 in the variable x.

(e) Adjust the notion of computable function and argue that β and β' from Section 6.2.1 satisfy the condition.

(f) Adjust the notion of computable function and give an informal argument that the functions ϕ and its inverse from Section 6.2.2 are computable.

5.1.2 Computability for partial functions

Readers may want to skip this and the following section on first reading, and return to the concepts introduced here when they reach Section 5.3.3. The ideas discussed here are also required for Section 5.5.

In that section we have to consider *partial functions* as those being computed by a While program since the program may fail to terminate for some inputs and so might not produce an output for some of its inputs.

Definition 13. A **partial function** $f : \mathbb{N} \rightharpoonup \mathbb{N}$ **is computable** if there is a program S with the property that

$$\{\ \mathtt{n} = n \in \mathbb{N} \wedge fn \text{ defined}\ \}\ \mathtt{S}\ \{\ \Downarrow \mathtt{x} = fn\ \}.$$

Note that if f happens to be a total function then we know that fn is defined for all potential inputs $n \in \mathbb{N}$, and then the condition given in the definition amount to

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow \mathtt{x} = fn\ \},$$

so for total functions this definition of computable agrees with that given in Definition 12. We may think of this definition as *extending* the one given earlier.

Further note that the given Hoare triple says nothing about what happens for inputs for which the function is not defined: the program may fail to terminate, or it may produce an output in which case we do not make any demands on that output.

Example 15. The partial function

$$\begin{aligned} f : \mathbb{N} &\rightharpoonup \mathbb{N} \\ x &\longmapsto \begin{cases} x^2 & x \text{ even} \\ \text{undefined} & \text{else} \end{cases} \end{aligned}$$

is computable since the following program satisfies the condition from Definition 13.

```
x:=n;
while 2≤n do
    n:=n−2;
if n=0 then
```

```
        x := x×x ;
    else
        skip
```

Call this program S. It is possible to provide a total correctness derivation for the Hoare triple

$$\{\ \mathtt{n} = n \in \mathbb{N} \wedge n \bmod 2 = 0\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = n^2\ \}.$$

Note that the program returns the input for values for which the partial function is not defined, but the Hoare triple does not make any claims about this situation.

Exercise 47.
Show that the following partial functions are computable. It is sufficient to give a program that satisfies the condition from the definition. What happens for inputs where the given partial function is not defined?

(a)

$$\begin{aligned} \mathbb{Z} &\longrightarrow \mathbb{Z} \\ x &\longmapsto \begin{cases} x/2 & x/2 \in \mathbb{Z} \\ \text{undefined} & \text{else.} \end{cases} \end{aligned}$$

(b)

$$\begin{aligned} \mathbb{N} &\longrightarrow \mathbb{N} \\ x &\longmapsto \begin{cases} \sqrt{x} & \sqrt{x} \in \mathbb{N} \\ \text{undefined} & \text{else.} \end{cases} \end{aligned}$$

(c) Assume that a, b and c are integers.

$$\begin{aligned} \mathbb{N} &\longrightarrow \mathbb{N} \\ x &\longmapsto \begin{cases} 1 & ax^2 + bx + c = 0 \\ \text{undefined} & \text{else.} \end{cases} \end{aligned}$$

(d) Assume that k is an integers. Adjust the notion of computability from Definition 13 to apply to the following partial function.

$$\begin{aligned} \mathbb{N} \times \mathbb{N} &\longrightarrow \mathbb{N} \\ (m, n) &\longmapsto \begin{cases} 1 & \exists x,\ y \in \mathbb{N}.\ mx + ny = k \\ \text{undefined} & \text{else.} \end{cases} \end{aligned}$$

5.1.3 Generalizing the notion of computability

In Exercise 46 we encourage students to think about extending our notion of computability to functions of integers, or pairs of natural numbers. But that then requires us to prove results about these functions separately, whereas if we instead make use of our encodings, we may provide a formal definition that ensures we only have to establish results about (partial) functions from $\mathbb{N}$ to $\mathbb{N}$.

Definition 14. Let S and T be two sets with injections

$$\gamma : \mathbb{N} \longrightarrow S \qquad \text{and} \qquad \delta : T \longrightarrow \mathbb{N}.$$

We say that a partial function $f : S \rightharpoonup T$ is **computable** if and only if this is the case for the composite

$$\mathbb{N} \xrightarrow{\gamma} S \xrightarrow{f} T \xrightarrow{\delta} \mathbb{N},$$

that is the partial function $\delta \circ f \circ \gamma$.

Assume we are given S and T and the injections required by the preceding definition. A partial function $f : S \longrightarrow T$ is computable if there is a program $\mathtt{S}$ with the property that

$$\{ \ \mathtt{n} = n \in \mathbb{N} \wedge f(\gamma n) \text{ defined } \} \ \mathtt{S} \ \{ \Downarrow \mathtt{x} = \delta(f(\gamma n)) \ \}.$$

Note that if S and T are both equal to $\mathbb{N}$ then we may use the identity function on $\mathbb{N}$ for both, γ and δ, and so this definition extends Definition 12.

Note that if f happens to be a total function, then so is the composite $\delta \circ f \circ \gamma$, and it that definition we know that $f\,s$ is defined for all inputs $s \in S$. In that situation we know that the notion of computable function agrees with that given in Definition 12, so this general definition works equally well for partial and total functions.

This allows us to restrict our studies to computable functions from $\mathbb{N}$ to $\mathbb{N}$ while still covering functions $S \longrightarrow T$ as long as it is possible to encode S and T as natural numbers using bijections.

For example where $S = \mathbb{N}$ and $T = \mathbb{B}$ see Sections 5.3 and 5.3.3.

See Section 6.2.4 for an example of how this idea applies to addition for integers.

Exercise 48.
In Exercise 46 we ask students to adjust the definition of computable function to allow for functions whose source and/or target set is not $\mathbb{N}$. Definition Definition 14 does that in a general way.

For the following functions consider your answer to Exercise 46 and discuss whether the new definition makes a difference to whether they are computable.

(a) The functions β and β' from Section 6.2.1.

(b) The function ϕ and its inverse from Section 6.2.2.

Exercise 49.
In Exercise 46 we ask students to adjust the definition of computable function to allow for functions whose source and/or target set is not $\mathbb{N}$. Definition Definition 14 does that in a general way.

For the following functions, explain how one might adjust the definition of computable function directly to cover them. Then look at what is required to establish that these functions are computable when using Definition 14.

(a) The function

$$\begin{aligned} \mathbb{N} \times \mathbb{N} &\longrightarrow \mathbb{N} \times \mathbb{N} \\ (x, y) &\longmapsto (y, x). \end{aligned}$$

(b) The function

$$\begin{aligned} \mathbb{N} \times \mathbb{N} &\longrightarrow \mathbb{Z} \\ (x, y) &\longmapsto x - y. \end{aligned}$$

5.2 There are uncomputable functions

Having defined the notion of a computable function the obvious question is whether every function $\mathbb{N} \longrightarrow \mathbb{N}$ is computable.

The shape of the argument that proves that the answer to this question is no may come as a surprise: We do not give a particular function and show that it is uncomputable—instead we rely on a mismatch between the size of the set of functions we can hope to compute, and the set of functions under consideration.

We know from Section 6.3 that the set of While programs is countable, which means that there are at most countably many computable functions from $\mathbb{N}$ to $\mathbb{N}$. Since $\mathsf{Fun}(\mathbb{N}, \mathbb{N})$ is uncountable by Theorem 23 there must be functions in that set that are not computable. There are infinitely many computable functions since for every $m \in \mathbb{N}$ the function $x \longmapsto m \cdot x$ is computable.

This mismatch of sizes provides us with the following result.

Proposition 15. *There are uncomputable functions of type* $\mathbb{N} \to \mathbb{N}$.

Note that we argue in Section 6.2.4 that considering computability for functions on the natural numbers does not significantly limit the scope of what we do—by making use of encodings of other data structures we are able to extend results for these functions to computing with data structures, and this is supported by our generalized definition of computable function given in Definition 14.

5.3 Decidability

In computer science a collection of questions that often are given particular attention are those with a yes/no answer. For example, the problem solved by Alan Turing and Alonzo Church is the *Entscheidungsproblem* which asked whether it is possible to produce an algorithm that gives a yes/no answer to a certain kind of problem, with both authors producing a different setting in which the answer is no.

5.3.1 Decidable properties

One might wonder whether the issue with functions $\mathbb{N} \longrightarrow \mathbb{N}$ is that there are both, infinitely many inputs, and infinitely many outputs, and a problem with a yes/no answer might be easier to cope with.

There are different ways in which we may think of a problem with a yes/no answer for natural number inputs.

We may think of the problem as providing a predicate P, where a given natural number either does, or does not, have the property defined by P. A commonly used notation is $P(x)$ if x satisfies the condition, and $\neg P(x)$ if it does not.

If the input is a natural number we may think of the problem as defining a function

$$f : \mathbb{N} \longrightarrow \mathbb{B}$$

which returns $\mathbf{tt}$ if the answer is yes and $\mathbf{ff}$ otherwise.

Alternatively we may think of it as a subset of $\mathbb{N}$ given by

$$\{n \in \mathbb{N} \mid \text{the answer for } n \text{ is yes}\}.$$

We use these ideas to apply our definition of computability to problems of this kind. We may encode boolean values as the natural numbers 0 and 1, which allows us to use While programs to try to compute answers to such problems.

We may apply Definition 14, where for γ we use the identity function on $\mathbb{N}$, and for δ we use the function

$$\begin{aligned} \delta : \mathbb{B} &\longrightarrow \mathbb{N} \\ x &\longmapsto \begin{cases} 1 & x = \mathbf{tt} \\ 0 & \text{else.} \end{cases} \end{aligned}$$

This means that a function $f : \mathbb{N} \longrightarrow \mathbb{B}$ is **computable** if and only if this is the case for the function $g : \mathbb{N} \longrightarrow \mathbb{N}$ given by

$$gx = \begin{cases} 1 & fx = \mathbf{tt} \\ 0 & \text{else.} \end{cases}$$

We may now define a corresponding notion for predicates.

Definition 16. A predicate P for the natural numbers is **decidable** if the function $\chi_P : \mathbb{N} \longrightarrow \mathbb{B}$ given by

$$\chi_P x = \begin{cases} \mathbf{tt} & P(x) \\ \mathbf{ff} & \text{else} \end{cases}$$

is computable. A predicate that is not decidable is **undecidable**.

The function χ_P from this definition is known as the **characteristic function of** P. A program that shows that χ_P is computable is known as a **decision procedure for** P—it provides an algorithm that allows us to decide whether P holds for a given input.

We investigate what this definition tells us about decidable predicates. For a predicate P to be decidable, its characteristic function χ_P must be computable. This means that we must have a program S with the property that

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ (P(n) \to \mathtt{x} = 1) \wedge (\neg P(n) \to \mathtt{x} = 0)\ \}.$$

Example 16. Here is a decision procedure for the following predicate P: Given $n \in \mathbb{N}$, is there a natural number i such that n is the sum of the first i natural numbers?

The characteristic function for this predicate is

$$\begin{aligned} \chi_P : \mathbb{N} &\longrightarrow \mathbb{B} \\ n &\longmapsto \begin{cases} \mathbf{tt} & \exists i \in \mathbb{N}.\ n = \sum_{j=1}^{i} j \\ \mathbf{ff} & \text{else.} \end{cases} \end{aligned}$$

The following program witnesses the computability of this function.

```
z := 1;  y := 1;
while z<n do
    y := y+1;  z := z+y;
if z=n then
    x := 1
else
    x := 0
```

This program terminates for all inputs provided via the variable n. It produces the output 1 in the variable x if the input is the sum of the first i natural numbers for some i, and 0 otherwise—in other words, the output is 1 if of P holds for the input, and it is 0 otherwise.

5.3.2 Combining predicates

Predicates, their characteristic functions and subsets of $\mathbb{N}$ have properties that connect them with logic: Given two predicates P and Q, we may ask, for example, if both P and Q hold for a particular natural number. Since characteristic functions take their values in $\mathbb{B}$ we are able to apply functions such as conjunction, disjunction, negation and implication to them. In what follows we treat these as functions with the following source and target:

$$\wedge : \mathbb{B} \times \mathbb{B} \longrightarrow \mathbb{B} \qquad \vee : \mathbb{B} \times \mathbb{B} \longrightarrow \mathbb{B}$$
$$\neg : \mathbb{B} \longrightarrow \mathbb{B} \qquad \rightarrow : \mathbb{B} \times \mathbb{B} \longrightarrow \mathbb{B}.$$

Assume that P and Q are predicates on the natural numbers with characteristic function χ_P respective χ_Q, and corresponding subsets A and B of $\mathbb{N}$.

- The characteristic function $\chi_{\neg P}$ for $\neg P$ is given by the assignment $x \longmapsto \neg\chi_P x$, and the corresponding subset of $\mathbb{N}$ is $\mathbb{N} \setminus A$. The characteristic function is the following composite:

$$\mathbb{N} \xrightarrow{\chi_P} \mathbb{B} \xrightarrow{\neg} \mathbb{B}.$$

- The characteristic function $\chi_{P\wedge Q}$ for $P \wedge Q$ is given by $x \longmapsto \chi_P x \wedge \chi_Q x$ and the corresponding subset of $\mathbb{N}$ is $A \cap B$. We may think of the function as the composite

$$\mathbb{N} \times \mathbb{N} \xrightarrow{(\chi_P \times \chi_Q)} \mathbb{B} \times \mathbb{B} \xrightarrow{\wedge} \mathbb{B}.$$

- The characteristic function $\chi_{P\vee Q}$ for $P \vee Q$ is given by $x \longmapsto (\chi_P x \vee \chi_Q x)$ and the corresponding subset of $\mathbb{N}$ is $A \cup B$. We may think of the function as the composite

$$\mathbb{N} \times \mathbb{N} \xrightarrow{\chi_P \times \chi_Q} \mathbb{B} \times \mathbb{B} \xrightarrow{\vee} \mathbb{B}.$$

- The characteristic function of $\chi_{P\to Q}$ for $P \to Q$ is given by $x \longmapsto (\chi_P x \to \chi_Q x)$ and the corresponding subset of $\mathbb{N}$ is $A \cup (\mathbb{N} \setminus B)$. Again we may write this function as a composite, namely

$$\mathbb{N} \times \mathbb{N} \xrightarrow{\chi_P \times \chi_Q} \mathbb{B} \times \mathbb{B} \xrightarrow{\rightarrow} \mathbb{B}.$$

We summarize this information in the following table.

predicate	subset	assignment $x \mapsto$
$\neg P$	$\mathbb{N} \setminus A$	$\neg\chi_P x$
$P \wedge Q$	$A \cap B$	$\chi_P x \wedge \chi_Q x$
$P \vee Q$	$A \cup B$	$\chi_P x \vee \chi_Q x$
$P \to Q$	$A \cup (\mathbb{N} \setminus B)$	$\chi_P x \to \chi_P x$

Students who have taken COMP11120 *Mathematical Techniques for Computer Science* should see a connection with the powerset semantics for logic, as well as the example on the Practice Sheet for Week 12 of Semester 1.

Example 17. We may combine the predicates P of evenness, and Q of being divisible by 3, for natural numbers to form the predicate $P \wedge Q$.

For the characteristic function for $P \wedge Q$ we find that

$$\chi_{P\wedge Q} : \mathbb{N} \longrightarrow \mathbb{B}$$
$$x \longmapsto \begin{cases} \mathbf{tt} & x \text{ divisible by 2 and 3} \\ \mathbf{ff} & \text{else.} \end{cases}$$

It is easy to show that we have an alternative characterization of this predicate since

$$(P \wedge Q)(x) \qquad \text{if and only if} \qquad x \text{ is divisible by } 6.$$

Exercise 50.
Argue that the following predicates are decidable.

(a) Evenness for natural numbers.

(b) Primeness for natural numbers.

(c) The final digit of $n \in \mathbb{N}$ is divisible by 3.

Proposition 17. *Assume that P and Q are decidable predicates. Then so are the following:*

(i) $\neg P$,

(ii) $P \wedge Q$,

(iii) $P \vee Q$,

(iv) $P \to Q$.

Proof: We give an argument for the first statement. Assume we have a program S witnessing the decidability of P. We consider the program T:

```
S;
if x=0 then
    x := 1
else
    x := 0
```

If we execute that program in a state σ that ensures that $\mathtt{n} = n \in \mathbb{N}$ we have τ with $\langle \mathtt{T}, \sigma \rangle \to \Downarrow \tau$ with

$$\tau \mathtt{x} = \begin{cases} 0 & \chi_P(\sigma \mathtt{n}) = 1 \\ 1 & \text{else.} \end{cases}$$

This is the characteristic function of $\neg P$, and so this predicate is indeed decidable. ∎

Exercise 51.
Show the remaining statements of Proposition 17. It is sufficient to provide an argument similar to the one given for $\neg P$ in the text.

Exercise 52.
For the case of negation and disjunction from Proposition 17 give a proof by providing a suitable total correctness argument.

5.3.3 Semidecidable predicates

For a predicate to be decidable we must have a program that returns the answer yes, or no. A generalization of this idea is one where we don't expect the characteristic function to be computable.

Definition 18. A predicate P on the natural numbers is **semidecidable** if and only if the partial function

$$\begin{array}{rcl} \mathbb{N} & \longrightarrow & \mathbb{B} \\ x & \longmapsto & \begin{cases} \mathsf{tt} & P(x) \\ \text{undefined} & \text{else.} \end{cases} \end{array}$$

is computable.

If we look into what the definition tells us, it means that there is a While program S with the property

$$\{\ \mathtt{n} = n \in \mathbb{N} \wedge P(n)\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = 1\ \}.$$

This means that a program S satisfying the condition may fail to terminate for some inputs, and that does not contradict the definition as long as these inputs does not satisfy the predicate P. It is also possible that the program may terminate for these inputs, and in that situation there is no condition on the output.

We note that there are no predicates which are semidecidable and not decidable which are simple to reason about, see below for some example.

Exercise 53.
Prove that a decidable predicate is also semidecidable.

Exercise 54.
If P and Q are semidecidable, which of the following are also semidecidable?

(a) $\neg P$,

(b) $P \wedge Q$,

(c) $P \vee Q$,

(d) $P \rightarrow Q$.

The reason for introducing this notion is that a number of famous problems are semidecidable or undecidable.

- The Halting Problem (see Sections 5.4 and 5.5.2) is semidecidable.
- Determining whether a context-free grammar is *universal*, that is generates all strings, or whether it is *ambigious*, are both undecidable.
- The problem of checking the validity of statements in first-order logic is semidecidable. This fact does not prevent us writing very effective theorem provers for first-order logic.
- Hilbert's tenth problem, the problem of deciding whether a Diophantine equation (multivariable polynomial equation) has a solution in integers, is undecidable.

In Section 4.7.4 we mention that it is unknown whether the program

```
while ¬n=1 do
    x:=n; m:=0;
    while 2≤x do
        x:=x−1; m:=m+1;
    if x=0 then
```

```
        n:=m
    else
        n:= 3×n+1
```

terminates for all inputs. If we call this program S we may use it to define a predicate P on the natural numbers given by

$$P(x) \quad \text{if and only if} \quad \texttt{S} \text{ terminates for input } \texttt{n} = x.$$

Just because it is currently unknown precisely for which natural numbers this predicate holds does not mean it is undecidable. And indeed, if it turns out that the predicate holds for all natural numbers then the program x := n provides a decision procedure for this predicate.

5.4 The Halting Problem

In Section 5.2 our first result for this chapter is the existence of uncomputable functions. However, at that point we do not give an example of such a function, we merely argue that they have to exist due to the mismatch in size between the set of functions from $\mathbb{N}$ to $\mathbb{N}$ and the set of While programs. In this section we provide such an uncomputable function.

In the introduction to this chapter we discuss the idea of self-reference as being an important aspect of the Incompleteness Theorem, and it also plays a crucial role for our example of an uncomputable function. We need to be able to use the While language to talk about itself, and we may do this by supplying While programs as inputs to such programs.

The fact that a While program may fail to terminate for some of its inputs is a behaviour that these notes address a number of times: In Chapter 2 we look at how this behaviour is described in the operational semantics. In such a case the small step operational semantics produces a never ending sequence of transitions, while the big step operational semantics is not able to make any statements about this situation. In Chapter 4 we provide a system for partial correctness which makes guarantees only if the program in question terminates in the situation under consideration.

A question one may ask is whether it is possible to check programs for non-termination for specific inputs. As usual we expect an input for our program to be given in the variable n, and so the claim that a program S terminates when run with a given input may be stated as

$$\{\ \texttt{n} = n \in \mathbb{N}\ \}\ \texttt{S}\ \{\ \Downarrow\ \}.$$

Of course we would like to be able to effectively *decide* whether our program terminates with a given input, so we might ask ourselves whether it is possible to write a program H (for 'halt') that

- takes as inputs two natural numbers,
  - one of which, given in the variable $\texttt{m} = m$, is interpreted as the index of a While program and
  - the other, given in the variable $\texttt{n} = n$, is interpreted as the input to that program, and which
- produces an output in the variable x such that
  - $\texttt{x} = 1$ if the program with index m terminates when run on the input n and
  - $\texttt{x} = 0$ if that program does not terminate when run on that input.

In other words, we are asking whether such a decision procedure exists in the While language, which would tell us that this version of the termination property is decidable.

Assume we had a program H with the property stated above.

In Section 6.3 we introduce the bijection ϕ_S between While programs and $\mathbb{N}$. Therefore given a natural number $n \in \mathbb{N}$ there is a While program S encoded by n. We refer to n as the **index of S**.

Assume that S_m is the program with index m. Below we use P for the predicate

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S_m}\ \{\ \Downarrow\ \} \qquad \text{holds.}$$

We may describe the behaviour of this program as follows:

$$\{\ \mathtt{m} = m \in \mathbb{N} \wedge \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{H}\ \{\ \Downarrow\ (P \to \mathtt{x} = 1) \wedge (\neg P \to \mathtt{x} = 0)\ \}.$$

We may now use this program and ensure that both inputs are equal, so the new program H' may be given as n := m; H. For this program the condition P becomes the condition P'

$$\{\ \mathtt{n} = m \in \mathbb{N}\ \}\ \mathtt{S_m}\ \{\ \Downarrow\ \} \qquad \text{holds,}$$

and the behaviour of the program may be described by

$$\{\ \mathtt{m} = m \in \mathbb{N}\ \}\ \mathtt{H'}\ \{\ \Downarrow\ (P' \to \mathtt{x} = 1) \wedge (\neg P' \to \mathtt{x} = 0)\ \}.$$

We now use the program H' to build a third program H'' which fails to terminate if H' produces the answer 1, and which produces the answer 1 if H' produces the answer 0.

```
H';
if x=1 then
   while tt do skip
else
   x := 1
```

What can we say about the behaviour of the program H''? In the following derivation we use L for the loop and I for the if statement. Note that we are not able to guarantee that H'' terminates and so we are only able to use partial correctness triples.

1. $\{\ \mathtt{m} = m \in \mathbb{N}\ \}\ \mathtt{H'}\ \{\ (P' \to \mathtt{x} = 1) \wedge (\neg P' \to \mathtt{x} = 0)\ \}$ by the construction of H'.

2. $\{\ (P' \to \mathtt{x} = 1) \wedge (\neg P' \to \mathtt{x} = 0) \wedge \mathtt{x} = 1\ \}$ **skip** $\{\ (P' \to \mathtt{x} = 1) \wedge (\neg P' \to \mathtt{x} = 0)\ \}$ by the skip rule.

3. $\{\ (P' \to \mathtt{x} = 1) \wedge (\neg P' \to \mathtt{x} = 0) \wedge \mathtt{x} = 1\ \}$ **while tt do skip** $\{\ \neg\mathbf{tt}\ \}$ by the while rule since any conjunction formed with $\neg\mathbf{tt}$ is equivalent to $\neg\mathbf{tt}$.

4. $\{\ (P' \to \mathtt{x} = 1) \wedge (\neg P' \to \mathtt{x} = 0) \wedge \mathtt{x} = 1\ \}$ **while tt do skip** $\{\ \neg P' \to \mathtt{x} = 1\ \}$ by the rule of consequence since $\neg\mathbf{tt} = \mathbf{ff}$ implies anything.

5. $\{\ (P' \to \mathtt{x} = 1) \wedge (\neg P' \to \mathtt{x} = 0) \wedge \neg(\mathtt{x} = 1)\ \}\ \mathtt{x} := 1\ \{\ \neg P' \to \mathtt{x} = 1\ \}$ by the := rule since we know that for the given precondition $\neg(\mathtt{x} = 1)$ implies $\neg P$.

6. $\{\ (P' \to \mathtt{x} = 1) \wedge (\neg P' \to \mathtt{x} = 0)\ \}$ **if** $\mathtt{x} = 1$ **then** L **else** $\mathtt{x} = 1$ $\{\ \neg P' \to \mathtt{x} = 1\ \}$ by the if rule.

7. $\{\ \mathtt{m} = m \in \mathbb{N}\ \}\ \mathtt{H'};\ \mathtt{I}\ \{\ \neg P' \to \mathtt{x} = 1\ \}$ from 1 and 6 by the ; rule.

We now check what happens if the input m to our program is the index ϕ_SH'' of the program H''. Spelling out the condition P', we have that it is

$$\{\ \mathtt{n} = \phi_S\mathtt{H''}\ \}\ \mathtt{H'}\ \{\ \Downarrow\ \} \qquad \text{holds.}$$

So for H'' we have

$$\{\ \mathtt{m} = \phi_S\mathtt{H''}\ \}\ \mathtt{H''}\ \{\ \neg P' \to \mathtt{x} = 1\ \}$$

but that says if H' fails to terminate when given the input ϕ_SH'' then H'' terminates with output $\mathtt{x} = 1$, but we can see that if H' fails to terminate with that input then H'' also fails to terminate.

Hence we have derived a contradiction, and this means that a program H from our assumption cannot exist.

The contradiction derived above to prove the following result is quite subtle—we use a predicate describing the behaviour of H' to show that we obtain a postcondition that cannot possibly hold. Readers may find this confusing, and that is a very typical reaction to being exposed to this argument.

Theorem 19 (Halting Problem)**.** *The problem whether the program with index m terminates when given the input n is undecidable.*

Above we claim that the Halting Problem is semidecidable, but in order to explain how that works we need to produce more machinery, which we do in the following section, see Section 5.5.2.

5.5 Universality of Computation

We now discuss what it means for a model of computation to be *universal*. This section is technical and contains a number of ideas that may take some time to digest.

Previously we asked whether given a partial function from $\mathbb{N}$ to $\mathbb{N}$, can we find a program that computes it. We now look at this from the opposite perspective: Given a While program, can we define a partial function it computes?

Definition 20. The **partial function computed by a** While **program** S is given by

$$\begin{aligned} \mathbb{N} &\longrightarrow \mathbb{N} \\ n &\longmapsto \begin{cases} k & \{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow \mathtt{x} = k\ \} \\ \text{undefined} & \text{else.} \end{cases} \end{aligned}$$

We think of S as computing a partial function that takes an input from $\mathbb{N}$ in the variable n and produces as output the value of x after termination. In Definition 13 we said that a partial function is computable if there is a program that computes its outputs for those inputs where it is defined; the definition above goes in the opposite direction by taking a program and defining the partial function it defines.

Example 18. The partial function computed by the program

```
x := n
```

is the identity function on $\mathbb{N}$, which is a total function.

Example 19. We know from Section 4.7.1 that the program

```
if 0≤n then
    x := 2×n
else
    x := −2×n−1
```

computes the (total) function β from Section 6.2.1 that encodes integers as natural numbers.

Example 20. The program

```
a := a ;  b := b ;  c := c ;
x := 0 ;  y := 0 ;
while ¬y=1 do
```

```
if a×x×x+b×x+c=0 then
    y:=1
else
    if a×x×x−b×x+c=0 then
        y:=1
    else
        skip
x:=x+1
```

(compare Example 2 and Section 4.7.3) computes the following partial function:

$$\mathbb{N} \longrightarrow \mathbb{N}$$

$$n \longmapsto \begin{cases} x & x \text{ is the smallest natural number solution of } ax^2+bx+c=0 \\ \text{undefined} & ax^2+bx+c=0 \text{ has no natural number solution.} \end{cases}$$

This description is a bit confusing. We are in fact in one of two situations.

- The equation $ax^2+bx+c=0$ has a solution in the natural numbers. In that case, the function computed is the constant (total) function which returns the smallest of these solutions for all inputs.
- The equation $ax^2+bx+c=0$ has no solution in the natural numbers. In that case, the function computed is the partial function that is undefined everywhere.

The program does not depend on the value of the variable n, hence it produces a constant output if it produces an output at all.

Of course the output of a program may vary depending on the values of the other variables at the start—we only use an output that works for *all states which provide the same input value for* n.

Example 21. The program

```
x:=n+y
```

computes the partial function that is undefined everywhere since different values of y when the program starts cause different output in the variable x upon termination, so there is no value for x that matches the requirements for k from Definition 20.

Exercise 55.
For each of the following While programs determine the partial function they compute.

(a)

```
if 100<n then
    x:=n;
else
    x:=y

if 0≤y then
    while 0<y do
        y:=y−1;
else
    while y<0 do
        y:=y+1;
```

```
x := y
```

(b)

```
while 0<y do
    n := n+n ; y := y-1 ;
x := n
```

(d)

```
n := n+2×y×y ;
while 0<y×y do
    n := n-2 ; y := y-1 ;
x := n
```

We further know from Section 6.3 that there is a bijection between natural numbers and While programs, and that for a given natural number i we may find the program with index i. Given $i \in \mathbb{N}$ we use this idea to define

$$\eta_i : \mathbb{N} \rightharpoonup \mathbb{N}$$

as the partial function computed by the program with index i.

Example 22. The program with index 0 is **skip**, and the partial function computed by this program is undefined everywhere, so $\eta_0 n$ is undefined for all input values n. The reason for this is that for different start states we may have different values in the variable x, and so we are unable to use a condition for the variable n to ensure a particular output.

Example 23. We assume that the variable x is x_0 for the purpose of the encoding function ϕ_S from Section 6.3. The program

```
x := 0
```

has the index

$$\phi_S(\mathtt{x} := 0) = 1 + 4 \cdot \phi(\phi_v \mathtt{x_0}, \phi_A 0) = 1 + 4 \cdot \phi(1, 0) = 1 + 4(\cdot 2^1 \cdot (2 \cdot 0 + 1) - 1) = 5,$$

so we have that η_5 is the total function that maps every input to 0.

Given sets S and T we use $\mathsf{PFun}(S, T)$ for the set of all partial functions from S to T. We may now define a function

$$\begin{aligned} h : \mathbb{N} &\longrightarrow \mathsf{PFun}(\mathbb{N}, \mathbb{N}) \\ i &\longmapsto \eta_i, \end{aligned}$$

which maps a natural number i to the function computed by the program with index i.

We note that this function is not injective since there are several functions that compute the same function

Example 24. The programs

```
x := n+2
```

```
x := n+2 ;
skip
```

both compute the (total) function

$$\begin{aligned} \mathbb{N} &\longrightarrow \mathbb{N} \\ x &\longmapsto x+2. \end{aligned}$$

We assume that x is x_0, and that n=x_1.

Since

$$\phi_A(\mathtt{n}+2) = 2+5\cdot\phi(\phi_A\mathtt{n}, \phi_A 2) = 2+5\cdot(2^{1+5}\cdot(2\cdot 10+1)-1) = 2+5\cdot(2^6\cdot 21-1) = 6717$$

we have

$$\phi_S(\mathtt{x} := \mathtt{n}+2) = 1+4\cdot\phi(\phi_S\mathtt{x_0}, \phi_A(\mathtt{n}+2)) = 1+4\cdot(2^0\cdot(2\cdot 6717+1)-1) = 1+4\cdot 13434 = 53737.$$

So η_{53737} is the function given above, but this is also true for the index i of the program $\mathtt{x} := \mathtt{n}+2$; **skip** which is equal to

$$2+4\cdot\phi(\phi_s(\mathtt{x} := \mathtt{n}+2), \phi_S(\mathbf{skip})) = 2+4\cdot(2^{53737}-1).$$

5.5.1 The universal partial function

In Section 6.4 we introduce the idea of a program that computes the small step operational semantics for a program and a state. In this section we introduce the *Universal Program*, which is a program which can simulate every While program.

We are claiming below that there is a program U which can implement any computable partial function. We begin by defining the *universal partial function*

$$\zeta : \mathbb{N}\times\mathbb{N} \rightharpoonup \mathbb{N}$$

by setting

$$\zeta(i,j) = \eta_i j.$$

In other words the output of ζ for input (i,j) is the result of applying the partial function computed by the program with index i to the input j.

If we keep the input i fixed we obtain a partial function

$$\begin{aligned} \zeta(i,-) : \mathbb{N} &\rightharpoonup \mathbb{N} \\ k &\longmapsto \zeta(i,k) = \eta_i k, \end{aligned}$$

which is the partial function computed by the program with index i.

Example 25. Based on the examples from the previous section we have that

- $\zeta(0,-)$ is the partial function that is undefined for all inputs;
- $\zeta(5,-)$ is the total function that maps every input to 0 and
- $\zeta(42993,-)$ is the total function that maps each input n to $n+2$.

Exercise 56.
Show that by choosing i appropriately, $\zeta(i,-)$ can be equal to the following functions. You may assume that x = x_0 and n = x_1.

(a) The identity function $\mathsf{id}_{\mathbb{N}} : \mathbb{N} \longrightarrow \mathbb{N}$ on $\mathbb{N}$.

(b) The constant function from $\mathbb{N}$ to $\mathbb{N}$ which maps each input to 2.

(c) The increment function from $\mathbb{N}$ to $\mathbb{N}$ which adds one to its input.

Given a While program S and an input n, we may use ζ to tell us what the output (in the variable x) of S is for the given input. Of course S may fail to terminate for that input, but our partial function is allowed to be undefined in that situation. To apply ζ we just have to work out the index of S by determining ϕ_SS and then $\zeta(\phi_S\mathrm{S}, n)$ provides the output we are looking for. In other words ζ can tell us about *every* While program.

But we are expecting even more from ζ: Given a computable partial function

$$f : N \times \mathbb{N} \rightharpoonup \mathbb{N}$$

we expect to be able to use ζ to express what f does: We expect to find a computable function

$$h : \mathbb{N} \longrightarrow \mathbb{N}$$

with the property that

$$f(i, j) = \zeta(hi, j) = \eta_{hi} j.$$

That means we are able to compute the index hi of the program that computes the partial function $f(i, -)$.

In other words, ζ can help us to express every computable partial function with two natural number inputs, and that is why we refer to it as universal. We look at this claim in Section 5.5.3.

5.5.2 The universal partial function is computable

We do not give the program that computes ζ, we merely provide an informal argument that this program exists. The reason becomes clear as we provide our explanation. It is known as the *universal program* because it can simulate all While programs.

Note that this implies that the Halting Problem is semidecidable: Given a program S and an input n we may run the program that witnesses the computability of ζ on the inputs ϕ_SS and n, and if S terminates for that input, then so does our universal program, and we obtain the correct output in the variable x. If the program S fails to terminate then so does the program witnessing the computability of ζ, but that does not matter since for semidecidable problems we do not have to provide an answer if said answer is no.

So what would one have to do to write a program that computes ζ? The program works with encodings of programs, and encodings of states, as described in Section 6.4. The program has to do enough to decode its input to be able to update the state appropriately. For example, if the input program is of the form x := 1; S then its encoding is of the form $2 + 4\phi(\phi_S(\mathrm{x} := 1), \phi_S\mathrm{S})$ and by determining the remainder when dividing by 4 the universal program is able to discover this. If x = x_0 in our encoding then $\phi_S(\mathrm{x} := 1) = 1 + 4\phi(\phi_A\mathrm{x_0}, \phi_A 1) = 1 + 4\phi(0, 5) = 44$, and the program is able to work out which variable is updated to obtain the new state, and it can then update the (encoding of) the input state accordingly. The program can then move on to deal with ϕ_SS, the encoding of the remaining program after x := 1 has been executed.

The above description hopefully gives an idea of what the universal program looks like—there is a lot of code dealing with encodings, but the nature of our encodings are such that it is possible to decode instructions step by step, compute their effect, and update the (encoding of) the state.

5.5.3 The Parameterization Theorem

In Section 5.5.1 we claim that the universal partial function ζ allows us to compute every computable partial function with two variables, because given such a partial function it is possible

$$f : \mathbb{N} \times \mathbb{N} \longrightarrow \mathbb{N}$$

we may define a computable (total) function

$$h : \mathbb{N} \longrightarrow \mathbb{N}$$

with the property that for all $i, j \in \mathbb{N}$ we have

$$f(i, j) = \eta_{hi} j,$$

and we would like this function h to be computable—but we have to use this term informally here, see below for an explanation.

We first look at the question of what it means for f to be computable, which means applying Definition 14 using the injection $\phi' : \mathbb{N} \longrightarrow \mathbb{N} \times \mathbb{N}$. The partial function f being computable means there exists a program S with the property that

$$\{ \ \mathtt{n} = n \in \mathbb{N} \wedge f(\phi' n) \text{ defined } \ \} \ \mathtt{S} \ \{ \ \Downarrow \ \mathtt{x} = f(\phi' n) \ \}.$$

But this description can be improved since we now that ϕ' is the inverse of ϕ, and this allows us to formulate the condition using ϕ rather than ϕ'. We know that $(i, j) = \phi' n$ if and only if $\phi(i, j) = n$, so the condition given above is equivalent to the following:

$$\{ \ i, j \in \mathbb{N} \wedge \mathtt{n} = \phi(i, j) \wedge f(i, j) \text{ defined } \ \} \ \mathtt{S} \ \{ \ \Downarrow \ \mathtt{x} = f(i, j) \ \}.$$

We further know that there is a program T witnessing the computability of the function ϕ, see for example Exercises 46 and 48. For the program T we have

$$\{ \ \mathtt{m} = i \in \mathbb{N} \wedge \mathtt{n} = j \in \mathbb{N} \ \} \ \mathtt{T} \ \{ \ \Downarrow \ \mathtt{x} = \phi(i, j) \ \}.$$

We may create a program where we execute T and feed its output to the program S.

```
T;
n:=x;
S
```

For this program we are able to establish that

$$\{ \ \mathtt{m} = i \in \mathbb{N} \wedge \mathtt{n} = j \in \mathbb{N} \ \} \ \mathtt{T;\ n := x;\ S} \ \{ \ \Downarrow \ \mathtt{x} = f(i, j) \ \}.$$

Instead of viewing i as an input to our program, we may instead consider the following program:

```
m:=i;
T;
n:=x;
S
```

If we call this program S(i), we can derive the property that

$$\{ \ \mathtt{n} = j \in \mathbb{N} \ \} \ \mathtt{S(i)} \ \{ \ \Downarrow \ \mathtt{x} = f(i, j) \ \}.$$

In other words, the program S(i) computes the function $f(i, -)$.

We may now compute the index $\phi_S \mathtt{S(i)}$ of the program S(i) and set

$$hi = \phi_S \mathtt{S(i)}$$

to obtain the following properties:

- The function $h : \mathbb{N} \longrightarrow \mathbb{N}$ is computable since the function ϕ_S is computable for systems that can take strings as inputs (see Section 6.3), and since generating the program S(i) is a computable process given the programs T and S.
- We have
$$f(i, j) = \eta_{hi} j,$$
since the partial function computed by the program S(i) (which has index hi) is the function $f(i, -)$.

We have shown the following result.

Theorem 21 (Parameterization Theorem)**.** *If $f : \mathbb{N} \times \mathbb{N} \rightharpoonup \mathbb{N}$ is a computable partial function there is a computable function $h : \mathbb{N} \longrightarrow \mathbb{N}$ such that*

$$f(i, j) = \eta_{hi} j.$$

We may now apply the Parameterization Theorem to our universal partial function ζ to obtain the existence of a computable function

$$h : \mathbb{N} \longrightarrow \mathbb{N}$$

with the property that for all $i, j \in \mathbb{N}$ we have that

$$\zeta(i, j) = \eta_{hi} j,$$

delivering on our promise from Section 5.5.1. This ensures that for any computable partial function in two variables we are able to compute the index of the program that realizes that function, and we may then run the universal program on that index to compute outputs for the given function.

5.5.4 Functions of several variables

Our considerations of partial functions computable by While programs above is restricted to partial functions with one input and one output since the other input of $f : \mathbb{N} \times \mathbb{N} \longrightarrow \mathbb{N}$ is used to identify a suitable While program. A typical While program may be thought of as computing a partial function with several inputs, and several outputs, and there is an obvious question whether our considerations from above generalize to this setting.

We may generalize our notion of the partial function computed by a given program S with index i to a function of several variables as

$$\begin{aligned} \eta_i^{(n)} : \mathbb{N}^n &\rightharpoonup \mathbb{N} \\ (i_1, i_2, \dots, i_n) &\longmapsto \begin{cases} k & \{\ \mathtt{x_1} = i_1 \wedge \dots \wedge \mathtt{x_n} = i_n\ \}\ \mathtt{S}\ \{\ \Downarrow \mathtt{x} = k\ \} \\ \text{undefined} & \text{else.} \end{cases} \end{aligned}$$

We note that our argument for the computability of the universal partial function ζ in Section 5.5.2 does not distinguish between the input variable and other variables, and indeed, to compute the effect of a given input program S it is necessary to follow all the changes to variables. Hence the argument provided there also applies to the computability of the universal $(n+1)$-ary partial function

$$\begin{aligned} \zeta^{(n)} : \mathbb{N}^{n+1} &\rightharpoonup \mathbb{N} \\ (i, x_1, x_2, \dots, x_n) &\longmapsto \eta_i^{(n)}(x_1, x_2, \dots, x_n). \end{aligned}$$

It is further possible to generalize the Parameterization Theorem to functions of this type; this is known as the *s-m-n* Theorem. Hence the ideas laid out here apply to such functions as well. We note that we do not have to worry about functions with several outputs since we may describe a function with k outputs by k functions with one output.

5.6 The Church-Turing Thesis

In these notes we consider one example of an abstract model of computation in the form of While and its operational semantics. We use that model of computation to ask questions about which functions are computable in this model, and whether there are undecidable problems.

A natural question is whether our results are only valid for this one model of computation, and whether looking at a different abstract model would lead to different results.

When logicians and mathematicians first began to think about these questions, they came up with different models of computation—we mention the λ-calculus and Turing machines in the introduction. These models look very different, and it was something of a surprise to find that the notion of computability does not depend on the model one studies. At the time notions such as 'computing' or a 'decision procedure' were new and without a settled idea of what that might mean. If readers are interested in an original approach to this question they are encouraged to look at the following paper:

Alan Turing. *On Computable Numbers, with an Application to the Entscheidungsproblem*, Proceedings of the London Mathematical Society, Series 2, 42 (1936–7), pp 230–265. An online version is available at http://www.abelard.org/turpap2/tp2-ie.asp.

Based on his careful considerations Turing made the argument that anything that could be said to be computed by carrying out a finite number of steps in what he described as a 'purely mechanical manner' could be computed by what we now call Turing machines. At much the same time, Church talked about 'effectively calculable' functions, prompting Turing to prove that his notion and Church's notions of computable are, in fact, equivalent.

Additional methods for computation were described in the thirties, and they all turned out to have precisely the same notion of computable function in the sense that their sets of computable functions agree. Both Church and Turing formulated the following idea:

Thesis 22 (Church-Turing)**.** *For sufficiently powerful models of computation the notion of computability is the same.*

There are some abstract systems of computation that are not sufficiently powerful to compute all the functions that the While language can compute—for example, we could consider an abstract model of computation based on finite state machines over the alphabet $\{0, 1, 2, \ldots, 9\}$, where we assign a natural number to each state, and we define a partial function $f : \mathbb{N} \rightharpoonup \mathbb{N}$ by setting $fn = m$ if and only if when viewing n as a word over the given alphabet, the automaton stops in state m, and that is an accepting state. This model of computation has many uncomputable function, for example, the identity function for $\mathbb{N}$ is not computable in this system since a finite state machine only has finitely many states and so for large enough inputs there is no state with that number.

But there are no systems that are more powerful than the While system discussed in these notes, and many examples have been added that reinforce the idea that Turing machines, the (untyped) λ-calculus, or indeed the While programming language allow us to say everything about computable functions that there is to be said.

Quite a bit of work has gone into questions such as what can be said about physical computation devices, or whether quantum computing may change what is computable.

5.7 Summary

In this section we ask the question what one might mean by 'computable', and whether there are problems whose solutions cannot be computed. In order to be able to make this rigorous our notion of computable refers to mathematics, and we ask which functions from $\mathbb{N}$ to $\mathbb{N}$ we may hope to compute. A particular instance of computability is concerned with questions that have yes/no answers, which leads to notions of decidability, and semidecidability. We finally give an explicit example of an undecidable predicate in the form of the Halting problem. We look more generally into the partial function computed by a program and reason that there is a While program that can simulate all While program. This is known as the *universal program* and its existence lets us reason that the Halting problem is semidecidable. We conclude by noting that our notion of computability is robust and agrees with that based on other abstract models of computation.

5.8 Solutions to Exercises

Solution 46.
We provide a suitable program for each part.

(a) Squaring an input may be carried out by the program

```
x := n×n
```

We call this program S. A suitable Hoare triple establishing that S computes the desired function is

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = n^2\ \}.$$

The assignment rule establishes that this triple hols.

(b) We may reemploy the program from Example 4, just changing the name of the variable that provides the output.

```
y := n−1;
z := 1;
while y>1 ∧ z=1 do
  r := n;
  while x≤r do
     r := r−y;
     if r=0 then
        x := 0;
     else
        skip;
   y := y−1;
```

This program has property that it terminates for all suitable inputs, and that once it has done so, the variable x will be 0 if n is not prime, and 1 otherwise, and so calling this program S one may derive the Hare triple

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ (n \text{ prime} \to \mathtt{x} = 1) \wedge (\neg(\mathtt{n} \text{ prime}) \to \mathtt{x} = 0)\ \},$$

but the question did not ask for this to be carried out.

(c) For a program R that witnesses the computability of the given composite function we expect the Hoare triple

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{R}\ \{\ \Downarrow\ \mathtt{x} = g(fn)\ \}$$

to hold.

Assume we have a program S that computes f and a program T that computes g. We may run the first followed by the second, provided that we take care to ensure that the output of the first program becomes the input of the second. We may do this as follows:

```
S; n := x; T
```

We know that for S we have

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = fn\ \},$$

and for T we have

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{T}\ \{\ \Downarrow\ \mathtt{x} = gn\ \}.$$

We also have

$$\{\ \ \}\ \mathtt{n} := \mathtt{x}\ \{\ \Downarrow\ \mathtt{n} = \mathtt{x}\ \}$$

by the assignment rule. Using the ; rule we may put together the triples for S and for the assignment to obtain

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S};\ \mathtt{n} := \mathtt{x}\ \{\ \Downarrow\ \mathtt{n} = fn\ \},$$

and noting that $\mathtt{n} = fn$ implies $\mathtt{n} \in \mathbb{N}$, we may use the ; rule again to combine this triple with the one about T to obtain

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S};\ \mathtt{n} := \mathtt{x};\ \mathtt{T}\ \{\ \Downarrow\ \mathtt{x} = g(fn)\ \}.$$

(d) Assume we have a program R that establishes the computability of the given function g. A suitable Hoare triple is

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{R}\ \{\ \Downarrow\ (fn \bmod 2 = 0 \to \mathtt{x} = 1) \wedge (fn \bmod 2 = 1 \to \mathtt{x} = 0)\ \}.$$

As indicated in the question we know that there is a program S with the property that

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = n \bmod 2\ \}.$$

Note that we need to ensure that it is the input n whose remainder when dividing by 2 is computed here.

If the function f is computable then we may find a program T with the property that

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{T}\ \{\ \Downarrow\ \mathtt{x} = fn\ \}.$$

To compute g we may use the program

```
T;
n:=x;
S;
if x=0 then
    x:=1;
else
    x:=0;
```

In other words, we carry out the program to implement f, then check whether the result is even, and note that we have to swap the values 0 and 1.

The assumption about T may be combined with

$$\{\ \mathtt{x} = fn\ \}\ \mathtt{n} := \mathtt{x}\ \{\ \Downarrow\ \mathtt{n} = fn\ \}$$

which is obtained from the assignment rule and the assumption about S to obtain

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{T};\ \mathtt{n} := \mathtt{x};\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = fn \bmod 2\ \}$$

using the ;= rule twice.

It remains to show that the if statement I gives the desired outcome.

1. $\{\ \mathtt{x} = fn \bmod 2 \wedge \mathtt{x} = 0\ \}\ \mathtt{x} := 1\ \{\ \Downarrow\ (fn \bmod 2 = 0 \to \mathtt{x} = 1) \wedge (fn \bmod 2 = 1 \to \mathtt{x} = 0)\ \}$ by the := rule, noting that the left input to the conjunction holds since we know that both, $fn\mathrm{mod} = 0$ and $\mathtt{x} = 1$, and the right input to the conjunction holds since $fn\mathrm{mod} = 1$ does not hold.

2. $\{\ x = fn \bmod 2 \wedge \mathtt{x} = 1\ \}\ \mathtt{x} := 0\ \{\ \Downarrow\ (fn \bmod 2 = 0 \rightarrow \mathtt{x} = 1) \wedge (fn \bmod 2 = 1 \rightarrow \mathtt{x} = 0)\ \}$ by the := rule for similar reasons to the previous statement.
3. $\{\ x = fn \bmod 2\ \}\ \mathtt{I}\ \{\ \Downarrow\ (fn \bmod 2 = 0 \rightarrow \mathtt{x} = 1) \wedge (fn \bmod 2 = 1 \rightarrow \mathtt{x} = 0)\ \}$ by the if rule.

We may now apply the ; rule to the Hoare triple for T; n := x; S and 3 from above to establish the desired Hoare triple for the entire program R using the rule of consequence as appropriate.

(e) We adjust the notion of computable function by allowing the source of the function to be $\mathbb{Z}$ rather than $\mathbb{N}$, which means asking for the existence of a program S with the property that

$$\{\ \mathtt{n} = n\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = \beta n.\ \}$$

We may also adjust our notion of computable function to allow outputs that are integers merely by changing the type of the function.

Suitable programs are given in Exercise 8.

(f) We may adjust the notion of computable to fit ϕ by making the source of our function $\mathbb{N} \times \mathbb{N}$ and assuming that we have two inputs given in m and n. A suitable Hoare triple for a program S that witnesses the computability of ϕ is

$$\{\ \mathtt{m} = m \in \mathbb{N} \wedge \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = \phi(m, n)\ \}.$$

A suitable program S is given in Exercise 9, which we repeat here.

```
y := 1;  x := 1;
while  y≤m  do
    x := 2×x;
    y := y := y+1;
x := x×(2×n+1)
```

Solution 47.
We provide a suitable program for each part.

(a) We know how to compute $m \bmod n$ for natural numbers from Example 1, and in Exercise 5 we look at the issue of computing the remainder for integer division, and we may adjust that program to look for integer division by 2. A suitable program is the following.

```
x := 0;
if  0≤ n  then
    while  2≤ n  do
            n:=n−2;  x:=x+1;
else
    while  n≤−2  do
        n:=n+2;  x:=x+1;
```

Note that our program does terminate for inputs for which the partial function is not defined—our specification says nothing about this case. Our program returns n mod 2 for all inputs.

If we wanted our program to match the given partial function by not terminating for inputs for which the partial function is not defined we could add the following at the end:

```
if  n=0  then
```

```
        skip
    else
        while tt do skip
```

(b) A suitable program for this is given in Example 5. Again note that this program always terminates, but there is no issue with this when it comes to satisfying the definition. The program returns the largest number whose square is less than or equal to the input.

(c) We merely have to check whether the input given in the variable n solves the problem, so the program

```
x := 0;
if a×n×n + b×n + c = 0 then
    x := 1
else
    if a×n×n − b×n + c = 0 then
        x := 1
    else
        skip
```

is suitable. This program terminates for all inputs and produces the output 0 in the variable x if the given partial function is undefined for the present input.

(d) The simplest way of adjusting Definition 13 is to allow partial functions with source set $\mathbb{N} \times \mathbb{N}$ and use the Hoare triple

$$\{\ \mathtt{m} = m \in \mathbb{N} \wedge \mathtt{n} = n \in \mathbb{N} \wedge f(m,n) \text{ defined }\ \}\ \mathtt{S}\ \{\ \Downarrow \mathtt{x} = f(m,n)\ \}.$$

In Exercise 7 we look at a program that finds a suitable solution if one exists. We may adjust that program to provide the desired output in the variable x as required by changing the value held in the variable x when a solution is found, which means that both loops have terminated.

```
x := 0;  y := 0;
while ¬(m×x+n×y=k) do
    x := x+1;  y := 0;
        while ¬(m×x+n×y=k)∧y<x do
            if m×y+n×x=k then
                w := x;  x := y;  y := w
            else
                y := y+1;
x := 1
```

Solution 48.
We discuss each part.

(a) In Exercise 46 the suggestion was to adjust the original definition Definition 12 by allowing integer inputs (or outputs).

The function β is from $\mathbb{Z}$ to $\mathbb{N}$. To apply Definition 14, we note that we need an injection $\gamma : \mathbb{N} \longrightarrow \mathbb{Z}$ and an injection $\delta : \mathbb{N} \longrightarrow \mathbb{N}$.

For γ we may use the function β' which is the inverse to β, and for δ we may use the identity function on $\mathbb{N}$. Hence for β to be computable we must have that the composite $\mathsf{id}_{\mathbb{N}} \circ \beta \circ \beta'$ is computable. but this is the identity function for $\mathbb{N}$, which certainly is computable.

In this situation it is easier to work with the general definition because in this case the composite function has a very simple program witness its computability.

The function β' has source set $\mathbb{N}$ and target set $\mathbb{Z}$. We may apply Definition 14 by setting γ to be the identity function on $\mathbb{N}$ and δ to be β. The resulting composite is also the identity function for $\mathbb{N}$, and so β' is also computable according to this definition.

(b) The situation is similar to the one from the previous part because we are able to use ϕ and its inverse in a similar manner.

Solution 49.
We discuss each part.

(a) For this function we could extend the definition of computable function by expecting to have a program S which takes input variables m and n in $\mathbb{N}$ and returns output variables x and y also in $\mathbb{N}$.

A suitable program is

```
x := n ;
y :=m
```

In order to show that the given function, say f, is computable by Definition 14 we require a function to take the role of γ, for which we may use ϕ', and a function to take the role of δ, for which we may use its inverse ϕ.

This gives a function $\phi \circ f \circ \phi'$, and this function maps a number k to $2^n(2m+1)-1$, where m is the highest power of 2 that divides $k+1$ and n is the number with the property that $2n+1$ is the largest odd divisor of k. Instead of writing a program that implements this directly it is easier to use several programs that appear in exercises in the notes. We have a program S that implements ϕ', and which produces outputs in the variables x and y, and a program T that implements the function ϕ. The following program shows that f is computable according to Definition 14:

```
S ;
m:= y ;
n := x ;
T
```

(b) Showing that this function is computable if we adjust the original definition amounts to providing a program that takes two natural number inputs in m and n and which produces an output in x.

```
x :=m-n
```

does that job.

If we want to use Definition 14 on the other hand we have to identify a function to take the place of γ, for which we may use ϕ' and a function to take the place of δ for which we may use β.

As in the previous part we use the fact that we have a program S that implements the function ϕ'. The program that implements β is short so we included it in full.

```
S ;
z := x-y ;
```

```
if 0≤z then
    x:=2z;
else
    x:=−2×z−1
```

Exercise 48 suggests it may be easier to use the general definition to show that a function is computable, and this exercise proves that this is not always the case.

Solution 50.
We provide a suitable program for each part.

(a) In Example 1 we have a program that for inputs in n and m returns m **mod** n in the variable r. We may change this program as follows to ensure it computes what we want it to compute, by using just one input n and dividing by 2, and eliminating irrelevant parts of the computation. This computes $\neg P$, so we have to change the output correspondingly.

```
r := n;
d := 0;
while 2 ≤ r do
    r := r−2;
if r=0 then
    x:=1
else
    x:=0
```

(b) This is exactly the same question as part (b) of Exercise 46.

(c) In Exercise 44 we provide a program that carries out this task.

Solution 51.
Assume we have programs S and T that witness the decidability of P and Q respectively. Assume that m is a variable that does not occur in S.

(a) We consider the following program.

```
m:=n;
S;
if x=1 then
    n:=m; T
else
    skip;
```

When this program is executed, the input value in n is remembered, and then the program S is executed. If that program returns the answer encoding **tt**, the original input is restored and T is executed. When that terminates the value in x is 1 if and only if both S and T return 1, that is of both, P and Q hold for the given input. If S terminates with the value of x not being 1 then that value must be 0, which means that P does not hold for the given input and so $P \wedge Q$ does not hold either. Hence this function computes the characteristic function of $P \wedge Q$.

(b) Here is a suitable program:

```
m:=n;
```

```
S;
if x=1 then
    skip
else
    n:=m; T
```

This program works on a similar principle to the previous one. If S tells us that the predicate P holds for the input we know that $P \vee Q$ also holds for that input and it is safe to return the value encoding **tt**. Otherwise we have to run T to find out whether Q holds for the given input, and it is safe to return that output.

(c) Again we provide a suitable program:

```
m:=n;
S;
if x=0 then
    x:=1
else
    n:=m; T
```

The idea for this program is similar to those from the previous parts. If P does not hold for the given input we know that $P \to Q$ does hold, so we return the value encoding **tt**. Otherwise we have to check whether Q holds, and the value we obtain from running T is the appropriate output.

Solution 52.
We use the programs provided in the solution for Exercise 51. Assume we have programs S and T that witness the decidability of P and Q respectively. Assume that m is a variable that does not occur in S. That means we have

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ (P(n) \to \mathtt{x} = 1) \wedge (\neg P(n) \to \mathtt{x} = 0)\ \}$$

and

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{T}\ \{\ \Downarrow\ (Q(n) \to \mathtt{x} = 1) \wedge (\neg Q(n) \to \mathtt{x} = 0)\ \}.$$

We use R for the first and R' for the second postcondition from above.

(a) The program given in the text for negation is

```
S;
if x=0 then
    x:=1
else
    x:=0
```

Below we use I for the if statement.

1. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ R\ \}$ by assumption about S.
2. $\{\ R \wedge \mathtt{x} = 0\ \}\ \mathtt{x} := 1\ \{\ \Downarrow\ (\neg P(n) \to \mathtt{x} = 1) \wedge (P(n) \to \mathtt{x} = 0)\ \}$ by the := rule where we note that the left input to the conjunction holds because both, $\neg P(n)$ and $\mathtt{x} = 0$, and the right input holds since $P(n)$ cannot hold for the precondition to hold.
3. $\{\ R \wedge \mathtt{x} = 1\ \}\ \mathtt{x} := 0\ \{\ \Downarrow\ (\neg P(n) \to \mathtt{x} = 1) \wedge (P(n) \to \mathtt{x} = 0)\ \}$ for similar reasons to 2.
4. $\{\ R\ \}\ \mathtt{I}\ \{\ \Downarrow\ (\neg P(n) \to \mathtt{x} = 1) \wedge (P(n) \to \mathtt{x} = 0)\ \}$ by the if rule.

5. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S};\ \mathtt{I}\ \{\ \Downarrow\ (\neg P(n) \to \mathtt{x} = 1) \wedge (P(n) \to \mathtt{x} = 0)\ \}$ from 1 and 4 by the ; rule.

(b) The program given for this case in the solution to Exercise 51 is

```
m:=n;
S;
if x=1 then
    skip
else
    n:=m; T
```

Again we use I for the if statement.

1. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{m} := \mathtt{n}\ \{\ \Downarrow\ \mathtt{m} = n \in \mathbb{N}\ \}$ by the := rule.
2. $\{\ \mathtt{m} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{m} = n \in \mathbb{N} \wedge R\ \}$ by assumption since m is not mentioned in S.
3. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge R \wedge \mathtt{x} = 1\ \}$ **skip** $\{\ \Downarrow\ \mathtt{m} = n \in \mathbb{N} \wedge R \wedge \mathtt{x} = 1\ \}$ by the skip rule.
4. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge R \wedge \mathtt{x} = 1\ \}$ **skip** $\{\ \Downarrow\ (P \vee Q)(n) \to \mathtt{x} = 1\ \}$ by the rule of consequences since for R and $\mathtt{x} = 1$ both to hold it must be the case that $P(n)$, and so $(P \vee Q)(n)$, and $\mathtt{x} = 1$ both to hold.
5. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge R \wedge \neg(\mathtt{x} = 1)\ \}\ \mathtt{n} := \mathtt{m}\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge R \wedge \neg(\mathtt{x} = 1)\ \}$ by the := rule.
6. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge \neg P(n) \wedge \mathtt{x} = 0\ \}\ \mathtt{T}\ \{\ \Downarrow\ (P \vee Q)(n) \to \mathtt{x} = 1\ \}$ since by the assumption for T the value of x is set to 1 if and only if $Q(n)$ holds, and $(P \vee Q)(n)$ holds when we know that $P(n)$ does not hold if and only if $Q(n)$ holds.
7. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge R \wedge \neg(\mathtt{x} = 1)\ \}\ \mathtt{n} := \mathtt{m};\ \mathtt{T}\ \{\ \Downarrow\ (P \vee Q)(n) \to \mathtt{x} = 1\ \}$ by 5 and 6 using the ; rule and noting that the postcondition of 5 implies the precondition of 6 since $\neg(\mathtt{x} = 1)$ can only hold in the presence of R if $\neg P(n)$.
8. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge R\ \}\ \mathtt{I}\ \{\ \Downarrow\ (P \vee Q)(n) \to \mathtt{x} = 1\ \}$ from 4 and 7 from the if rule.
9. $\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{m} := \mathtt{n};\ \mathtt{S};\ \mathtt{I}\ \{\ \Downarrow\ (P \vee Q)(n) \to \mathtt{x} = 1\ \}$ from 1, 2 and 8 using the ; rule.

Solution 53.
Assume we have a decidable predicate P and a While program S which witnesses that. Then we know that

$$\{\ \mathtt{n} = n \in \mathbb{N}\ \}\ \mathtt{S}\ \{\ \Downarrow\ (P(n) \to \mathtt{x} = 1) \wedge (\neg P(n) \to \mathtt{x} = 0)\ \}.$$

Hence we also have

$$\{\ \mathtt{n} = n \in \mathbb{N} \wedge P(n)\ \}\ \mathtt{S}\ \{\ \Downarrow\ (P(n) \to \mathtt{x} = 1) \wedge (\neg P(n) \to \mathtt{x} = 0) \wedge P(n)\ \},$$

and the postcondition implies $\mathtt{x} = 1$, so we obtain the required

$$\{\ \mathtt{n} = n \in \mathbb{N} \wedge P(n)\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = 1\ \}$$

by the rule of consequence.

Solution 54.
We provide a suitable program for each part.

(a) If P is semidecidable we may not in general assume that the same is true for $\neg P$. The program from the proof of Proposition 17 does not work since we are not able to guarantee that S terminates if $P(n)$ does not hold.

It may be possible that every program that witnesses the fact that P is semidecidable fails to terminate for at least one input for which P does not hold, so it is not possible to take that program and use its output to produce a program that solves the given problem.

(b) Assume that S and T are programs that respectively witness that P and Q are semidecidable. For a given input we have to ensure that we build a program that terminates with the correct answer if both P and Q holds for that input.

For this purpose we may re-use the program that is given in the solution for part (a) of Exercise 51, which we repeat here for convenience.

```
m:=n;
S;
if x=1 then
   n:=m; T
else
   skip;
```

We may now provide a total correctness proof. We use I to refer to the if statement and R for the entire program.

1. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P(n) \wedge Q(n)\ \}\ \mathtt{m} := \mathtt{n}\ \{\ \Downarrow\ \mathtt{m} = \mathtt{n} = n \in \mathbb{N} \wedge P(n) \wedge Q(n)\ \}$ by the := rule.
2. $\{\ \mathtt{m} = \mathtt{n} = n \in \mathbb{N} \wedge P(n) \wedge Q(n)\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = 1 \wedge \mathtt{m} = n \in \mathbb{N}\ \}$ by the assumption about S and since m is not mentioned in S.
3. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge P(n) \wedge Q(n) \wedge \mathtt{x} = 1\ \}\ \mathtt{n} := \mathtt{m}\ \{\ \Downarrow\ \mathtt{n} = n \in \mathbb{N} \wedge P(n) \wedge Q(n)\ \}$ by the := rule.
4. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P(n) \wedge Q(n)\ \}\ \mathtt{T}\ \{\ \Downarrow\ \mathtt{x} = 1\ \}$ by the assumption about T.
5. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge P(n) \wedge Q(n) \wedge \mathtt{x} = 1\ \}\ \mathtt{m} := \mathtt{n};\ \mathtt{T}\ \{\ \Downarrow\ \mathtt{x} = 1\ \}$ from 3 and 4 by the ; rule.
6. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge P(n) \wedge Q(n) \wedge \mathtt{x} = 1 \wedge \mathtt{x} = 0\ \}$ **skip** $\{\ \Downarrow\ \mathtt{x} = 1\ \}$ by the skip rule and the rule of consequence since the precondition is equivalent to **ff**.
7. $\{\ \mathtt{m} = n \in \mathbb{N} \wedge P(n) \wedge Q(n)\ \}\ \mathtt{I}\ \{\ \Downarrow\ \mathtt{x} = 1\ \}$ by the if rule.
8. $\{\ \mathtt{n} = n \in \mathbb{N} \wedge P(n) \wedge Q(n)\ \}\ \mathtt{R}\ \{\ \Downarrow\ \mathtt{x} = 1\ \}$ from 1, 2 and 7 by the ;= rule.

(c) We note that $P \vee Q$ holds for an input if (at least) one of the two predicates holds. Assume that we have programs S and T that witness the semidecidability of P and Q respectively.

We cannot use the program from Exercise 51 here since for any given input, S may fail to terminate while T may produce $\mathtt{x} = 1$, in which case our program would fail to terminate despite the fact that it should produce this result. Clearly swapping the order in which we run the programs does not help here.

It is, however, possible to completely rewrite the code from the two programs in such a way that one *interleaves* the two programs to ensure that if one of them terminates with the result $\mathtt{x} = 1$ then so does this program.

(d) We know that $P \to Q$ is logically equivalent to $\neg P \vee Q$, and since $\neg P$ need not be semidecidable by part (a) this predicate may also fail to be semidecidable.

Solution 55.
Solutions to both parts are as follows:

(a) For this program S we are able to show that

$$\{\ \mathtt{n} = n \in \mathbb{N} \wedge n > 100\ \}\ \mathtt{S}\ \{\ \Downarrow\ x = n\ \},$$

and for any inputs less than or equal to 100 there is nothing we are able to say about the output since it depends on the value of the variable y when the program starts. Hence the partial function computed by this program is

$$\begin{aligned} \mathbb{N} &\longrightarrow \mathbb{N} \\ x &\longmapsto \begin{cases} x & 100 < x \\ \text{undefined} & \text{else.} \end{cases} \end{aligned}$$

(b) Whatever the value of y when the program starts, by the time the if statement has been executed we have y = 0, and so this program computes the total function that maps every input to 0.

(c) For this program the value held in x upon termination depends on the value of y when the program starts, but this is not the case for all inputs n: If n = 0 the output in x is the input n, and outside of this case there's nothing we can say. We note that the loop will always terminate. Hence the best one can show about the value of the output variable x when the program terminates is

$$\{\ \mathtt{n} = 0\ \}\ \mathtt{S}\ \{\ \Downarrow\ \mathtt{x} = 0\ \}.$$

Hence the partial function computed by this program is

$$\begin{aligned} \mathbb{N} &\longrightarrow \mathbb{N} \\ x &\longmapsto \begin{cases} 0 & x = 0 \\ \text{undefined} & \text{else.} \end{cases} \end{aligned}$$

(d) Again we note that the loop always terminates. If at the start $\mathtt{n} = n \in \mathbb{N}$, the value of n after the loop terminates is $n + 2y^2 - 2 \cdot y^2 = n$, and so the partial function defined by this program is the identity function on $\mathbb{N}$, which is total.

Solution 56.
We provide a suitable program for each part.

(a) The program x := n computes the identity function since

$$\{\ \mathtt{x} = n \in \mathbb{N}\ \}\ \mathtt{x} := \mathtt{n}\ \{\ \Downarrow\ \mathtt{x} = n = \mathsf{id}_{\mathbb{N}} n\ \}.$$

If x is x_0 and n is x_1 then this is the program with index

$$1 + 4 \cdot \phi(\phi_S \mathtt{x}, \phi_A \mathtt{n}) = 4 \cdot (2^0 \cdot (2 \cdot 6 + 1) - 1) = 48,$$

so $\zeta(48, -)$ is the function we are looking for.

(b) Assuming that x = x_0, the program x := 2 which computes the given function has index

$$1 + 4 \cdot \phi(0, \phi_A 2) = 1 + 4 \cdot (2^0 \cdot (2 \cdot 10 + 1) - 1) = 1 + 4 \cdot 20 = 81,$$

so $\zeta(81, -)$ provides the function we are looking for.

(c) The program $\mathtt{x} \coloneqq \mathtt{n} + 1$ computes the given function. To compute its index we note that

$$\phi_A(\mathtt{n} + 1) = 2 + 5 \cdot \phi(\phi_A \mathtt{n}, \phi_A 1) = 2 + 5 \cdot (2^6 \cdot (2 \cdot 5 + 1) - 1) = 3517.$$

We now compute the index of our program as

$$1 + 4 \cdot \phi(0, \phi_A(\mathtt{n} + 1)) = 1 + 4 \cdot (2^0 \cdot (2 \cdot 3517 + 1) - 1) = 28137$$

Hence the function we are looking for is $\zeta(28137, -)$.

Chapter 6

Mathematical notions and results

In this chapter we provide notions and results from mathematics which are part of the taught material of this course unit, and which we make use of in the notes, in particular in Chapter 5.

There are two big topics in this chapter. We answer the question of how many functions from $\mathbb{N}$ to $\mathbb{N}$ there are and provide a rigorous argument for this claim, and we also provide mathematical infrastructure that ensures that when we talk about computing functions from $\mathbb{N}$ to $\mathbb{N}$ we do not reduce what we are able to compute.

The reason for this is that we may *encode* sets such as $\mathbb{N} \times \mathbb{N}$, $\mathbb{N}^k$ and various collections of syntactic entities as natural numbers,, and so we may, for example, look at computing functions $\mathbb{N}^m \longrightarrow \mathbb{N}^n$ by

- encoding the input as a natural number;
- computing a suitable function $\mathbb{N} \longrightarrow \mathbb{N}$ and
- decoding the output from an element to $\mathbb{N}$ to one of $\mathbb{N}^n$.

Hence considering functions $\mathbb{N} \longrightarrow \mathbb{N}$ also covers functions with different source and target sets. Also, our encoding functions are themselves *computable* (see Exercise 46 for an exercises that encourages readers to think about notions of computability for encodings), so it is possible for the encoding and decoding to be carried out by programs.

Our final concern in this section is to gain an understanding of how many programs there are.

6.1 The size of the set of functions from $\mathbb{N}$ to $\mathbb{N}$

In Section 5.1 we define the notion of a computable function from $\mathbb{N}$ to $\mathbb{N}$, and give some examples of such functions. An obvious question to ask is whether all functions of that type are computable. We provide an answer to that question in Section 5.2, and we rely on the following result.

Theorem 23. *The set* $\mathsf{Fun}(\mathbb{N}, \mathbb{N})$ *of functions from* $\mathbb{N}$ *to* $\mathbb{N}$ *is uncountable.*

The proof idea we use for this result is an example of a *diagonal argument*, first used by Georg Cantor to show that the set of real numbers is uncountable.

Proof: We know that if we wanted to show that $\mathsf{Fun}(\mathbb{N}, \mathbb{N})$ is countable we would have to produce an injection, say

$$\phi : \ \mathsf{Fun}(\mathbb{N}, \mathbb{N}) \longrightarrow \mathbb{N}.$$

This would allow us to create an ordered list of functions, where we put f before g on the list if and only if $\phi f < \phi g$. This makes sense because i is injective: given a natural number n, at most one function is mapped to it, so for every f and g in $\mathsf{Fun}(\mathbb{N}, \mathbb{N})$ we must have $\phi f < \phi g$ or $\phi g < \phi f$. We now use this list to number the functions:

- The first function on the list is f_0,
- the second function on the list is f_1,
- ...
- the nth function the list is f_n.
- ...

We know that every element $f \in \mathsf{Fun}(\mathbb{N}, \mathbb{N})$ must appear on that list and ϕf gives an upper bound to the position where it appears on the list. But such a list cannot exist:

We may define a function $g : \mathbb{N} \longrightarrow \mathbb{N}$ as follows:

$$g(n) = f_n(n) + 1.$$

This is clearly a function of the correct type. But it cannot appear on our list: It differs from every function on that list for at least one of the arguments: Given $n \in \mathbb{N}$, g is not equal to the function f_n because at input n the output of g is one bigger than the output of f_n. Hence it is not possible to obtain such a list, and that means that we cannot have an injection ϕ from $\mathsf{Fun}(\mathbb{N}, \mathbb{N})$ to $\mathbb{N}$. Hence the set $\mathsf{Fun}(\mathbb{N}, \mathbb{N})$ is uncountable. ∎

A similar idea may be used to show that the set of real numbers, or the set of numbers in the real interval $[0, 1]$ are both uncountable. The idea also applies to showing that the power set $\mathcal{P}\mathbb{N}$ of the set of natural numbers is uncountable.

Exercise 57.
Sketch a proof for each of the following statements:

(a) The set $\mathsf{Fun}(\mathbb{N}, \mathbb{B})$ is uncountable.

(b) The set $\mathsf{Fun}(\mathbb{B}, \mathbb{N})$ is countable.

If S is a countably infinite then the set of functions from S to T is also uncountable, provided T has at least two elements. We may deduce that there are uncomputable functions of that type.

We do not provide a full formal argument for the claim that $\mathsf{Fun}(S, T)$ is uncountable in this situation, but we provide a sketch:

- There's a bijection f from the natural numbers to S.
- We may use that to obtain a list of all the elements s_0, s_1, ... of S, where $s_i = fi$.
- If we had a list of all the functions from S to T we could construct a function $g : S \longrightarrow T$ that is not on the list by ensuring that $g s_i$ is different from the ith function f_i on the list at input s_i by choosing an output in T different from $f_i s_i$.

6.2 Encoding data structures as natural numbers

In this section we provide encodings of various structures as natural numbers. The underlying idea is to provide a *bijection* from the given set of structures to the natural numbers: We may then use that function to encode instances of these structures as natural numbers, and its inverse to decode a natural number into the relevant structure.

6.2.1 Integers

A bijection $\beta : \mathbb{Z} \longrightarrow \mathbb{N}$ is given by the following function:

$$\beta x = \begin{cases} 2x & x \geq 0 \\ -2x - 1 & \text{else.} \end{cases}$$

The inverse function β' for β is given by the following assignment:

$$\beta' x = \begin{cases} x/2 & x \text{ even} \\ -(x+1)/2 & \text{else.} \end{cases}$$

It is easy to verify that these two functions are each other's inverses, which establishes that they are indeed bijections.

Exercise 58.
Show that the two functions β and β' are each other's inverse function.

Note that both β and β' are computable by Exercise 46.

6.2.2 Pairs of natural numbers

Being able to compute with two inputs (and two outputs) does not allow us to do anything we cannot already do with one input and output, since we are also able to encode pairs of natural numbers as natural numbers.

Consider

$$\begin{aligned} \phi : \mathbb{N} \times \mathbb{N} &\longrightarrow \mathbb{N} \\ (m, n) &\longmapsto 2^m(2n+1) - 1. \end{aligned}$$

We claim that this function is a bijection. We provide some idea of how ϕ works via the following tables, where on the left we show $\phi(m, n)$ as a natural number in the usual decimal system, and on the right in binary.

		n			
		0	1	2	...
	0	0	2	4	...
m	1	1	5	9	...
	2	3	11	19	...
	3	7	23	39	...
	...	...	...	...	...

		n			
		0	1	2	...
		0	1	100	...
m	1	1	101	1001	...
	2	11	1011	10011	...
	3	1111	10111	100111	...
	...	...	...	...	...

This allows us to see that in binary, the function returns the second input n in binary, followed by 0, followed by m many 1s.

We show that this function is indeed a bijection, beginning with a proof of injectivity.

Assume we have (m, n) and (m', n') in $\mathbb{N} \times \mathbb{N}$ with

$$2^m(2n+1) - 1 = \phi(m, n) = \phi(m', n') = 2^{m'}(2n'+1) - 1.$$

This holds if and only if

$$2^m(2n+1) = 2^{m'}(2n'+1).$$

We have two numbers which are the product of a power of 2 and an odd number, and by the Unique Prime Factorization Theorem Fact 2 these are equal if and only if

$$2^m = 2^{m'} \qquad \text{and} \qquad 2n+1 = 2n'+1,$$

and these two pairs of numbers are equal as indicated if and only if

$$m = m' \qquad \text{and} \qquad n = n'.$$

For surjectivity, assume we have $k \in \mathbb{N}$. We consider two cases:[1]

- If $k+1$ is not divisible by 2, then there exists $n \in \mathbb{N}$ with $k+1 = 2n+1$, and

$$k = 1 \cdot (2n+1) - 1 = 2^0(2n+1) - 1 = \phi(0, n).$$

- If $k+1$ is divisible by 2 let m be the highest power of 2 that divides $k+1$. Then $k+1$ is equal to 2^m multiplied with an odd number, so there is $n \in \mathbb{N}$ so that this odd number is equal to $2n+1$, which means that

$$k = 2^m(2n+1) - 1 = \phi(m, n).$$

We give an example of recovering (m, n) from $\phi(m, n)$. Assume we want to know which pair is encoded by the number 13. If we add 1 to 13 we get 14, and the highest power of 2 that divides 14 is 2 since $14 = 2 \cdot 7$. Hence we must have $m = 1$. Further we have that $7 = 2 \cdot 3 + 1$, and so $n = 3$. We may confirm that this is correct by checking what happens when we apply ϕ to the pair $(1, 3)$:

$$\phi(1, 3) = 2^1 \cdot (2 \cdot 3 + 1) - 1 = 2 \cdot 7 - 1 = 14 - 1 = 13.$$

Exercise 59.
For the function ϕ from above describe its inverse function ϕ'. Then show that the two are indeed each other's inverses.

Note that being able to encode pairs of natural numbers as natural numbers allows us to encode tuples of natural numbers: Assume we want to encode triples of natural numbers. We may apply ϕ to the second and third components of that triple, and then apply ϕ again to pair consisting of the first component of the input triple and the result of this calculation. We may describe the resulting function as follows:

$$\mathbb{N} \times \mathbb{N} \times \mathbb{N} \xrightarrow{\mathsf{id}_{\mathbb{N}} \times \phi} \mathbb{N} \times \mathbb{N} \xrightarrow{\phi} \mathbb{N},$$

but you may find the following description easier to understand:

$$\begin{aligned} \mathbb{N} \times \mathbb{N} \times \mathbb{N} &\longrightarrow \mathbb{N} \\ (x, y, z) &\longmapsto \phi(x, \phi(y, z)). \end{aligned}$$

We claim that this function is also a bijection. The easiest way of making that argument is to point out that this function is the composite of two bijections, which is described above. We already know that ϕ is a bijection, and it is also easy to see that the function

$$\begin{aligned} \mathbb{N} \times \mathbb{N} \times \mathbb{N} &\longrightarrow \mathbb{N} \times \mathbb{N} \\ (x, y, z) &\longmapsto (x, \phi(y, z)) \end{aligned}$$

is a bijection: We may immediately recover the input x from the output, and we know that we are able to recover the inputs y and z from the output $\phi(y, z)$ since ϕ is a bijection.

We may use the same idea to encode arbitrary tuples, that is elements of $\mathbb{N}^k$, where $k \in \mathbb{N}$.

[1] It's possible to unify these to one case.

Exercise 60.
For the following natural numbers work out the tuple of natural numbers they encode.

(a) 40815 for the encoding of pairs of natural numbers,

(b) 40815 for the encoding of triples of natural numbers,

(c) 40815 for the encoding of quadruples of natural numbers,

(d) 11212 for the encoding of pairs of natural numbers.

Hint: You can check whether you have found the correct numbers by applying ϕ as appropriate.

6.2.3 More complex structures

As indicated above we are also able to encode k-tuples of integers by applying our encoding repeatedly.

To encode more interesting data structures we make use of *recursion*.

For example we may encode lists making use of the recursive definition of the structure, which for lists with natural number entries is given as follows.

base case list $[\,]$ is a list.

step case list If l is a list and $n \in \mathbb{N}$ then $n : l$ is a list.

We may use the following recursive function, making use of ϕ from above.

base case ψ $\psi[\,] = 0$.

step case ψ $\psi(n : l) = 2^n(2\psi l + 1) = \phi(n, \psi l) + 1$.

We may think of this as encoding the most recently added element of the list, and then encoding the remainder of the list.

We may use similar ideas to encode other structures, for example binary trees, but we will not provide further detail on such encodings.

6.2.4 Encoding operations

We do not only want to encode data structures, we also want to encode operations for them since we want to simulate computations on more complex structures while only performing computations on natural numbers.

For example, we have the addition operation on integers. We use the function β to encode integers as natural numbers. It is then possible to define a function $f : \mathbb{N} \times \mathbb{N} \longrightarrow \mathbb{N}$ with the property that if we apply f to the encodings of two integers m and n, it provides the natural number k with the property that the decoding $\beta' k$ of k is equal to $x + y$.

The function f for this purpose is given as follows:

$$f : \mathbb{N} \times \mathbb{N} \longrightarrow \mathbb{N}$$

$$(x, y) \longmapsto \begin{cases} x + y & x,\ y \text{ even} \\ x + y + 1 & x,\ y \text{ odd} \\ x - y - 1 & x \text{ even},\ y \text{ odd}, x \geq -y \\ x - y - 2 & x \text{ even},\ y \text{ odd}, x < -y \\ y - x - 1 & y \text{ even},\ x \text{ odd}, y \geq -x \\ y - x - 2 & y \text{ even},\ x \text{ odd}, y < -x \end{cases},$$

since

$$\beta'(f(\beta x, \beta y)) = x + y$$

in this situations. See Definition 14 for a definition that allows us to think about a function being computable if it has a source and/or target set other than $\mathbb{N}$.

Hence we are able to study addition for integers based on functions of the type $\mathbb{N} \longrightarrow \mathbb{N}$, and we may do this more generally for functions on structures that we may encode as natural numbers.

This is the reason that by restricting ourselves to studying computability for functions from $\mathbb{N}$ to itself we are not imposing restrictions on what we study.

Similar considerations apply to other encodings. These look complicated when described in detail, and we do not provide additional examples, but it is important that students understand that this is a general principle which allows us to carry out computations for various data structures in While using only natural number inputs and outputs. Below we go one step further and argue that we may encode both, While programs and states, and the small step operational semantics.

6.3 Encoding programs

In the introduction to Chapter 5 we suggest that *self-reference* is a key notion to proving important results. If we are able to encode While programs as natural numbers we are able to write programs that take (encodings of) programs as inputs. We put this idea to use in Section 5.4 and Section 5.5. As a side benefit it also provides us with the size of the set of While programs, which we make use of to show the existence of uncomputable functions in Section 5.2.

Encoding While programs as natural numbers is a more complicated task than the ones we solve above. Just as we have to look at arithmetic and boolean expressions when we want to address the operational semantics, or correctness, of While programs, we have to look at them when we want to encode programs.

We need to have an encoding of variable names as natural numbers, that is a bijection

$$\phi_V : \mathsf{Vars} \longrightarrow \mathbb{N} \ ,$$

but note that this is not the encoding we use when a variable appears as part of an arithmetic expression.

It makes sense to use the bijection ϕ_V to *number* the available variables, and so from now on we assume we have variables `x_0`, `x_1`, `x_2`,... when we look at encodings, and that for $i \in \mathbb{N}$ we have

$$\phi_V \mathtt{x_i} = i.$$

We are then able to define bijections

$$\phi_A : \ \mathrm{AExp} \longrightarrow \mathbb{N}$$

$$\phi_B : \ \mathrm{BExp} \longrightarrow \mathbb{N}$$

$$\phi_S : \ \mathrm{Stmt} \longrightarrow \mathbb{N},$$

where the functions for more sophisticated constructs rely on the functions for the others.

We recursively define the bijection

$$\phi_A : \ \mathrm{AExp} \longrightarrow \mathbb{N}$$

as follows:

$$\begin{aligned}
\phi_A n &= \quad 5 \cdot n \\
\phi_A \mathtt{x} &= 1 + 5 \cdot \phi_V \mathtt{x} \\
\phi_A(\mathtt{a} + \mathtt{a'}) &= 2 + 5 \cdot \phi(\phi_A \mathtt{a}, \phi_A \mathtt{a'}) \\
\phi_A(\mathtt{a} - \mathtt{a'}) &= 3 + 5 \cdot \phi(\phi_A \mathtt{a}, \phi_A \mathtt{a'}) \\
\phi_A(\mathtt{a} \times \mathtt{a'}) &= 4 + 5 \cdot \phi(\phi_A \mathtt{a}, \phi_A \mathtt{a'}).
\end{aligned}$$

Note that this function makes use of the bijection $\phi : \mathbb{N} \times \mathbb{N} \longrightarrow \mathbb{N}$ from Section 6.2.4. We do not formally show that this is a bijection—that would require a proof by induction over the structure of the arithmetic expression. But we can get an idea of how the inverse function works: By computing the remainder when dividing by 5 we find out what kind of expression we have (a sum, a difference, a product, a variable or a natural number), and we may then delve further into the result to extract additional details. We look into examples of this when working out which While program is encoded by a given natural number below.

We move on to defining a bijection

$$\phi_B : \text{BExp} \longrightarrow \mathbb{N}$$

by recursion over the structure of the boolean expression.

$$\begin{aligned}
\phi_B\,\mathbf{ff} &= 0 \\
\phi_B\,\mathbf{tt} &= 1 \\
\phi_B(\mathtt{a} = \mathtt{a'}) &= 2{+}4 \cdot \phi(\phi_A\mathtt{a}, \phi_A\mathtt{a'}) \\
\phi_B(\mathtt{a} \leq \mathtt{a'}) &= 3{+}4 \cdot \phi(\phi_A\mathtt{a}, \phi_A\mathtt{a'}) \\
\phi_B(\neg\ \mathtt{b}) &= 4{+}4 \cdot \phi_B\mathtt{b} \\
\phi_B(\mathtt{b} \wedge \mathtt{b'}) &= 5{+}4 \cdot \phi(\phi_B\mathtt{b}, \phi_B\mathtt{b'})
\end{aligned}$$

This finally leaves us with being able to tackle defining

$$\phi_S : \text{Stmt} \longrightarrow \mathbb{N}$$

which we do by recursion over the structure of the statement.

$$\begin{aligned}
\phi_S(\mathbf{skip}) &= 0 \\
\phi_S(\mathtt{x} := \mathtt{a}) &= 1{+}4 \cdot \phi(\phi_V\mathtt{x}, \phi_A\mathtt{a}) \\
\phi_S(\mathtt{S;S'}) &= 2{+}4 \cdot \phi(\phi_S\mathtt{S}, \phi_S\mathtt{S'}) \\
\phi_S(\mathbf{if}\ \mathtt{b}\ \mathbf{then}\ \mathtt{S}\ \mathbf{else}\ \mathtt{S'}) &= 3{+}4 \cdot \phi(\phi_B\mathtt{b}, \phi(\phi_S\mathtt{S}, \phi_S\mathtt{S'})) \\
\phi_S(\mathbf{while}\ \mathtt{b}\ \mathbf{do}\ \mathtt{S}) &= 4{+}4 \cdot \phi(\phi_B\mathtt{b}, \phi_S\mathtt{S})
\end{aligned}$$

Proving that this is a bijection requires a proof by induction, and it needs to be preceded by a proof by induction that ϕ_A and ϕ_B are also bijections, and we do not include a formal argument here.

But we give an idea of why this works: For injectivity, we may see that if two statements S and S' satisfy $\phi_A\mathtt{S} = \phi_A\mathtt{S'}$ it must be because they are both the same kind of statement since the remainder when dividing this natural number by 4 tells us what kind of statement it was. Assume we know that both programs are loops. Then it must be the case that for the two boolean expressions b and b' and the two bodies of the loop B and B' respective we have $\phi(\phi_B\mathtt{b}, \phi_S\mathtt{L}) = \phi(\phi_B\mathtt{b'}, \phi_S\mathtt{L'})$, and since ϕ is a bijection this implies that $\phi_B\mathtt{b} = \phi_B\mathtt{b'}$, and since ϕ_B is a bijection this implies $\mathtt{b} = \mathtt{b'}$, and also that $\phi_S\mathtt{L} = \phi_S\mathtt{L'}$. One may now use the induction hypothesis to conclude that $\mathtt{L} = \mathtt{L'}$ must holds, and the two together give $\mathtt{S} = \mathtt{S'}$.

Surjectivity may be shown by another proof by induction. We give an idea of how one may turn a natural number into a While program by providing an example.

We would like to know which While program is encoded by the number 42. Since this leaves the remainder 2 when divided by 4 we can see that this must be a program of the form S;S', and that $\phi(\phi_S\mathtt{S}, \phi_S\mathtt{S'}) = 10$. The number $11 = 10 + 1$ is equal to $2^0 \cdot 11$, so we know that $\phi_S\mathtt{S} = 0$ which means that $\mathtt{S} = \mathbf{skip}$ and $\phi_S\mathtt{S'} = 5$.

Since 5 leaves the remainder 1 when divided by 4 we may see that S must be an assignment, and that $\phi(\phi_V\mathtt{x}, \phi_A a) = 1$, which means that $\phi_V\mathtt{x}$ must be 1 since 2 is the highest power of 2 that divides $1 + 1$ and $\phi_A a$ must be 0, and so S' is x_1 := 0, and overall we have the program

$$\mathbf{skip}\,;\mathtt{x_1} := 0.$$

Exercise 61.
For the following numbers work out the program they encode.

(a) 111

(b) 11212

(c) 44846

Note that this allows us to do the following:

- We may think of a While program as being given a number, namely ϕ_S applied to that program. It makes sense then to talk about While program 5445. In the literature this number is sometimes called the *index* of the given program, or its Gödel number because of the parallels with Gödel's incompleteness argument.
- Given a natural number, we may work out which program is described by that natural number. We use that idea in Sections 5.5 and 5.4.

As a side benefit by providing the above bijection we have established the following by providing a bijection between the set of While programs and the set of natural numbers.

Proposition 24. *There are countably infinite many* While *programs.*

Indeed, something stronger is true: We are able to compute the function ϕ_S that assigns an index to each While program—we just aren't able to do that in the While language. The reason for this is that this requires a language that accepts strings as inputs. But given such a language, we may implement the various components of our encoding.

6.4 Computing the stepwise operational semantics

As pointed out above, by encoding While programs as natural numbers we may use programs as inputs for While programs. We want to do more, however: We want to be able to *compute the behaviour* of a While program via another While program.

In order to compute the small steps operational behaviour of a While program we need to encode *states* as well as programs.

Recall from Definition 1 that a state is an assignment from variables to numbers such that only finitely many variables hold a value distinct from 0. This means that for every state σ there is a largest $i \in \mathbb{N}$ such that σ`x_i` $\neq 0$. This allows us to encode a state where all variables have natural number assignments as a *list of natural numbers*: Given such a list we read it as telling us the values of the variables `x_0`, `x_1`, `x_2`, ... in that order and that when the list is exhausted, all variables with a higher index have the value 0.

This representation has an issue, however: there are several lists that encode the same state. For example, the list $[0, 0]$ encodes the same state as the empty list, and the list $[0, 1, 0]$ encodes the same state as $[0, 1]$ (or $[1, 0]$, depending in which order we consider the entries). But this is not a significant issue since we are able to define the update of a state independent of which list presents:

We are able to perform updates on states in the following manner: Assume we want to update a given state by setting the value of the variable `x_j` to n, and assume we have a list that represents this state. We want to describe a list that represents the updated state. We go through the entries in the list until one of the following:

- We hit the entry corresponding to the variable `x_j`, that is, the $(j+1)$th entry, and we replace the number found there with n.

- We run out of entries before we have reached the $(j+1)$th entry. In this situation we need to add 0s to the list until the list has j many entries, and we then add the value n to the list.

That means we are able to encode a state via the bijection introduced for lists in Section 6.2.3. This then allows us to encode a pair consisting of a While program S and a state σ as

$$\phi(\phi_S \mathrm{S}, \psi\sigma).$$

It is now possible to write a While program that takes as its input (the encoding of) a While program and a state, and which computes the small step operational semantics for that program in that state. Writing such a program would be a big task, but what is important here is that the reader can see that it can be done.

This idea is put to use in Section 5.5.

6.5 Summary

In this section we provide a proof that there are uncountably many functions from $\mathbb{N}$ to $\mathbb{N}$. We then turn our attention to using natural numbers to encode structures such as pair, or tuples of natural numbers, lists of natural numbers, While programs, and states. We further argue that functions between such structures may be translated to functions of encodings, which makes them functions from $\mathbb{N}$ to $\mathbb{N}$. Hence when we study which functions of that type are computable in Section 5.1, our considerations apply to a collection of functions with different source and target sets. We finally argue that it is possible to write a program that works on encodings of While programs and states which implements the small step operational semantics.

6.6 Solutions to Exercises

Solution 57.
We look at the two claims separately.

(a) For the first claim we are able to adjust the argument from Theorem 23. If the given set was countable we would be able to find a function, say γ, from $\mathsf{Fun}(\mathbb{N}, \mathbb{B})$ to $\mathbb{N}$, and we would once again be able to use that to construct a list of functions whose entries we may call f_0, f_1, f_2, and so on. On our list a function f appears before a function f' if and only if $\gamma f < \gamma f'$.

But once again we are able to construct a function $g : \mathbb{N} \longrightarrow \mathbb{B}$ that is distinct from all the f_i, by setting

$$gn = \neg(f_n n).$$

If we compare g to a function f_i from the list, then for input i they produce a different result. Hence g is different from all the functions on the list, and that means the list did not contain all functions from $\mathsf{Fun}(\mathbb{N}, \mathbb{B})$. Hence an injective function γ of the given type cannot exist, and so this set of functions is also uncountable.

(b) We may think of a function $f : \mathbb{B} \longrightarrow \mathbb{N}$ as being encoded by two natural numbers, namely $f\mathbf{tt}$ and $f\mathbf{ff}$. This defines a bijection

$$\theta : \ \mathsf{Fun}(\mathbb{B}, \mathbb{N}) \longrightarrow \mathbb{N} \times \mathbb{N}.$$

Given a function $f : \mathbb{B} \longrightarrow \mathbb{N}$ we set $\theta f = (f\mathbf{tt}, f\mathbf{ff})$. The inverse function θ' takes as its input a pair of natural numbers (m, n) and returns the function $g : \mathbb{B} \longrightarrow \mathbb{N}$ given by

$$gx = \begin{cases} m & x = \mathbf{tt} \\ n & \text{else.} \end{cases}$$

It is easy to show that θ and θ' are indeed mutually inverse. In Section 6.2.2 we give a bijection (and so in particular an injection) ϕ from $\mathbb{N} \times \mathbb{N}$ to $\mathbb{N}$, and the composite of θ and ϕ provides an injection from $\mathsf{Fun}(\mathbb{B}, \mathbb{N})$ to $\mathbb{N}$, which shows that this set is countable.

An alternative injection one could use here is to map a function $g : \mathbb{B} \longrightarrow \mathbb{N}$ directly to $\mathbb{N}$ by using $2^{g\mathbf{tt}} \cdot 3^{g\mathbf{ff}}$. Other options are available.

Solution 58.
We have to show that composing the two given functions in either order gives the identity function on the appropriate set.

If n is an integer, then if $n \geq 0$ we have

$$\beta'(\beta n) = \beta'(2n) = (2n)/2 = n,$$

and if $n < 0$ then

$$\beta'(\beta n) = \beta'(-2n - 1) = -(-2n - 1 + 1)/2 = -(-2n/2) = n.$$

On the other hand if n is a natural number then if n is even we have that

$$\beta(\beta' n) = \beta(n/2) = 2 \cdot (n/2) = n,$$

while when n is odd we have

$$\beta(\beta' n) = \beta(-2(n+1)/2) = -2 \cdot (-(n+1)/2) - 1 = -1 \cdot -(n+1) - 1 = n + 1 - 1 = n.$$

Solution 59.
In Exercise 9 we have looked at a program that computes ϕ'. Let k be a natural number. By the prime factorization theorem there exists a unique $m \in \mathbb{N}$ such that

$$k + 1 = 2^m \cdot l,$$

such that l is odd. We are now able to find $n \in \mathbb{N}$ with $l = 2n + 1$. We set $\phi' k = (m, n)$.

We have to show that composing the two given functions in either order gives the identity function on the appropriate set.

Let $(m, n) \in \mathbb{N} \times \mathbb{N}$. Then

$$\phi(m, n) = 2^m(2n + 1) - 1.$$

The definition of ϕ' says that when applying ϕ' to this number, we should add 1, which gives $2^m(2n + 1)$. Now m is indeed the highest power of 2 that divides this number and for the odd number l which remains after dividing by 2^m we have that $l = 2n + 1$, so ϕ' applied to $2^m(2n + 1) - 1$ does indeed return the pair (m, n).

If k is a natural number then we may construct (m, n) with $\phi' k = (m, n)$. If we now apply ϕ to (m, n) we get $i = 2^m(2n + 1) - 1$, and we can see that m gives us the highest power of m that divides $i + 1$, and for the odd number l that remains when we divide $i + 1$ by 2^m we have that $l = 2n + 1$. Hence $i + 1$ is equal to $k + 1$, and so $i = k$, which establishes that $\phi(\phi' k) = k$ as required.

Solution 60.
We solve each part.

(a) To find the pair (m, n) with $\phi(m, n) = 40815$ we need to solve the equation

$$2^m \cdot (2n + 1) - 1 = 40815,$$

so we have to find the highest power of 2 that divides $40815+1$, which is 4 since $40816 = 2^4 \cdot 2551$. Hence the first component of our pair is 4. For the second component, we need to subtract 1 from 2551 and divide by 2, which gives 1275, so the given number encodes the pair $(4, 1275)$.

(b) We know from the previous part that $40815 = \phi(4, 1275)$, so we know that the first component of our triple is 4. We now have to find the highest power of 2 that divides $1275 + 1$, which is 2 since $1276 = 4 \cdot 319$. Hence the second component of the triple we are looking for is 2. To find the third component we have to subtract one from 319 and divide by 2 which gives 159, so 40815 encodes the triple $(4, 2, 159)$.

(c) We know from the previous part that the quadruple we are interested in has the first two components 4 and 2. We now have to find the highest power of 2 that divides $159 + 1$ which is 5 since $160 = 2^5 \cdot 5$. Hence the third component of our quadruple is 5, and for the final one we have to subtract one from 5 and divide by 2, which gives 2. Hence the quadruple encoded is $(2, 4, 5, 2)$.

(d) We have to find the highest power of 2 that divides $11212 + 1$, which is 1 since this is an

odd number. Hence the first component of the pair we are looking for is 0 and the second component is 5606 since $11213 = 5606 \cdot 2 + 1$, so 11212 encodes the pair $(0, 5606)$.

Solution 61.

We look at each given number. We note that the shape of the required program is determined by the remainder the given number leaves when dividing by 4.

(a) When dividing 111 by 4 we get the remainder 3, so our program is an if statement. If we subtract 3 from 111 we get 108, and we need to find the pair which produces $108/4 = 27$ when ϕ is applied to it. The highest power of 2 that divides $27 + 1$ is 4 since $28 = 4 \cdot 7$, so the boolean expression encoded here is encoded by the number 2, which means it is the equality test $0 = 0$. We have that $7 = 2 \cdot 3 + 1$, and so we now need to find out which two statements S and S' we may have with the property that $\phi(\phi_S \mathtt{s}, \mathtt{S'}) = 3$. To find these statements, we note that the highest power of 2 that divides $3 + 1$ is 4, and so 3 is the encoding of the pair $(4, 0)$. That means that S is the statement **while ff do skip**, while S' is the statement **skip**, and so we have encoded the program

```
if 0=0 then
   while ff do skip;
else
   skip
```

(b) When dividing 11212 by 4 we get the remainder 0, so our program is a while loop. Subtracting 4 from 11212 and dividing by 4 we have that for the boolean expression b and the loop body B of our program it must be the case that $\phi(\phi_B \mathtt{b}, \phi_S \mathtt{B}) = 2802$. The highest power of 2 that divides $2802 + 1$ is 1, so $\phi_B \mathtt{b} = 0$, and so our boolean expression is **ff**. For the body of the loop we must have that $\phi_S \mathtt{B} = 1401$, which leaves the remainder 1 when divided by 4 and so B is an assignment. The assignment is such that for the variable x and the arithmetic expression a involved we must have that $\phi(\phi_V \mathtt{x}, \phi_A \mathtt{a}) = 350$. The highest power of 2 that divides 350+1 is 1, and so the variable we are looking for is x_0, and we are looking for the arithmetic expression a with $\phi_A \mathtt{a} = 175$. This leaves the remainder 0 when divided by 5, so this is the number 35. Hence our program is

```
while ff do
   x_0 := 35
```

(c) The number 44850 leaves the remainder 2 when divided by 4, so our program is a composite of two statements. We are looking for statements S and S' with the property that $\phi(\phi_S \mathtt{S}, \phi_S \mathtt{S'}) = 11211$. The highest power of 2 that divides $11211 + 1$ is 4, so $\phi_S \mathtt{S} = 4$, which means that S is the while loop **while ff do skip**. We must have $\phi_S \mathtt{S'} = 1401$, which leaves the remainder 1 when divided by 4, so this is again an assignment. We computed this assignment in the previous part so the program is the following.

```
while ff do skip
x_0 := 35
```

Appendix A

Mathematical background

In this chapter we provide mathematical material that is taught as part of COMP11120 *Mathematical Techniques for Computer Science* and which we make use of above. We repeat definitions and some explanations. Use the glossary for the COMP11120 notes if you want to refresh your memory regarding some of these concepts.

A.1 Numbers

We variously use the fact that integers may be presented in a particular form which supports integer division:

Fact 1. *Given x and y in $\mathbb{Z}$ with $x \neq 0$ there exist unique numbers k and l in $\mathbb{Z}$ such that*

- $0 \leq l < |x|$ *and*
- $y = kx + l$.

If we have x, y, l and k as above then

$$y \operatorname{div} x = k \qquad \text{and} \qquad y \bmod x = l.$$

We also use the fact that every element of $\mathbb{N}$ other than 0 may be uniquely decomposed into its prime factors. This is also known as the *Unique Prime Factorization Theorem*, or the *Fundamental Theorem of Arithmetic*.

Fact 2. *Let p_i be the ith prime number, and let $n \in \mathbb{N} \setminus \{0\}$. Then there exists $k \in \mathbb{N}$ and $i_j \in \mathbb{N} \setminus \{0\}$ for $1 \leq j \leq k$ such that*

$$n = p_1^{i_1} \cdot p_2^{i_2} \cdot \dots \cdot p_k^{i_k} = \prod_{j=1}^{k} p_j^{i_j},$$

and k, $i_1, \dots i_k$ are uniquely determined.

The following facts may be useful for the material in Chapter 3.

Fact 3. *Let x, x', y and y' be elements of $\mathbb{R}$. Then the following hold:*

For all x, x' in $\mathbb{R}$	*we have*	$x \leq x'$ *or* $x' \leq x$.
If $x \leq x'$ and $y \leq y'$	*then*	$x + y \leq x' + y'$.
If $x \leq x'$ and $y \geq 0$	*then*	$x \cdot y \leq x' \cdot y$.
If $x \leq x'$ and $y \leq 0$	*then*	$x \cdot y \geq x' \cdot y$.
If $x \leq y$	*then*	$-x \geq -y$.
If $x \leq y < 0$ or $0 < x \leq y$	*then*	$x^{-1} \geq y^{-1}$
If $x \geq 1$ and $y \leq y'$	*then*	$x^y \leq x^{y'}$.
If $x > 1$, $y, y' > 0$ and $y \leq y'$	*then*	$\log_x y \leq \log_x y'$.

A.2 Functions

Recall that a function

$$f : S \longrightarrow T$$

comes with

- a *source set* S, the set where the inputs live,
- a *target set* T, the set where the outputs live,
- and that to completely describe a function we need an *assignment* that describes how to turn an input into the relevant output.

Further recall that it's possible to describe a function without giving it a name, and that for example

$$\begin{aligned} \mathbb{Z} &\longrightarrow \mathbb{N} \\ x &\longmapsto x^2 \end{aligned}$$

describes a function that takes as its input an integer and returns the square of that integer as the output in the natural numbers.

We may compose functions provided that the target set of the function we apply first matches the source set of the function we apply second.

Definition 25. Given two functions

$$f : S \longrightarrow T \qquad \text{and} \qquad g : T \longrightarrow U$$

where the target of f is the source of g, the **composite of f and g**

$$\begin{aligned} g \circ f : S &\longrightarrow U \\ a &\longmapsto g(fa), \end{aligned}$$

is defined by first applying f to s and then g to the result, that is

$$\begin{aligned} s &\longmapsto fs \longmapsto gfs \\ S &\longrightarrow T \longrightarrow U, \end{aligned}$$

and so overall we map $s \in S$ to $gfs \in U$.

Basic properties of functions that we need are the following:

Definition 26. A function $f : S \longrightarrow T$ is **injective** if and only if

$$\text{for all } s \text{ and } s' \text{ in } S \qquad fs = fs' \qquad \text{implies} \qquad s = s'.$$

Under these circumstances we say that f is an **injection**.

Note that some people call injective functions 'one-on-one' instead.

Definition 27 (surjective)**.** A function $f : S \longrightarrow T$ is **surjective** if and only if

$$\text{for all } t \in T \qquad \text{there exists } s \in S \qquad \text{with} \qquad t = fs.$$

We also say in this case that f is a **surjection**.

In other words a function is surjective if its range is the whole target set, or, to put it differently, if its image reaches all of the target set. Readers may have heard 'onto' for a function that is surjective.

Definition 28. A function $f : S \longrightarrow T$ is **bijective** if and only if it is both, injective and surjective. We say in this case that it is a **bijection**.

Bijective functions are ones which we may undo, see Theorem 30, leading to another concept:

Definition 29. A function $g : T \longrightarrow S$ is the **inverse function** of $f : S \longrightarrow T$ if and only if

$$g \circ f = \mathsf{id}_S \qquad \text{and} \qquad f \circ g = \mathsf{id}_T.$$

Theorem 30. *A function $f : S \longrightarrow T$ is a bijection if and only if it has an inverse function.*

We sometimes need to talk about the set of all functions from a given set S to another, say T, and we use

$$\mathsf{Fun}(S, T)$$

for this purpose.

A.3 Partial functions

When we look at what may be computed by While programs in Chapter 5 we would like to think of such programs as computing functions. However, While programs may fail to terminate, in which case there is no output produced, so in general we should think of programs as computing *partial functions*.

Definition 31. Let S and T be sets. A **partial function from S to T** is an assignment where

$$\text{for every } s \in S \qquad \text{there is at most one } t \in T \qquad \text{with} \quad s \longmapsto t.$$

This means that when we apply a partial function f from S to T to an input $s \in S$, two things may happen: fs is undefined, or fs is defined and is an element of T.

Note that every function from S to T is also a partial function between those two sets. In a situation where partial functions and functions are under consideration, we sometimes refer to *total functions* to underline that they are defined for all their inputs.

We use a different kind of arrow, one with only 'half a tip'

$$f : S \rightharpoonup T$$

to indicate that the function described is partial.

The **domain of definition of a partial function** $f : S \rightharpoonup T$ is defined as

$$\mathsf{dom}\, f = \{s \in S \mid f\, s \text{ is defined}\}.$$

It is possible to compose compatible partial functions:

The **composite of partial functions** $f : S \rightharpoonup T$ **and** $g : T \rightharpoonup U$ is defined as:

$$(g \circ f)s = \begin{cases} g(f\, s) & f\, s \text{ and } g(f\, s) \text{ are both defined} \\ \bot & \text{else.} \end{cases}$$

We sometimes need to talk about the set of all partial functions from a given set S to another, say T, and we use

$$\mathsf{PFun}(S, T)$$

for this purpose.

A.4 Binary relations

We use relations to describe the operational semantics of While, and we provide some relevant concepts here.

A **relation** R **from a set** S **to a set** T is given by a subset of $S \times T$. This is sometimes written as

$$R : S \longrightarrow\!\!\!\!\!\!\!+\; T,$$

but this is not universal.

A **binary relation** is a relation where the two sets S and T agree.

Relations are a little like functions in that one can define their composition.

Let R be a relation from S to S', and let R' be a relation from S' to S''. The **relational composite** $R\,;R'$ is given by

$$R\,;R' = \{(s, s'') \in S \times S'' \mid \exists s' \in S'.((s, s') \in R \text{ and } (s', s'') \in R')\}.$$

For the transition relation of the small step operational semantics we are concerned with the reflexive transitive closure, and we provide here relevant definitions.

Definition 32 (reflexive)**.** A binary relation R on a set S is **reflexive** if and only if it is the case that we have

$$\text{for all } s \text{ in } S \qquad (s, s) \in R.$$

Given a binary relation R on S the **reflexive closure of** R is given by

$$R \cup I_S = R \cup \{(s, s) \mid s \in S\}.$$

When we look at $O(-)$ in Chapter 3 we ask ourselves whether certain relations are symmetric, or anti-symmetric.

Definition 33 (symmetric)**.** A binary relation R on a set S is **symmetric** if and only if we have

$$\text{for all } s, s' \in S \qquad (s, s') \in R \qquad \text{implies} \qquad (s', s) \in R.$$

Definition 34 (antisymmetric)**.** A binary relation R on a set S is **anti-symmetric** if for all elements s and s' from S we have that

$$(s, s') \in R \text{ and } (s', s) \in R \qquad \text{implies} \qquad s = s'.$$

Definition 35 (transitive)**.** A binary relation R on a set S is **transitive** if and only if we have that

$$\text{for all } s, s', s'' \in S$$
$$(s, s') \in R \text{ and } (s', s'') \in R \qquad \text{implies} \qquad (s, s'') \in R.$$

We may define the **transitive closure of a binary relation R on a set S** as adding all those pairs

$$(s_1, s_n) \qquad \text{to} \qquad R$$

for which we can find a list of elements

$$s_1,\ s_2,\ \ldots,\ s_n \text{ in } S, \qquad \text{with } n \geq 2,$$

such that

$$\text{for all } 1 \leq i \leq n-1 \qquad \text{we have} \qquad (s_i, s_{i+1}) \in R.$$

Alternatively we may use the following recursive definition.

Base case. $R^0 = I_S$.

Step case $R^{n+1} = R^n \,;\, R$.

Then the transitive closure of R is given by

$$\bigcup_{n \in \mathbb{N} \setminus \{0\}} R^n,$$

and the reflexive transitive closure of R is given by

$$\bigcup_{n \in \mathbb{N}} R^n$$

A.5 Countable and uncountable sets

We are able to measure the sizes of sets relative to each other by using the following definition.

Definition 36. Let S and T be sets. We say that the **size of a set of S is smaller than or equal to that of T** if and only if there is an injection from S to T.

This defines a reflexive transitive relation on sets.
We distinguish

- countably infinite and
- uncountable sets.

This helps us make an argument that there are functions that cannot be computed.

Proposition 37. *A set S is infinite if and only if there is an injective function from S to itself which is not surjective.*

Proposition 38. *The following properties hold:*

(i) If S is an infinite set then there is an injection from $\mathbb{N}$ to S.

(ii) If there is an injection from of $\mathbb{N}$ to a set S then S is infinite.

Definition 39. A set is **countable** if and only if there is an injection from it to the natural numbers. A set is **uncountable** if and only if there is no injection from it to the natural numbers. A set is **countably infinite** if it is both, countable and infinite.

Note that every finite set is countable. The notion of the size of a set from above works with these definitions in that if we have a set that has at least the size of a set that is at least countably infinite then this set is also infinite, and a set that has at least the size of an uncountable set is uncountable.

Examples of countably infinite sets are:

- The set of natural numbers $\mathbb{N}$.
- The set of integers $\mathbb{Z}$.
- The set of rational numbers $\mathbb{Q}$.
- The set of finite subsets of $\mathbb{N}$, $\mathcal{P}_f\mathbb{N}$.
- The set of all programs in your favourite programming language.
- The set of all strings over a finite alphabet.

Examples of uncountable sets are:

- The set of real numbers $\mathbb{R}$, and any real interval whose upper and lower bounds are distinct,
- the set of complex numbers $\mathbb{C}$,
- the set of all subsets of $\mathbb{N}$, $\mathcal{P}\mathbb{N}$,
- the set of all functions from $\mathbb{N}$ to $\mathbb{N}$.